International Journal of Information Security (2025) 24:78
https://doi.org/10.1007/s10207-025-00992-7

REGULAR CONTRIBUTION

Large language models for software vulnerability detection: a guide for researchers on models, methods, techniques, datasets, and metrics

Seyed Mohammad Taghavi Far[1] · Farid Feyzi[1]

Accepted: 29 January 2025 / Published online: 14 February 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract
Large language models (LLMs) have emerged as transformative tools in the domain of software vulnerability detection and management, offering sophisticated capabilities in identifying, analyzing, and mitigating security risks. This article delves into the utilization of LLMs, examining their role in revolutionizing traditional approaches to software vulnerability detection. We explore the various categories of LLMs, such as bidirectional encoder representations from transformers (BERT) and generative pre-trained transformer (GPT), and how these models are being leveraged to improve the accuracy and efficiency of vulnerability detection. This article reviews how LLMs are being integrated into existing software security frameworks, synthesizing research findings on their performance in various contexts. It includes insights into how LLM-based methods complement traditional techniques like static analysis and fuzz testing, without engaging in a direct comparative analysis of these approaches. The comparison highlights the strengths of LLMs, such as their ability to generalize across diverse codebases and programming languages, while also addressing their limitations, such as susceptibility to biases from training data and the hallucination. The article synthesizes findings from recent research, showcasing how LLMs have been successfully employed to detect a range of vulnerabilities, from buffer overflows to SQL injections, and outlines how these models enhance productivity by automating the detection and reporting of security flaws. Additionally, we discuss the inherent challenges in applying LLMs to software vulnerability detection, such as the need for high-quality datasets, and the ethical implications related to the deployment of LLM-based systems in security-critical applications. Addressing these challenges is crucial for the future advancement of LLM technologies in the cybersecurity domain. A comprehensive introduction to foundational and specialized datasets is provided, including datasets such as CVEfixes, Big-Vul, and LineVul, which are tailored for software vulnerability detection. These datasets serve as crucial resources for training and benchmarking LLMs. Moreover, we introduce evaluation metrics such as F1-score, precision, recall, and AUC-ROC that are used to assess the performance of models in detecting and mitigating vulnerabilities, offering a structured way to gauge the success and limitations of LLMs. In addition, the article explores fine-tuning techniques such as full fine-tuning, feature extraction, adapter-based fine-tuning, and LoRA (low-rank adaptation), highlighting how each method can enhance LLM performance in vulnerability detection. By focusing on parameter-efficient fine-tuning approaches, such as adapter layers and prefix-tuning, and LoRa, we outline ways to optimize model performance while reducing computational overhead. By providing a comprehensive review of the literature and practical insights into LLM integration, this article aims to fill the gap in existing research and serve as a foundational guide for future investigations. Researchers and practitioners in the field of software security will benefit from the comparative analyses, detailed case studies, and strategic recommendations provided herein, which collectively highlight the potential of LLMs to complement and enhance traditional software vulnerability detection techniques.

Keywords Security and privacy · Software and application security · Software security engineering · Computing methodologies · Machine learning · Machine learning approaches · Large language models (LLMs) · Software vulnerabilities · Software vulnerability detection · Software vulnerability datasets · Vulnerability detection metrics · Fine-tuning techniques

✉ Farid Feyzi
feizi@guilan.ac.ir

Seyed Mohammad Taghavi Far
m_taghavifar@webmail.guilan.ac.ir

[1] Computer Engineering, University of Guilan, Rasht, Iran

Abbreviations

AI	Artificial intelligence
API	Application programming interface
APR	Automatic program repair
BERT	Bidirectional encoder representations from transformers
BLEU	Bilingual evaluation understudy
BMC	Bounded model checker
CAPEC	The common attack pattern enumerations and classification
CVE	Common vulnerabilities and exposures
CWE	Common weakness enumeration
DAST	Dynamic application security testing
DiCE	Diverse counterfactual explanations
DL	Deep learning
GNNs	Graph neural networks
IAST	Interactive application security testing
LLM	Large language model
LLMCs	Large language models of code
LSTM	Long short-term memory
MDR	MacroData refinement
MRR	Mean reciprocal rank
MTL	Multi-task learning
NIST	National Institute of Standards and Technology
NLP	Natural language processing
NMT	Neural machine translation
NVD	National vulnerability database
PDG	Program dependency graph
PEFT	Parameter efficient fine-tuning
PII	Personally identifiable information
RLHF	Reinforcement learning from human feedback
SARD	Software assurance reference dataset
SAST	Static application security testing
SDLC	Software Development LifeCycle
SE	Software engineering
SHAP	SHapley Additive exPlanations
SQL	Structured query language
SuperGLUE	Super general language understanding evaluation

1 Introduction

The presence of implementation bugs within software applications can manifest as program crashes, data corruption, performance decline, and exploitable security vulnerabilities. This underscores the criticality of early bug identification and rectification. Established methodologies, such as static analysis, possess limitations in their ability to accurately detect faults, and recent forays into Deep Learning (DL)-based approaches may not comprehensively capture the intricacies of software code. While static analysis and DL techniques offer demonstrable value in the initial stages of bug detection, they are prone to generating false positives and other challenges (Data Requirement, Interpretability, Feature Selection). These false detections can negatively impact developer productivity [1–3].

The recent breakthroughs in DL have garnered significant attention within the SE domain, prompting exploration of their potential to resolve longstanding issues [4, 5]. Several approaches have been proposed to leverage DL techniques in addressing program defects. Some prominent examples include DLFix [6] and DeepRepair [7], which utilize DL to directly repair these defects. CURE [8] employs a DL model trained on a corpus of source code to parse, analyze, and model code structure. DEAR [9] takes a hybrid approach, combining spectrum-based fault localization with DL to enhance contextual understanding of the code. It is important to acknowledge that each of these methods possesses its own strengths and weaknesses, necessitating further research to determine the most effective strategies for DL-based program defect mitigation.

However, the rise of Large Language Models (LLMs), powered by revolutionary transformers architecture, has significantly impacted the landscape of Natural Language Processing (NLP), ushering in a new era of capabilities and applications, resulting in unprecedented advancements [10–12]. Recent studies have demonstrated the effectiveness of LLMs in detecting software vulnerabilities, sometimes surpassing DL and traditional methods [12].

1.1 Problem statement (challenges)

As software vulnerabilities become increasingly sophisticated and exploitable, concerns grow about organizations' capacity to prevent data breaches and individuals' ability to safeguard their digital assets. The 2023 Verizon Cost of Data Breach Report reveals that companies typically require 197 days to identify a security breach and an additional 69 days to contain it, emphasizing the need for swift and effective incident response strategies [13, 14]. Extending the incident response duration exposes companies to significant financial and operational setbacks, including unplanned outages and decreased productivity. The need for computers to handle and interpret vast volumes of language data, especially in natural language interactions and software vulnerability management tasks like identifying flaws in software code, has become paramount [13, 15].

Conventional methods for detecting vulnerabilities, which hinge on human experts defining weaknesses, are slow and

laborious, therefore, there is an ever-increasing need for automated tools to detect and handle software vulnerabilities. So, we decided to embark on an endeavor to develop a novel method for leveraging LLMs in detecting and fixing software vulnerabilities. However, the absence of a comprehensive handbook or established guidelines for this innovative path posed significant challenges. This raised numerous questions, not only regarding the application of LLMs and potential methodologies, but also concerning the necessary and available resources. The research questions presented here served as foundational inquiries necessary to initiate the development of our proprietary methodologies, and answering them marked the beginning of our work. The objective of this review is to systematically address these questions, providing a valuable resource for researchers interested in pursuing this area of study. We aspire for this article to be as beneficial to future researchers as it has been to us.

To provide a comprehensive overview of the LLMs use in the field of software vulnerabilities, it is important to fully comprehend how these models are currently being applied, the challenges they face, and their potential. Thus, we aim to provide a systematic literature review of the application of LLMs in this fields. This study thus aims to answer the following research questions:

1.2 Research questions

- RQ1: Why should we use LLMs for software vulnerabilities detection?
- RQ2: What roles can LLMs play in software vulnerability detection and mitigation?
- RQ3: What specific types of LLMs have been utilized in software vulnerabilities detection?
- RQ4: In comparison to traditional methods and tools, how do LLMs perform in detecting and handling software vulnerabilities?
- RQ5: In what ways have LLMs been applied to the detection of software vulnerabilities?
- RQ6: How does fine-tuning LLMs affect their performance in software vulnerability detection and repair?
- RQ7: What techniques are used to fine-tune LLMs for software vulnerability detection?
- RQ8: What datasets are used to train, fine-tune, or test LLMs for software vulnerability purposes?
- RQ9. What metrics are used to assess LLMs in addressing software vulnerabilities?
- RQ10: What are the challenges of using LLMs in software vulnerability detection?
- RQ11. How can we enhance LLMs effectiveness in software vulnerability detection?

1.3 Research goals

By answering the proposed questions, we seek to achieve the following goals:

- Establish a Foundational Understanding: To offer a clear introduction to LLMs, their various architectures, and their potential applications within software security domains.
- Analyze Existing Research: To conduct a review of existing literature on LLM applications in software vulnerability detection. This analysis will encompass the exploration of datasets employed, frameworks developed, and the effectiveness of these approaches as measured by established metrics.
- Identify Research Gaps and Opportunities: By critically evaluating existing research, we aim to identify areas where current methodologies face limitations. This will highlight promising avenues for future research efforts to contribute to the advancement of LLM-based solutions in vulnerability management.
- Provide Guidelines for Future Research: To equip researchers with practical insights and recommendations for effectively utilizing LLMs in their investigations of software vulnerabilities management. This includes considerations for dataset selection, framework development, and performance evaluation.

By achieving these goals, this paper intends to serve as a valuable resource for researchers embarking on investigations in this exciting and dynamic field. We aim to foster a deeper understanding of LLM capabilities in vulnerability management and encourage the development of novel and more effective solutions for a safer digital landscape.

1.4 Overview

In the forthcoming sections, we begin by detailing the methodology used for source material collection and analysis. This is followed by a comprehensive background section that provides an in-depth overview of LLMs, their categories, and their applications in the domain of SE. Special emphasis will be placed on their relevance to software vulnerability detection, highlighting their strengths and potential limitations.

A comparative analysis table will then be presented to contrast LLMs with established software vulnerability detection methods. This table will elucidate the relative advantages and shortcomings of each approach, offering a clearer perspective on how LLMs can complement traditional methods. Researchers looking to integrate LLMs with existing vulnerability detection techniques will find this table particularly

beneficial. This thorough comparison is essential for advancing the field and fostering the development of more robust, hybrid approaches to software security.

Next, we will explore the challenges faced in effectively employing LLMs for software vulnerability management. These challenges include issues related to training data quality, computational resource demands, and the generalization ability of the models. Addressing these concerns will lead to a clearer identification of gaps in the current research and provide insight into the areas that need further development.

Finally, the paper will conclude with a summary of our findings and recommendations for future research. Our goal is to provide a thorough understanding of how LLMs can be integrated into software security practices, with the aim of improving both the detection and mitigation of vulnerabilities in modern software systems.

2 Methodology

To comprehensively explore the landscape of using LLMs in the detection and handling of software vulnerabilities, a systematic literature review was conducted. The following steps outline the methodological approach employed:

2.1 Identification of keywords

A two-person research team conducted this study. The initial research phase involved a Google Scholar search utilizing two keyword sets: “Large Language Models” combined with either “Software Engineering” or “Software Security”. The resulting search yielded articles [16] and [17] as the primary sources, from which subsequent keywords were extracted.

The following keywords were extracted from the two main articles and used to identify subsequent articles and guide the search process: Bidirectional Encoder Representations from Transformers (BERT), FalconLLM, Pre-trained model, Cyber Threat Detection, Software Fix, Security Testing, Penetration Testing, Software Vulnerability Detection, Code Languages, Large language of code, Pre-trained Transformers, NLP, ChatGPT, GPT-4, Bard, Code Generation, Vulnerability Discovery, Transformer, Overfitting, Prompt, and Code Analysis.

The arXiv, IEEE, Science Direct, and ACM Digital Library were selected as the primary sources for the next phase of the search. Our final search across the four mentioned databases yielded 59 articles. The search results were meticulously reviewed by both researchers, with a focus solely on titles, abstracts, and keywords to identify articles aligned with the research objectives. From 59 articles, 51 were selected, and 8 were excluded.

The exclusion of the 8 papers was based on their misalignment with the research scope and objectives. Some focused on general machine learning or NLP tasks unrelated to software vulnerability detection or software engineering, while others presented purely theoretical frameworks without practical applications or relevant empirical evidence. Additionally, some were too specific and advanced, addressing niche topics beyond the focus of this study. This careful screening ensured that only the most relevant studies contributing to the understanding of LLMs in software security and engineering were included in the final selection.

2.2 Review of the initial search results

Subsequent to the initial screening, the remaining articles underwent a rigorous evaluation by both researchers to determine final inclusion. A double review process was employed to reduce bias and improve the reliability of our review. This involved having each paper reviewed independently by both researchers to ensure consistency and accuracy. The criteria mentioned below were applied to each article. Adherence to even a single exclusion criterion resulted in the article’s elimination from further consideration. Upon conclusion of this phase, a total of forty articles were selected for further analysis.

Inclusion criteria:

- Papers incorporating an LLM within their framework for software vulnerability detection.
- Papers exploring the utilization of LLMs for the task of program repair.
- Papers exploring the various use cases of LLMs, such as code generation, code completion, debugging, development, testing, and maintenance.
- Papers that include comprehensive descriptions of the methodology and implementation details related to the use of LLMs for software vulnerability detection and handling. Specifically, these descriptions should cover:
  - The architecture of the LLM used, including model size, pre-training data, and fine-tuning processes.
  - The specific datasets or benchmarks utilized for evaluating the LLM’s effectiveness in vulnerability detection and handling.
  - A clear explanation of the metrics or criteria used to assess the performance of the LLM in this context.
  - Any novel techniques or adaptations applied to the LLM to enhance its suitability for software vulnerability tasks.
- Papers offering a literature review on any of the two primary keyword sets ("Large Language Models" combined with either "Software Engineering" or "Software Security"). A "comprehensive" review is defined as one that:
  - Covers a wide range of relevant studies, including both foundational and recent works.

  - Discusses the evolution of research in the area, identifying key trends, gaps, and challenges.
  - Provides a critical analysis of different approaches, methodologies, and technologies used.
- Papers including supplementary materials such as datasets or codes. While the inclusion of supplementary materials (e.g., datasets, code repositories, or detailed appendices) is highly valued, it is not mandatory. Papers without these materials will still be considered if they provide substantial theoretical, methodological, or empirical contributions relevant to the use of LLMs in software vulnerability mitigation frameworks.
- Papers that have been referenced in the selected papers and offer significant insights or data relevant to our research. "Additional information" refers to:
  - New methodologies, frameworks, or approaches not covered in the selected paper.
  - Further experimental results or case studies that complement the findings of the selected paper.
  - Comparative analyses or critiques that provide a different perspective on the selected paper's topic.

Exclusion criteria:

- Non-academic or non-professional sources, such as blog posts or promotional materials, unless they provide verifiable and unique insights that are not available in academic literature. "Unique insights" must be:
  - Based on insider knowledge, such as details from industry practitioners, that directly relate to LLMs and software vulnerability detection.
  - Supported by empirical evidence, data, or case studies.
  - Unavailable in other academic or peer-reviewed sources.
- Sources that do not directly contribute to addressing the research questions or objectives.
- Eliminating sources with insufficient or unclear information relevant to the study. "Insufficient" information refers to papers that lack the necessary detail in their methodology, results, or analysis to support their conclusions. "Unclear" information refers to papers where the data, methods, or arguments are presented in a way that is ambiguous, not replicable, or poorly explained, making it difficult to assess their relevance or validity for the research on LLMs and software vulnerability detection.
- Excluding articles not published in the English language.
- The complete text of the paper needs to be accessible or the paper will be excluded.

2.3 Snowballing technique

We initiated data extraction from the selected articles and employed a snowballing technique to expand the dataset. By systematically examining the reference lists of these initial articles, we identified and incorporated 68 additional relevant studies.

2.4 Critical review

Selected articles underwent a critical review, with a focus on extracting key insights, methodologies, and findings. The quality of the research was assessed to ensure the inclusion of robust and relevant contributions. Selected articles underwent a rigorous critical review process, which involved multiple stages of analysis. Initially, articles were evaluated for their relevance and alignment with the research objectives. Following this, each article was meticulously analyzed to extract key insights, methodologies, and findings. This process included a thorough examination of the research design, and analytical approaches used by the authors. This comprehensive evaluation ensured that only high-quality and pertinent studies were included, thereby enhancing the reliability and credibility of the review.

2.5 Synthesis and integration

A comprehensive synthesis of information gleaned from the selected articles was conducted to identify overarching themes, recurring patterns, and knowledge gaps within the existing literature. These findings served as the foundational framework for subsequent sections of the research paper.

2.6 Use of artificial intelligence (AI) assistance

To expedite the identification of relevant articles for unanswered questions, we employed Microsoft Copilot, ChatGPT 4.0, and Gemini. We formulated the following prompt to query databases like arXiv or ACM:

> "Please give a list of articles from [database] that can help answer the following question. The question is: …"

This method allowed us to leverage the capabilities of AI and the extensive database of arXiv.org to fill in any gaps in our research.

2.7 Citation and referencing

The selected articles were cited. Ensuring proper acknowledgment of the sources and inclusion in the reference section of the paper.

2.8 Potential biases in the review process

To ensure a comprehensive and unbiased review, we did not limit our selection to papers that reported only positive outcomes of using LLMs in software vulnerability detection and handling. By including studies with varied results, we aimed to present a balanced view of the current state of research in this field.

3 Background

3.1 Introduction to large language models

What is an LLM? LLMs are a subset of Pre-trained Language Models (PLMs) that are trained on a large corpus of text data. Examples of LLMs include models like GPT4 [18], BERT [19], FalconLLM series [20, 21], and others. They have demonstrated remarkable capabilities across various NLP tasks and stand as the current state-of-the-art in AI models, constructed upon neural network architectures, particularly DL [19, 22]. The term "LLM" is used to distinguish language models based on their size, specifically referring to large-sized PLMs. However, there isn't a formal consensus on the minimum parameter scale for LLMs, as the model's capacity is intertwined with both data size and total computing resources [16, 23, 24].

Celebrated for their extensive training datasets, intricate neural network structures, and advanced linguistic abilities, LLMs showcase remarkable proficiency in mimicking, generating, and fine-tuning natural language. Developers employ a comprehensive body of texts to train LLMs, covering diverse linguistic styles, domains, and subjects. This training process enables LLMs to cultivate a statistical grasp of language, producing coherent, context-aware text and excelling in various NLP tasks. The substantial neural networks within LLMs, comprising tens or hundreds of billions of parameters, facilitate the discernment of subtle semantic links and effective generalization across diverse language tasks. The scale and architecture of LLMs augment their capacity to encompass vast linguistic knowledge, showcasing a form of artificial creativity reflective of their training data [25].

Additionally, LLMs are finding applications in various real-world tasks such as machine translation, content creation, chatbots, and factual topic summarization. While these models offer impressive capabilities, it's important to be aware of potential biases inherited from their training data and the need for responsible development. The field of LLM research is rapidly evolving, with ongoing efforts to improve their capabilities, address limitations, and explore new applications [25].

3.2 LLMs categorization

The most powerful LLMs today are built on the Transformer architecture, which was introduced in 2017 [26]. Depending on the specific application, Transformer models can be configured as encoder-only (e.g., BERT), decoder-only (e.g., GPT), or encoder-decoder networks (e.g., T5). Additionally, sparse models represent a recent innovation, enabling more efficient processing of very large sequences [10, 16, 27].

3.2.1 Encoder-only

Encoder-only models are adept at understanding and summarizing information from input sequences. They don't generate new sequences but produce fixed-size representations suitable for tasks like classification. Despite their effectiveness in tasks requiring comprehensive understanding, like SoC security verification, their inability to generate outputs based on the learned context limits their application in scenarios demanding further generation [10, 23, 28–30].

LLMs, particularly encoder-only types like BERT and its variants, focus on encoding input sentences to capture word relationships and contextual information [16, 19, 23, 29].

BERT, based on transformer's encoder architecture, is known for its bidirectional attention mechanism that considers both left and right context during training. Specialized models such as CodeBERT, GraphCodeBERT, RoBERTa, ALBERT, BERTOverflow, and CodeRetriever have tailored features for Software Engineering (SE) tasks, incorporating program structures, new pre-training tasks, or engaging different modalities for improved application in code-related tasks [16, 19, 23, 31].

CodeBERT improves code understanding by predicting subsequent tokens, aiding tasks like code completion, code search, bug detection, and Automatic Program Repair (APR) [16, 28, 31, 32]. GraphCodeBERT leverages the relationships between code elements by representing them as a graph, which enhances its performance in tasks such as code summarization, vulnerability prediction, and program analysis. These models can be particularly effective in understanding and processing tasks like code review, bug report analysis, and identifying entities within code structures [10, 16, 23, 28, 33].

3.2.2 Encoder-decoder LLMs

Encoder-decoder LLMs are language models that utilize both an encoder and a decoder module. The encoder module is responsible for encoding the input sentence into a hidden space, and the decoder is used to generate the target output text. This structure allows for more flexible training strategies [30]. The transformer model, introduced in 2017, is a prominent implementation of this architecture, consisting of

two distinct stacks of Transformer blocks that serve as the encoder and decoder [26, 27].

Encoder-decoder models like BART, PLBART, T5, CodeT5, AlphaCode, and CoTexT showcase flexibility in tasks such as summarization, translation, and question-answering [10, 16, 23, 27–29].

3.2.3 Decoder-only LLMs

Decoder-only LLMs exclusively use the decoder module for sequential token prediction, contrasting the encoder-decoder architecture. Models like the GPT series (GPT-1, GPT-2, GPT-3, GPT-3.5, GPT-4, ChatGPT), CodeGPT, InstructGPT, Codex, Copilot, GPT-J, GPT-Neo, GPT-NeoX, LLaMA, and Vicuna follow this architecture. They excel in downstream tasks without additional prediction heads or fine-tuning, making them valuable tools in SE tasks [16, 23, 28]. Decoder-only LLMs have established impressive benchmarks in numerous NLP tasks, especially in the generation of free-form text [10].

2022 saw a surge in decoder-only LLM development, further accelerated in 2023 with major companies like Google, Meta, and Microsoft launching products such as Bard, LLaMA, Llama 2, Bing Chat, etc. These newer models haven't found widespread application in SE yet, presenting untapped potential for specific tasks. The ongoing progress in decoder-only architectures signifies active exploration and innovation in this domain.

3.2.4 Sparse model

Sparse models, particularly those inspired by the mixture-of-experts approach, are emerging as a prominent area of advancement in LLM architecture. These models focus on activating a select subset of parameters for each input, leading to improved computational efficiency without sacrificing model capacity. Unlike dense models where every input activates all parameters, sparse models, especially MoE-based ones, activate specific "expert" parameters tailored to each input. Examples like Switch Transformer and GShard demonstrate how such sparse activation methods can achieve, or even surpass, the performance of dense models while requiring less computation. In the context of SoC security, where data can be both voluminous and complex, sparse models offer a valuable balance between computational efficiency and task-specific accuracy [10].

The distinct capabilities and limitations inherent to each LLM architecture underscore the significance of judiciously selecting the most suitable model architecture. This selection is pivotal in enhancing precision for a given task. The choice of architecture should be guided by a comprehensive understanding of the specific requirements of the task and the strengths and weaknesses of the available LLM architectures.

3.3 Comparative analysis of software security approaches and vulnerability detection methods

Given the importance of comprehending the distinctions and similarities among the premier and stablished methodologies for software vulnerability detection, Table 1 has been constructed to facilitate a more profound understanding of the subject matter. We created this table to systematically compare various methods used for software vulnerability detection, aiming to highlight their unique characteristics, strengths, and limitations. This structured comparison enables a clear understanding of how different techniques operate, their effectiveness in various contexts, and how they might complement each other in a comprehensive security strategy. By organizing key criteria such as purpose, code representation, adaptability, and primary use case, the table facilitates an informed evaluation of which methods are best suited for specific scenarios or stages in the software development lifecycle.

The creation of this table involved a thorough analysis of each method, drawing from academic literature, industry practices, and tool documentation. We carefully examined the functional mechanisms, adaptability, feedback mechanisms, and coverage of each technique. References were incorporated to ensure the comparisons are accurate and grounded in authoritative sources. This table is particularly beneficial for researchers considering the integration of LLMs with other methods, as it provides a comprehensive overview that aids in understanding potential synergies and complementary functionalitiesWe will discuss the information represented in the table in more detail after the table.

Based on this table we can see that DAST and IAST both focus on testing applications during runtime. However, their methodologies diverge. DAST operates based on predefined rules and simulates external attacks, which makes it suitable for high-level, external vulnerability detection. Conversely, IAST combines both static and dynamic analysis, offering more context-sensitive results by integrating directly into the application environment. This distinction makes IAST preferable when a deeper understanding of internal application behaviors and vulnerabilities is required.

We can see certain tools are optimized for specific phases of the software lifecycle. Static tools like Manual Code Review and SAST focus on the early development stages, offering a proactive approach to vulnerability detection. Dynamic tools like DAST, IAST, and fuzz testing, however, align more with deployment and runtime scenarios, showcasing a reactive strategy tailored to live environments. This delineation highlights the importance of strategically selecting tools based on project phases to maximize efficiency and coverage.

Table 1 Comparing the characteristics of LLMs with other established software vulnerability detection methods

	Purpose and design	Code representation	Learning and adaptation	Generalization	Feedback and iteration	Coverage	Basis of operation	Adaptability	Primary use case	References
LLMs	Designed to understanding and generating human-like text, including code	Token sequences	Yes (Continuously learn from training data; adapt based on seen patterns)	Yes (across various coding languages, patterns and styles)	Yes (provides explanations and justifications for generated code)	Comprehensive range of functionalities	Statistical analysis and pattern recognition	Yes (highly adaptable and flexible)	Text, image, video, or sound understanding, generation, and contextual reasoning	[26, 32, 34–37]
Manual Code Review	Designed to identify security vulnerabilities through human expertise	Source code	Yes (static knowledge and experience of reviewer)	Limited (to reviewer's expertise)	Yes (provides detailed reports and recommendations)	Comprehensive, but depends on reviewer 's skills	Human expertise, knowledge, and tools	Limited (to reviewer's skills)	Security vulnerability detection and code review	[10, 38, 39]
Manual Penetration Testing	Designed to identify security vulnerabilities through simulated attacker actions	Source code	Yes (static knowledge and experience of reviewer)	Limited (to reviewer's expertise)	Yes (provides detailed reports and recommendations)	Comprehensive, but depends on tester's skills	Human expertise, knowledge, and tools	Limited (to tester's skills)	Security vulnerability assessment and exploitation	–
Dynamic Application Security Testing (DAST)	Designed to identify security vulnerabilities in running applications through simulated attacks	Running application	No (rely on predefined rules and signatures)	No (precise, specific, and based on known patterns and signatures)	Yes (provides automated reports with limited context)	Comprehensive coverage, but limited control	Rule-based (Simulating attacker behavior)	Limited (but generally designed to be adaptable to different applications)	Security vulnerability detection in various application types including online applications, APIs and mobile apps	[23, 40]

Table 1 (continued)

	Purpose and design	Code representation	Learning and adaptation	Generalization	Feedback and iteration	Coverage	Basis of operation	Adaptability	Primary use case	References
Fuzz Testing (Fuzzing)	Designed to identify unknown vulnerabilities by providing invalid, unexpected, or random data as inputs	Inputs and program behavior	No (rely on predefined rules and signatures)	Yes (across various coding languages, patterns and styles)	Limited (provides feedback on root cause)	Comprehensive input coverage, but limited code paths	Rule-based (simulating attacker behavior by sending unexpected inputs)	Limited (to input variations)	Security vulnerability detection	[10, 41]
Interactive Application Security Testing (IAST)	Designed to identify vulnerabilities in running applications by combining static and dynamic analysis techniques, using an agent to monitor application behavior	Running application	Limited (some IAST tools might employ machine learning for pattern recognition and improvement, this is not a core characteristic)	No (tailored to specific application behavior)	Yes (provides precise, context-aware feedback on vulnerabilities, detailing their exact locations and exploitation conditions)	Comprehensive application behavior coverage limitated in identifying condition specific vulnerabilities or in complex code logic	Monitoring application behavior and interactions	Limited (IAST tools can adapt to different application environments, but their effectiveness might vary depending on the specific application and its architecture)	Security testing and vulnerability detection in web applications during the development and testing phases	[23, 42]

Table 1 (continued)

	Purpose and design	Code representation	Learning and adaptation	Generalization	Feedback and iteration	Coverage	Basis of operation	Adaptability	Primary use case	References
Code Sniffers	Designed to detect and flag programming errors, bugs, stylistic errors, and suspicious constructs that degrade readability, updatability, and overall maintainability	Source code	No (relies on predefined metrics and thresholds to detect smells)	No (specific to coding standards)	Yes (provides automated reports based on rule violations)	Comprehensive coverage of potential issues, limited to what is measurable by the predefined metrics	Rule-based (predefined coding rules)	No (to fixed set of rules)	Enforcing coding standards and best practices and automated code smell detection	[43]
Software Composition Analysis (SCA)	Designed to identify known vulnerabilities in open-source components	Bill of Materials (BOM) and component metadata	No (rely on predefined rules and signatures)	No (focuses on known vulnerabilities in components	Yes (provides a list of vulnerable components)	Limited to identification of known vulnerabilities in components	Rule-based (matching component versions against vulnerability databases)	Limited (to vulnerability databases)	Security vulnerability detection	[44–46]
Runtime Application Self-Protection (RASP)	Designed to detect and prevent real-time attacks	Running application	Yes (can learn from attack patterns)	Yes (adapts to application behavior and attack patterns)	Yes (provides real-time alerts and mitigation suggestions)	Limited to known and very few unknown attacks	Monitoring application behavior and interactions	Yes (can adapts to application behavior and attacks)	Real-time attack prevention	[47–49]

Table 1 (continued)

	Purpose and design	Code representation	Learning and adaptation	Generalization	Feedback and iteration	Coverage	Basis of operation	Adaptability	Primary use case	References
Static application security testing (SAST)	Designed to detect security vulnerabilities early in the development process by scanning the static code (source code or binaries) without executing the program	Source code	No (rely on predefined rules and signatures)	Limited (specific to implemented rules)	Yes (immediate and direct reports on specific issues)	Comprehensive coverage of vulnerabilities in the codebase	Rule-based (program structure, control flow graphs, and data flow analysis)	Limited (customizable rules to target specific coding standards or security requirements)	Code quality assurance, bug detection	[42, 50–52]

A deeper comparison uncovers overlaps among tools like Fuzz Testing and DAST. Fuzz Testing injects unexpected or random inputs to identify unusual program behaviors, offering extensive input coverage. DAST, while also external, focuses on predefined attack patterns and rules, making it less versatile but more targeted. Scenarios requiring detection of highly specific vulnerabilities might benefit more from DAST, whereas Fuzz Testing is advantageous for uncovering unexpected, edge-case issues. The overlaps between these tools imply that while they may address similar objectives, the choice depends on specific requirements such as depth of analysis, adaptability, and the type of feedback needed. For example, when adaptability and large-scale application are priorities, LLMs outperform manual methods. Conversely, where human insight into contextual nuances is critical, manual approaches prevail. Understanding these nuances ensures informed selection and complementary use of tools for comprehensive evaluation.

A striking observation is the difference between rule-based tools and adaptive learning-based approaches. Tools like SAST and DAST, which depend heavily on predefined rules and patterns, provide reliable precision in detecting known vulnerabilities but lack the flexibility and breadth of modern learning-based systems such as LLMs. These AI-driven solutions are particularly noteworthy for their ability to generalize across diverse languages and scenarios, signaling a trend toward increasingly adaptable and scalable security methodologies. Another critical fact is that reliance on human skill underscores inherent limitations in scalability and consistency when applied to large-scale or time-sensitive projects. This limitation contrasts sharply with automated tools like DAST and fuzz testing, which excel in executing repetitive tasks efficiently, albeit with less contextual depth.

The complementary nature of these tools emerges as a central theme. For instance, the precision of SAST in detecting early-stage vulnerabilities in source code pairs effectively with the runtime testing capabilities of DAST. Together, they create a comprehensive security posture that addresses both development-phase issues and vulnerabilities arising during deployment. Similarly, fuzz testing and manual penetration testing can operate in tandem, with the former identifying edge-case anomalies through random input injections and the latter validating these findings through simulated exploitation scenarios. Strengths and limitations become more apparent when tools are grouped by their primary methodologies. Rule-based tools, like SAST and DAST, are lauded for their clarity, repeatability, and efficiency but struggle with novel or unforeseen vulnerabilities. In contrast, adaptive tools like LLMs offer remarkable flexibility, learning over time to address gaps in static methodologies. Yet, these adaptive solutions are not without flaws, as they may produce false positives or require significant computational resources.

Patterns also emerge regarding feedback mechanisms. LLMs offer iterative, explanatory feedback, fostering a deeper understanding and resolution of issues. By contrast, fuzz testing and DAST are more automated, providing concise reports that may require additional interpretation. This divergence highlights the need to balance automation with interpretability based on the project's specific goals.

A further point of interest lies in the disparity of code representation among the tools. Some, like SAST and Manual Code Review, analyze source code, which allows them to identify structural and logical flaws directly within the codebase. Others, such as DAST and fuzz testing, focus on inputs and runtime behavior, bypassing the need for access to source code altogether. This distinction makes runtime tools invaluable in scenarios where source code is unavailable, such as testing third-party applications or assessing APIs in black-box environments. The table also underscores variations in generalization capabilities. Tools like LLMs and fuzz testing demonstrate high generalization potential, making them versatile across different languages, patterns, and applications. In contrast, rule-based systems like DAST excel in predefined scenarios but falter when encountering novel or emergent threats, further emphasizing the growing demand for adaptive technologies.

The interplay of these tools underscores the importance of an integrated approach. No single method comprehensively addresses all vulnerabilities; instead, their combined use leverages individual strengths while mitigating weaknesses. The table, therefore, not only categorizes tools by their capabilities but also serves as a blueprint for designing multi-faceted, effective testing strategies tailored to complex, modern applications.

3.4 Large language models in software development life cycle (SDLC)

Security vulnerabilities are often introduced during the construction stage of the Software Development Life Cycle (SDLC), but they may not become apparent until they result in security failures during the operational stage. This delayed detection can lead to increased repair costs, harm to the company's reputation, and potentially irreversible damage to the application. To mitigate these risks, it's essential to employ specialized vulnerability detections tools, in all the stages of SDLC [53].

In light of this, we encourage other researchers to explore the integration of LLMs for detecting and handling software vulnerabilities across all stages of the SDLC. The SDLC can be subcategorized into six stages [16]:

Requirements engineering;
Software design;
Software development;

Software quality assurance;
Software management;
Software maintenance.

In the context of the SDLC, LLMs are particularly valuable for detecting and managing software vulnerabilities during stages 3 (Software Development) 4 (Software Quality Assurance) and 6 (Software Maintenance). However, due to their vast potential, the utility of LLMs extends beyond just these stages. Moreover, this field is still in its infancy, offering a vast and largely untapped potential for research and innovation. The following table outlines how various LLM models have been utilized in different stages of the SDLC (Table 2).

4 RQ1: Why should we use LLMs for software vulnerabilities detection?

As businesses increasingly integrate with the digital realm, the landscape of cyber threats evolves rapidly, growing more intricate and severe. This surge in connectivity amplifies the urgency for robust cybersecurity measures. Amidst this challenge, emerging research highlights the potential of NLP techniques in fortifying cyber defense mechanisms. Specifically, NLP applications exhibit promise in detecting vulnerabilities within software code, a critical aspect in preventing cyber-attacks. It's well-documented that software bugs serve as prime entry points for malicious actors, precipitating cyber crimes. Despite advancements, the persistence of software vulnerabilities remains evident, as illustrated by the annual update of the Common Vulnerabilities and Exposures (CVE) list. Traditional error identification methods, once relied upon, now face scrutiny due to their susceptibility to inaccuracies and misdiagnoses. This underscores the pressing need for innovative approaches, including those grounded in NLP, to enhance cyber resilience in an increasingly interconnected world [13]. AI's ability to process vast datasets in real-time, extract insights, and anticipate potential threats has the potential to revolutionize proactive cybersecurity measures [71]. By using LLMs like GPT along with suitable datasets such as the Software Assurance Reference Dataset (SARD) benchmark dataset and SeVC dataset, we can analyze to find weak code in different programming languages including C + + /C and Java [72].

While machine learning has been used for vulnerability detection, traditional methods require complex and error-prone manual feature engineering to define what information the machine should analyze. Deep learning approaches using Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) have been explored, but they necessitate specifically formatted code data, which can be challenging. Giving rise to the Transformer-based neural network architectures, known for their success in natural language processing. The goal is to develop a model that automatically learns both syntactic (structure) and semantic (meaning) information from various programming languages, paving the way for even more powerful vulnerability detection systems using LLMs [13, 73]. Moreover, transformer-based language models hold appeal and promise over RNNs due to their capacity for parallelization in model computation, facilitating quicker processing compared to RNNs. This is crucial for decreasing the time required for model training and testing, especially when dealing with large-sized models like transformer-based ones. Additionally, their capability to transition from natural language processing tasks to related tasks, via transfer learning, allows for broadening their applicability across various domains [74].

Leveraging transformer-based models such as GPT-4 and open-source models like Llama for vulnerability detection can offer numerous benefits, including enhanced accuracy and natural language processing capabilities. These models obviate the necessity for manual input in static analysis tools, streamlining the detection process into a faster, more automated procedure [13].

RQ1 Answer: *Large language models should be utilized for software vulnerabilities detection due to their ability to automate the identification of code vulnerabilities across various programming languages. They offer enhanced accuracy and natural language processing capabilities, streamlining the detection process and facilitating quicker processing compared to traditional methods.*

5 RQ2: What roles can LLMs play in software vulnerability detection and mitigation?

5.1 Code analysis and vulnerability detection

LLMs can be trained to analyze source code and identify potential vulnerabilities. By understanding programming languages and common vulnerability patterns, LLMs can flag insecure code practices, such as buffer overflows [12, 17, 23, 24, 52, 72, 75–77], SQL injection points [12, 24, 38, 58, 61, 78, 79], and error handling [1, 38, 80], thereby assisting developers in preemptively fixing issues.

5.2 Automated threat detection in logs

LLMs can process large volumes of log data from servers, applications, and network devices to detect anomalies that might indicate a security breach. By recognizing patterns and deviations from normal behavior, LLMs can help in identifying potential threats in real time, allowing for quicker response and mitigation [11, 70, 81].

Table 2 Utilization of LLM models across software development life cycle (SDLC) stages

DLC stage	LLM models	Use case	References
Requirements engineering	BERT, SpanBERT, ChatGPT, ELECTRA, T5, T-BERT	Anaphoric ambiguity treatment, Requirements classification, Coreference detection, Requirements elicitation, Software traceability	[66, 109, 183, 217, 296, 338], as cited in [16, 49, 54–56]
Software design	BERT, GPT, DALL·E,	GUI retrieval, Software specification synthesis, Rapid prototyping	[151, 205, 346], as cited in [15, 16, 22, 55, 57]
Software development	GPT-4, Codex, BERT series including CodeBERT, Amazon CodeWhisperer, GitHub Copilot, Codex, CodeGen, InCoder, CodeGeeX, AlphaCode, CodeT5, PolyCoder, StarCoder, phi-1, Code Llama	Code generation, Code completion, Code summarization, Program synthesis, API recommendation, API synthesis, Code comment generation, Code representation, Code retrieval, Code optimization, Code example recommendation	[8, 15, 16, 30, 33, 34, 36, 37, 41, 54, 58–61]
Software quality assurance	BERT, GPT-3.5, CodeT5	Test generation, Vulnerability detection, Test automation, Verification, Bug localization, Failure-inducing test identification, Flaky test prediction	[30, 32, 42, 72, 136, 283, 303, 391], as cited in [12, 16, 29, 54, 62–64]
Software management	BERT	Effort estimation	[65]
Software maintenance	Codex, ChatGPT, BERT, CodeBERT, T5	Program repair, Code review, Patch generation, Debugging, Bug report analysis, Code clone detection, Logging, Bug prediction, Bug triage, Bug report replay, Decompilation, Sentiment analysis, Program merge conflicts repair, Traceability recovery, Duplicate Detection,	[28, 39, 44, 73, 106, 122, 127, 137, 158, 163, 211], as cited in [1, 12, 16, 28, 29, 66, 67], as cited in [25, 54, 68, 69], as cited in [27, 70]

5.3 Phishing email and website detection

By analyzing the content and structure of emails, LLMs can detect phishing attempts. These models can identify suspicious elements, such as unusual sender addresses, misleading links, and deceptive language, helping to filter out harmful emails before they reach end-users [70, 81–85]. This capability extends to software vulnerability detection, as phishing emails often serve as vectors for exploiting security weaknesses in software systems, making LLMs valuable in both identifying and mitigating such vulnerabilities.

5.4 Automated patch recommendation and APR

Leveraging LLMs for APR has become a highly researched area, particularly following recent advancements in LLM technologies. When a vulnerability is detected, LLMs can suggest or even generate patches to fix the issue. By understanding the context of the code and the nature of the vulnerability, LLMs can provide precise modifications that developers can implement to enhance security [8, 9, 17, 28, 34, 86–88].

5.5 Natural language processing and incident response

LLMs play a crucial role in both interpreting security reports and supporting incident response efforts. By extracting key information from lengthy security reports and advisories, LLMs provide actionable insights and recommendations, enabling security teams to stay informed and respond more effectively to emerging threats. During security incidents, LLMs can further assist by analyzing attack vectors, correlating events, and offering insights into the attacker's methods, helping teams understand breaches and implement effective countermeasures [11, 27, 70].

5.6 LLMs for security vulnerability insertion and detection in SoC

LLMs like GPT-3.5 and GPT-4 can be employed to identify and insert potential vulnerabilities into hardware designs by following natural language prompts. These models detect security rule violations, such as unreachable states or static deadlocks, and pinpoint weak areas in hardware designs. By analyzing design logic, LLMs can introduce vulnerabilities for testing or identify issues like duplicate encoding, aiding in preemptive security efforts [10]. This highlights the crucial role of LLMs in both detecting and handling security vulnerabilities, extending their application to hardware-level vulnerabilities within the broader scope of software vulnerability detection and mitigation.

5.7 LLMs for penetration testing

Researchers have explored the use of LLMs such as GPT-3.5 to enhance penetration testing processes. The LLM was employed to assist penetration testers by providing high-level task planning and low-level vulnerability hunting. Specifically, it helped in planning security testing assignments and analyzing the state of vulnerable virtual machines to suggest concrete attack vectors. These suggestions were then automatically executed, demonstrating the potential of LLMs to augment human expertise in identifying and exploiting security vulnerabilities [89].

5.8 Training and simulation

LLMs can be used to create realistic cybersecurity training scenarios. By simulating attacks and defenses, they can help train security personnel in recognizing and responding to threats, improving overall preparedness and resilience.

Each of these examples illustrates a different aspect of how LLMs can be leveraged to enhance the detection and handling of software vulnerabilities and cybersecurity threats.

RQ2 Answer: *The application of LLMs in the detection and handling of software vulnerabilities includes, but is not limited to Code Analysis and Vulnerability Detection, Automated Threat Detection in Logs, Phishing Email and Website Detection, Automated Patch Recommendation and APR, Natural Language Processing and Incident Response, Training and Simulation.*

6 RQ3 and RQ4: performance of LLMs in software vulnerability detection: models and comparisons with traditional methods

This section addresses two key research questions: RQ3, *What specific types of LLMs have been utilized in software vulnerabilities detection*? And RQ4, *In comparison to traditional methods and tools, how do LLMs perform in detecting and handling software vulnerabilities*? Both questions will be answered through analysis of the data presented in Table 3.

To answer this question, and assist researchers, we have compiled Table 3 detailing the specific LLM models used in various studies, the purposes for which they were tested, the results of the experiments, and the corresponding research papers. This table provides a foundation for understanding how different LLMs have been applied and how their performance compares to traditional methods. The datasets used in these and other studies will be discussed in detail in the section addressing RQ6.

RQ3 Answer: *Various types of LLMs have been utilized in software vulnerabilities detection, including:*

Table 3 Summary of LLM utilization for software vulnerability detection and results

Models name	Domain	Use case	Datasets and size (if stated)	Evaluation metrics	Results	References
GPT-3.5-turbo	Software vulnerabilities and formal verification	Integrating LLMs with formal verification to verify and repair software vulnerabilities in C code	AI generated dataset (1000 C code samples, each consisting of 20 to 50 lines of code)	Code Repair Success Rate	Success rate of up to 80% for the presented framework	[17]
GPT-3.5 GPT-4	Software on chip Security	Inserting hardware vulnerabilities into designs Mitigation of hardware vulnerabilities Hardware trojan detection (contextual test)	Not stated	Success rate in insertion, detection and mitigation of hardware vulnerabilities	GPT-3.5: success rate of up to 94.08% in insertion of hardware vulnerabilities GPT-3.5: Success rate of up to 96.43% in mitigation of hardware vulnerabilities GPT-3.5: 16.36% Success rate in hardware trojan detection GPT-4: 91.67 Success rate in hardware trojan detection	[10]
BERT as the foundation for creating SecurityBERT	Cybersecurity	Identification of fourteen distinct types of attacks in IoT and IIoT networks including SQL-injection, Password, DDoS-HTTP, Uploading, Backdoor, Port-Scanning, and etc	Edge-IIoTset cyber security dataset (including 15 attack type related to IoT)	Precision, Recall, F1-score accuracy	98.2% Accuracy	[11]

Table 3 (continued)

Models name	Domain	Use case	Datasets and size (if stated)	Evaluation metrics	Results	References
all-mpnet-base-v2 all-distilroberta-v1 all-MiniLM-L12-v2	Application security	Detecting logical vulnerabilities in Web applications	Custom-generate dataset using an auto-regressive LLM with SOTA few-shot prompting techniques	Precision, Recall, F1-score, Accuracy MCC	Model1: Accuracy: 87.19% Model2: Accuracy: 87.53% Model3: Accuracy: 86.01%	[23]
FalconLLM40B as the foundation for creating SecureFalcon 121M	Software vulnerability detection	SecureFalcon was used to classify C/C + + code samples as vulnerable and non-vulnerable	FormAI. AI-generated dataset including 112,000 compilable and independent C programs	Precision, Recall, F1-score, and Accuracy	SecureFalcon achieved a 94% accuracy when learning rate was 2e-5 and 87% accuracy when learning rate was 2e-2	[75]
GPT-4 (July/August 2023)	Software vulnerability mitigation	Using LLMs for multistage bug fixing, inspired by the Tree-of-Thoughts (ToT) framework	QuixBugs dataset containing of 40 defective Python files	Number of fixed vulnerabilities	Using ToT framework, the model was capable of correcting complex bugs that even GPT-4 failed to resolve using traditional prompt engineering	[86]
GPT-4	Software vulnerability detection and mitigation	Using LLMs to detect and fix software vulnerabilities	129 code samples across eight programming languages	Number of detected vulnerabilities and fixes	GPT-4 found 4 × more vulnerabilities than Snyk and Fortify (static analyzers)	[12]
CodeGen 2.7B	Software Security	Controlled code generation enhanced with SVEN method for security hardening and adversarial testing	CrossVul, Big-Vul, VUDENC with the total of 1606 programs which 1440 was used for training and 166 program was used for validation	HumanEval benchmark for evaluating the functional correctness	92.3% success in generation of secure code with security hardening and 36.8% success rate during adversarial testing	[58]

Table 3 (continued)

Models name	Domain	Use case	Datasets and size (if stated)	Evaluation metrics	Results	References
GPT-3.5-turbo	Blockchain Security	Combining GPT with static analysis for logic vulnerability detection in smart contracts	3 Dataset: Top200, Web3Bugs, and DefiHacks, containing 400 contract projects, 3K Solidity files and 472K lines of code	Recall, F1 score, Accuracy TP, TN, FP, FN	Dataset Name / TP / TN / FP / FN / Sum Top200: 0 / 283 / 13 / 0 / 296 Web3Bugs: 40 / 154 / 30 / 8 / 232 DefiHacks: 10 / 19 / 1 / 4 / 34	[90]
CodeBERT 125M GraphCodeBERT 125M PLBART 140M CodeT5 220M UniXcoder 125M	Software vulnerability repair	Fine-tuning LLMCs for APR in various repair scenarios under the NMT fine-tuning paradigm	BFP dataset SequenceR dataset Recoder dataset Defects4J dataset CPatMiner dataset VulRepair dataset TFix dataset	Perplexity BLEU	Experimental results indicate that even without any post-processing strategies, LLMCs outperform many previous APR methods	[34]
JavaBERT 4HS CodeBERT 4HS CodeBERT PO	Software vulnerability detection	Using transformers for multi-label classification of vulnerabilities in Java language	A dataset built from Juliet Test Suite for Java. including 113,898 methods, 80,269 of which are non-vulnerable and 33,629 are vulnerable. split 80:10:10 portion for training, validation and test	Accuracy, F1score, Precision, and Recall	Up to 99% in accuracy, and up to 94% in f1-score	[73]
CodeBERT GraphCodeBERT	Software vulnerability repair	For vulnerability repair in a dataset of C programming language	CWE121, CWE190, CWE369, CWE401, and CWE457 from Juliet C/C++ (11,221 file in total). Split to 6:2:2 for train/valid/test	Accuracy	single-line repair accuracy 95.47%, multi-line repair accuracy 90.06%	[87]

Table 3 (continued)

Models name	Domain	Use case	Datasets and size (if stated)	Evaluation metrics	Results	References
Google-T5	Cybersecurity	Using LLMs for mapping vulnerabilities to attack patterns based on its text description	CWE	Accuracy	VWC-MAP links vulnerabilities to weakness types with 87% accuracy and to new attack patterns with 80% accuracy	[91]
GPT-3.5 GPT-4	Cybersecurity	Utilization of ChatGPT for detecting phishing sites	1,000 phishing sites and an equal number of non-phishing sites. Gather using web crawling	Accuracy, Precision, Recall, and F-measure	GPT-3.5 / GPT-4 Precision: 98.3% / 98.3% recall: 86.7% / 98.4% accuracy: 92.6% / 98.4 F-measure: 92.1% / 98.4	[83]
GPT-3 (Davinci) ChatGPT (GPT-3.5-turbo)	Software vulnerability detection	Vulnerability Detection in a real-world dataset of Java files through binary and multi-label classification tasks	Manually collected dataset of Java files from open GitHub repositories. Containing both vulnerable and patched code	Accuracy Precision Recall F1-Score AUC	The results showed that capabilities of models for effectively detecting vulnerabilities in code are limited and no better than a dummy classifier	[92]
GPT-3, GPT-3.5, GPT-4, StarCoder, WizardCoder, LLAMA 2, Code LLAMA, Mixtral-MoE, StarChat-β, MagiCoder, Mistral	Software vulnerability detection	Vulnerability detection in code, specifically distinguishing between vulnerable and fixed versions of functions	100 functions from SVEN dataset	Balanced Accuracy, Precision, Recall	The models performed with a balanced accuracy of 0.5–0.63, showing difficulty distinguishing between vulnerable and non-vulnerable code versions. They failed in 76% of cases to correctly identify 57% of LLM responses contained errors, and the models frequently predicted incorrect locations of buggy code and misidentified bug types	[77]

Table 3 (continued)

Models name	Domain	Use case	Datasets and size (if stated)	Evaluation metrics	Results	References
GPT-4, GPT-3.5, chat-bison, codechat-bison, codel-lama7b,13b,34b, and starchat-beta	Software security	Evaluating the reliability of LLMs in detecting and reasoning about security vulnerabilities in code	Novel dataset containing 228 code scenarios (48 hand-crafted, 30 real-world, and 150 with code augmentations)	Rouge score, Cosine Similarity, GPT-4 as evaluator	Results show that LLMs generate unpredictable responses, exhibit flawed and unreliable reasoning, and underperform in real-world applications	[93]
DeepSeekCoder 7B, 33B Llama3 8B, 70B GPT-4 and GPT3.5 Mistral 7B Gemma 7B	Software security and static analysis for vulnerabil-ity detection	Detecting security vulnerabilities in large-scale Java projects by combining LLMs with static analysis in a framework called IRIS	Novel dataset called CWE-Bench-Java. Containing 120 manually validated security vulnerabilities in real-world Java projects	Number of detected vulnerabili-ties	IRIS detected 69 out of 120 vulnerabilities but state-of-the-art static analysis tool only detected 27	[50]
GPT-4 turbo GPT-4 Claude 3 Opus	Software vul-nerability detection	Comparing the performance of LLMs against static code analyzers	Juliet Java 1.3. containing vulnerabilities from 112 different CWEs	Accuracy Precision, Recall F1-score	Experimental results indicate GPT-4's superiority in Recall and F1 score, while CodeQL excels in Precision and SportBugs in Recall	[38]
6 open source LLMCs and CodeBERT-base Mistral-7b-base Mixtral-8 × 7b-base CodeLlama 7B & 13B GPT-4-base	Software vul-nerability detection	Binary classification of vulnerable and non-vulnerable code	Custom dataset from GitHub + DiverseVul (13,532), Devign (27,318), Lin2017 (621), Choi2017 (14,000), LineVul (18,864), PrimeVul (25,908)	Precision, Recall F1-score	Results shows that LLMs vulnerability detection accuracy is limited by their training data and may not consistently perform well across datasets but Fine-tuning enables resource-constrained models to achieve superior performance in tailored contexts compared to larger models	[94]

OpenAI's GPT family (GPT-3.5, GPT-3.5-turbo, GPT-4), LLAMA models, BERT, GraphCodeBERT, Sentence-Transformers models (all-mpnet-base-v2, all-distilroberta-v1, all-MiniLM-L12-v2), the FalconLLM family, CodeGen, CodeBERT, PLBART, CodeT5, JavaBERT, Google-T5, Mistral models, and others.

These LLMs demonstrate proficiency in tasks such as vulnerability detection, malware classification, code summarization, and code generation, showcasing their potential in contributing to cybersecurity research and practice. However, the table also highlights that the results weren't always positive, with even advanced models occasionally performing with less than 50% accuracy, meaning a simple guessing machine might outperform them. This clearly indicates room for improvement, and fine-tuning these models could greatly enhance their accuracy and positive results.

RQ4 Answer: *Table 3 demonstrates that while Large Language Models (LLMs) hold great potential for producing positive outcomes and even outperforming traditional methods, they also present certain challenges and limitations. We hope that by analyzing this section, researchers will be motivated to explore innovative approaches and contribute to advancing this area of study. With continued progress, LLMs could become valuable tools for enhancing software security.*

7 RQ5: In what ways have LLMs been applied to the detection of software vulnerabilities?

To address the research question, we explore a range of studies that showcase the diverse applications of LLMs in various facets of software security enhancement. Each piece of research provides a distinct approach, utilizing LLMs within custom-built frameworks specifically designed for vulnerability detection. By examining the methodologies, we can obtain a comprehensive view of how LLMs contribute to improving vulnerability detection and mitigation strategies.

Explaining the experimental frameworks used in these studies is critical for understanding how LLMs are applied to enhance software security. These frameworks serve as the foundation on which the LLMs operate, and they provide valuable context for interpreting the results. The way the LLMs are integrated into the broader security environment can significantly influence their performance, and understanding this context helps validate the results. By detailing these frameworks, we can clearly see the novel contributions made by each study, including how the use of LLMs led to improved accuracy, faster threat mitigation, or more efficient automation. Furthermore, this discussion provides comparative insights that allow readers to evaluate the strengths and limitations of different approaches in similar settings.

Finally, by detailing the experimental frameworks, this section lays the groundwork for future research, offering potential pathways for refining or expanding upon the methodologies discussed. Whether for mitigating new types of threats or extending the capabilities of LLMs to other areas of software security, understanding the underlying frameworks is essential for advancing the field.

LLMs for APR (1): In article [17], LLMs have been applied to the detection of software vulnerabilities by being integrated into an automated code repair framework. In this framework, LLMs are used to generate modified code after initial verification or falsification by a Bounded Model Checker (BMC). The LLMs help in identifying software vulnerabilities, such as buffer overflow and pointer dereference failures, and suggest corrective patches. This iterative process of verification and repair enables LLMs to effectively address vulnerabilities, achieving a success rate of up to 80% [17]. Figure 1 shows the framework of the proposed model.

The study evaluated over 1000 C programs and found that combining BMC and LLM can effectively detect and fix software vulnerabilities. The approach was particularly successful in addressing buffer overflows and pointer dereference failures, achieving a success rate of up to 80%.

GPT-4 as a code analyzer: Real-world examples are essential for bridging the gap between theoretical research and practical application. They validate the effectiveness of methodologies and provide insights into how LLMs perform in real-world software security environments. By examining actual cases where LLMs-based systems have been used to identify and mitigate vulnerabilities, we can better understand their strengths, limitations, and practical implications. This case study explores a concrete example, illustrating how LLMs can be applied to detect and address software vulnerabilities, and highlighting the tangible benefits they offer in enhancing cybersecurity measures. A notable example of an off-the-shelf LLM, is ChatGPT.

In [12] the authors of the study employed 7 LLMs from OpenAI, including GPT-4, for detecting and fixing software vulnerabilities using a structured process. The significance of this study lies in its approach, which avoids the use of a sophisticated framework, fine-tuning, or modifications to the models. This minimalist methodology provides a clear assessment of the inherent effectiveness of LLMs when compared to other methods and tools for software vulnerability detection.

To evaluate how effectively LLMs can identify and correct security vulnerabilities, the models were accessed automatically through the OpenAI API, using a specific instruction (referred to as a system context): "*act as the world's greatest static code analyzer for all major programming languages. I will give you a code snippet, and you will identify the language and analyze it for vulnerabilities. Give the output in a format: filename, vulnerabilities detected as a numbered*

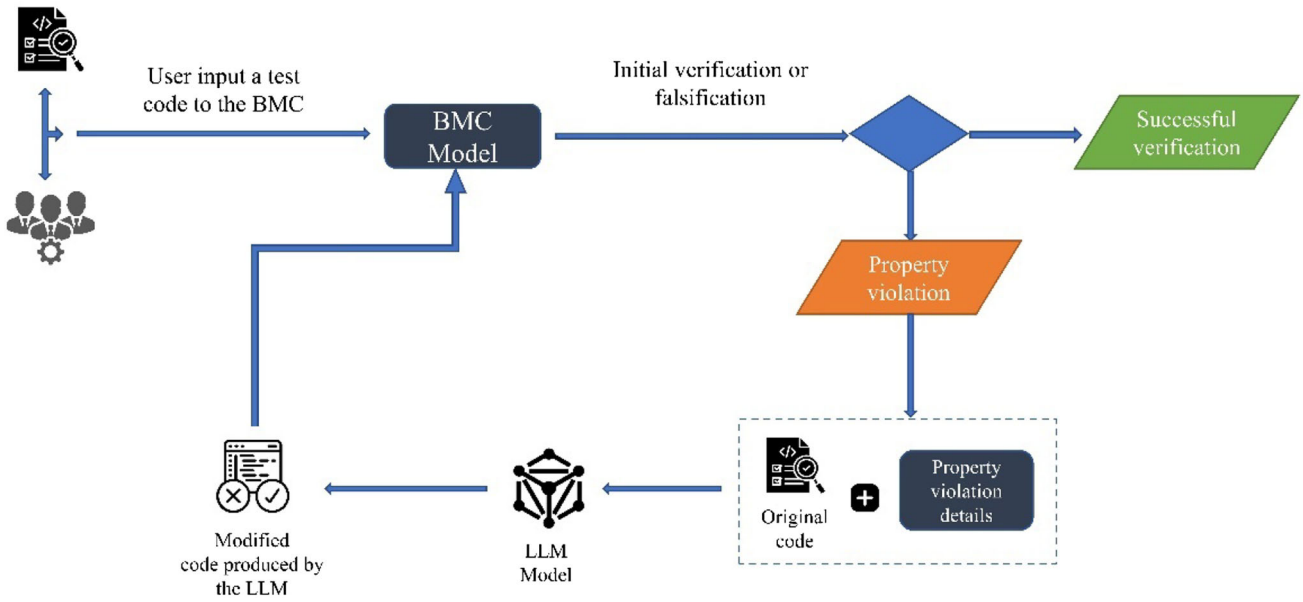

Fig. 1 Automated code repair framework. Inspired by Fig. 1 in [17]

list, and proposed fixes as a separate numbered list.". The instruction guided the models to detect vulnerabilities in code, identify the programming language, and suggest fixes. The results were returned in a clear format, listing the vulnerabilities and their proposed solutions.

To test the models, researchers provided them with 128 code snippets that represented vulnerabilities in eight widely used programming languages, including C, Ruby, PHP, Java, JavaScript, C#, Go, and Python. These code snippets were selected to cover 33 different types of security flaws, ranging from Buffer Overflows to Sensitive Data Exposure. Many of these examples, particularly in PHP, involved issues like file inclusion and command injection vulnerabilities.

To compare the LLMs' performance with existing tools, researchers also analyzed the same code snippets using a well-known static code analysis tool called Snyk. This tool is widely used by software developers and provides detailed reports about vulnerabilities. It categorizes issues based on their severity (e.g., critical or low risk), whether they can be fixed, and how likely they are to be exploited. By using Snyk, the study aimed to highlight the types of vulnerabilities these tools could find and how they compared to the LLMs. Here, GPT-4 identified 393 vulnerabilities—almost double the number found by GPT-3 (213) and four times as many as Snyk (99). Indicates that larger models are more advanced and better at tasks that require understanding and reasoning. Notably, when GPT-4 proposed fixes for these vulnerabilities, it provided 398 specific corrections, showcasing its ability to not only detect issues but also propose actionable solutions.

The results showed that GPT-4 is highly effective at reducing vulnerabilities. When the codebase corrected by GPT-4 was analyzed using Snyk, the number of high-severity vulnerabilities dropped by 94%, medium-severity vulnerabilities by 75%, and low-severity issues by 92%. Moreover, GPT-4 added about 11% more lines of code to the project, which likely reflects the inclusion of security checks and validations to mitigate vulnerabilities.

The tool also performed well across all programming languages, even correctly identifying the language of each snippet without being explicitly told. This suggests GPT-4 or other LLMs could be a valuable tool for automating the analysis of large, complex codebases.

To better understand this case study, we will present an example of the code tested and how GPT-4 and Snyk responded to it. This example highlights the differences in their ability to identify and address software vulnerabilities, providing a practical illustration of the strengths and limitations of each tool.

By analyzing a sample Python code, we demonstrate the types of vulnerabilities that can exist in real-world applications and how these tools detect and resolve them. This comparison also emphasizes GPT-4's capacity to go beyond identification by offering detailed explanations and actionable fixes, which are crucial for improving code security.

Now, let us explore this specific example from the study:

Code	GPT-4 and Snyk Findings
analyzer/python/ikos/scan.py ''' Run the given command and return the exit code ''' log.debug('Running %s' % command_string(cmd)) try: proc = subprocess.Popen(cmd)	*Snyk analysis:* Unsanitized input from a command line argument flows into subprocess.Popen, where it is used as a shell command. This may result in a Command Injection vulnerability *GPT-4 analysis* Language: Python Vulnerabilities Detected: Command Injection: Using subprocess.Popen() directly with user-controlled input can introduce command injection vulnerabilities, especially if cmd can be influenced externally Logging Sensitive Data: Logging the exact command being executed might expose sensitive data or internal workings that could be exploited if logs are not secured or inadvertently disclosed Incomplete Code: The provided code snippet appears to be incomplete. Without handling the process results or exceptions properly, there could be unexpected behavior or undisclosed errors Proposed Fixes: Sanitize Input: Ensure that cmd is properly sanitized, especially if it can be influenced by external input. Prefer using the list format for the cmd argument in Popen which avoids shell interpretation Instead of: subprocess.Popen("ls -l") Use: subprocess.Popen(["ls", "-l"]) Limit Logging: Consider logging only necessary information without exposing the exact command or arguments. If detailed logs are essential for debugging, ensure that the logs are protected and access controlled Handle Process Result: Ensure you handle the result of the process. Capture stdout, stderr, and handle any potential exceptions from the Popen call. Consider using subprocess.run() for more straightforward command execution with better error handling

LLMs for APR (2): As shown in Fig. 2, Huang et al. [34] proposed a workflow for applying LLMs in APR, consisting of several steps designed to enhance the model's capability to understand and generate code fixes. Initially, data pre-processing transforms raw source code into a format suitable for LLM processing, employing techniques like code abstraction and different code representations to enhance the model's understanding of fixing patterns. Subsequently, model training and tuning extend LLMs into the Neural Machine Translation (NMT) architecture, focusing on encoder-only and encoder-decoder models for their superior performance. Through iterative training on the dataset, the model learns domain knowledge for defect repair. During this process, checkpoints are evaluated using various metrics to identify the best-performing model for patch generation.

Following model evaluation, the patch generation phase employs the beam search strategy to synthesize patches from multiple repair models. Plausible patches are filtered using test cases, and manual validation is conducted to assess the correctness of the generated patches. This validation process helps ensure the accuracy of the generated fixes. Overall, this workflow aims to systematically explore preprocessing techniques, model architectures, evaluation metrics, and patch generation strategies to enhance the LLM's capability for automated program repair.

The paper demonstrates that Large Language Models of Code (LLMCs) exhibit strong repair capabilities across various scenarios under the NMT fine-tuning paradigm. Even without employing post-processing strategies, LLMCs achieve impressive results, surpassing many existing APR approaches.

Vulnerability repair solution: The framework for the vulnerability repair solution presented in [87] is built around leveraging pre-trained programming language models such as CodeBERT and GraphCodeBERT to handle both single-line and multi-line security vulnerabilities in code. The process begins by constructing a dataset of vulnerability code paired with corresponding fixes, known as bug-fix pairs (BFPs). These BFPs serve as the training data for fine-tuning the pre-trained PL models.

The model selection phase was crucial in choosing the appropriate PL models for this task. The authors chose CodeBERT and GraphCodeBERT due to their ability to handle multiple programming languages and incorporate programming-specific features, such as data flow information in GraphCodeBERT. The system was then fine-tuned on the downstream task of vulnerability repair.

During the fine-tuning process, the model learns to generate fixes based on vulnerability code inputs. For testing, vulnerability code segments are fed into the model,

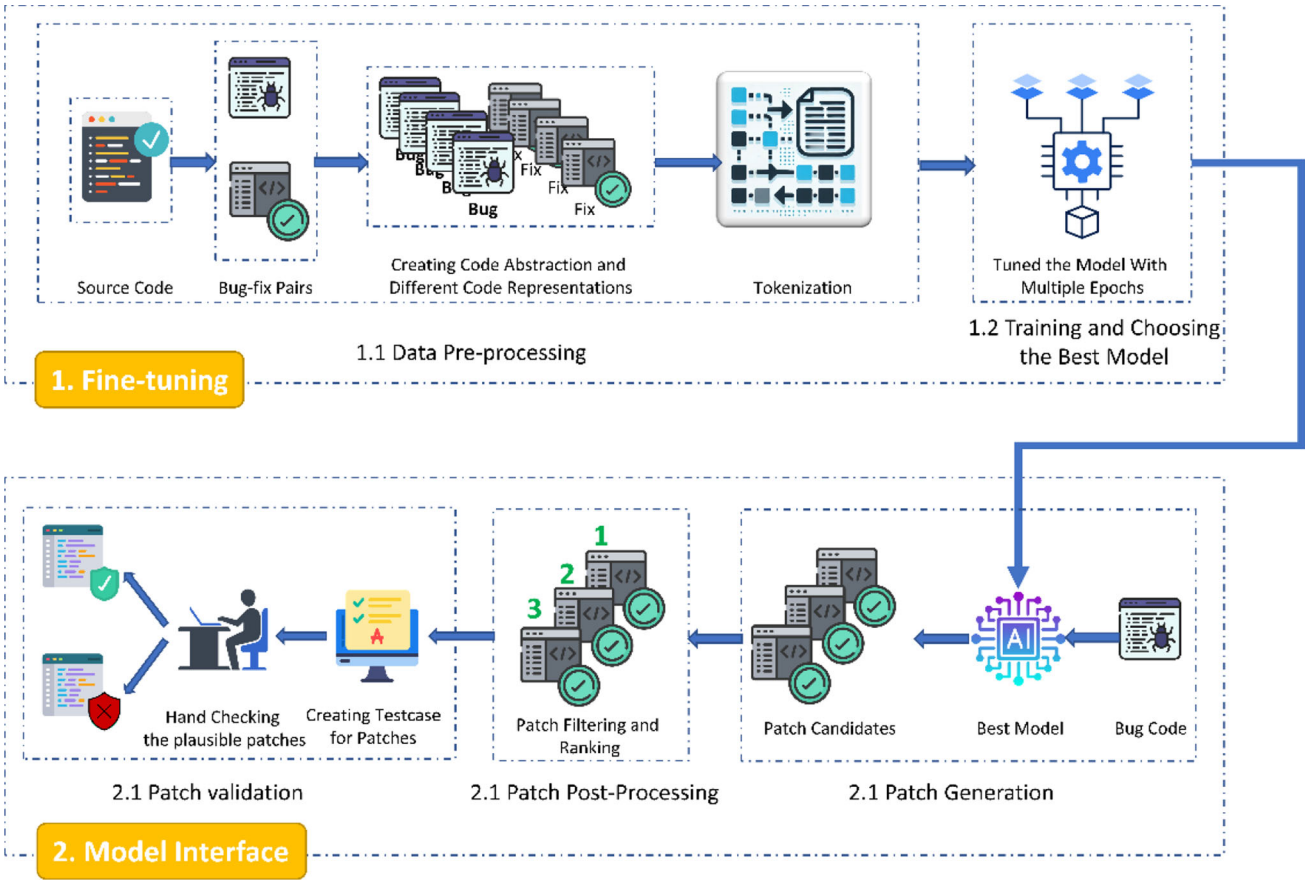

Fig. 2 Overview of the APR workflow using NMT fine-tuning for LLMCs. Inspired by Fig. 1 in [34]

which predicts the repaired code. The models' performances were evaluated using the Bilingual Evaluation Understudy (BLEU) score, a metric commonly used for evaluating the quality of machine-generated text. The best-performing model in terms of BLEU score was selected for further testing.

GraphCodeBERT introduces an additional complexity by incorporating a data flow graph, which provides a representation of the relationships between different variables in the code. This feature is especially useful in handling more complex multi-line vulnerabilities, as it allows the model to understand the dependencies between various parts of the code more effectively. (Fig. 3).

In terms of results, the pre-trained PL models performed exceptionally well, achieving a high repair accuracy, particularly on single-line vulnerability repairs where CodeBERTFix achieved 95.47% accuracy, and GraphCodeBERTFix achieved 94.04%. Importantly, these models also showed strong performance in the multi-line vulnerability repair task, with GraphCodeBERTFix achieving 90.06% accuracy and outperforming DLFix, which had little success with multi-line vulnerabilities. The use of data flow information in GraphCodeBERT proved to be a significant advantage in this regard, enabling the model to handle the complexity of multi-line repairs better than its counterparts.

While these results demonstrated the effectiveness of pre-trained PL models in vulnerability repair, the study also identified some limitations. CodeBERTFix and GraphCodeBERTFix sometimes produced syntactically incorrect fixes, such as missing or redundant braces, and struggled with excessively long code segments. These shortcomings highlight the need for further improvements, particularly in ensuring structural integrity and handling large-scale code problems. Nonetheless, the use of PL models, especially with the inclusion of data flow information, represents a promising direction for automating vulnerability repair.

VulDetect: The authors of [13] introduce, a classification model designed for automatic software vulnerability detection, leveraging the extensive language model GPT-2. VulDetect employs a fine-tuned GPT-2 model to identify vectors associated with vulnerable code segments extracted from the target source code. Their NLP vulnerability model accepts a lengthy character string as input, typically a C source file. In the subsequent step, a tokenizer partitions the string into individual words and sub-words. Notably, syntax characters such as periods, semicolons, parentheses, and

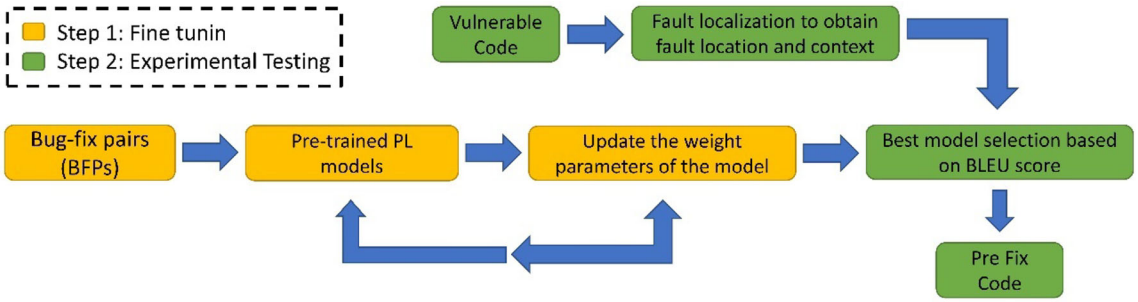

Fig. 3 Overview of vulnerability repair solution. Inspired by Fig. 1 in [87]

Table 4 Detection accuracy of GPT-2, CodeBERT, and LSTM using VulDetect technique on SeVC and SARD dataset

Dataset	Model	Detection accuracy
SeVC	GPT-2	87.63
	CodeBERT	83.23
	LSTM	71.59
SARD	GPT-2	92.59
	CodeBERT	89.28
	LSTM	76.86

Based on Table 1 in [13]

brackets are treated as distinct entities. Following this, the encoder transforms these words into vector representations. These vectorized words are then inputted into the model either sequentially (token by token) or in larger segments. In the implementation of this vulnerability detection technique, the output vector corresponds to the number of vulnerability classes present in the training dataset. Specifically, with 124 unique vulnerability classes, the output vector has a dimension of 124. This output vector is further processed through a Softmax function, which normalizes it into a probability distribution where the sum of all probabilities equals 1. Each element within the output vector indicates the predicted probability of the corresponding vulnerability class being present in the analyzed code file.

In the defense performance evaluation, the researchers chose to evaluate the effectiveness of their technique by conducting tests on three classifiers: GPT-2, CodeBERT, and Long Short-Term Memory (LSTM). These tests were performed using two standardized benchmark datasets: SARD and SeVC. The objective was to gauge the ability of their technique in detecting vulnerabilities. Results presented in Table 4 illustrate a notably higher classification accuracy when implementing the VulDetect technique. Notably, the GPT-2 classifier demonstrated the highest accuracy, reaching up to 92.59% when tested on the SARD dataset, while the LSTM classifier exhibited the lowest accuracy, achieving only 65.78% when evaluated on the SeVC dataset. Overall, GPT-2 consistently outperformed the other architectures (CodeBERT and LSTM), which is in line with its reputation as a transformer-based model renowned for achieving state-of-the-art performance in various linguistic tasks, including sentiment analysis and sentence classification.

The framework for the vulnerability repair solution presented in the paper is built around leveraging pre-trained programming language models such as CodeBERT and GraphCodeBERT to handle both single-line and multi-line security vulnerabilities in code. The process begins by constructing a dataset of vulnerability code paired with corresponding fixes, known as bug-fix pairs (BFPs). These BFPs serve as the training data for fine-tuning the pre-trained PL models.

VulD-transformer: VulD-Transformer [95], as shown in Fig. 4, is a comprehensive framework for source code vulnerability detection. It comprises four main components: the input module, code parser, vulnerability detector, and output module. Each part plays a crucial role in the detection process.

Input module: This module accepts the source code as input and initiates the detection process.

Code parser: The code parser processes the source code according to specific syntax rules to generate code slices, which are segments of code containing potential vulnerabilities. This process involves 4 steps:

1. Generating a Program Dependence Graph (PDG) using the Joern tool.
2. Identifying vulnerability candidates within the PDG based on predefined syntax rules.
3. Generating code slices from the identified vulnerability candidates.
4. Cleaning and normalizing the code slices to remove irrelevant characters and standardizing identifiers and variable names.

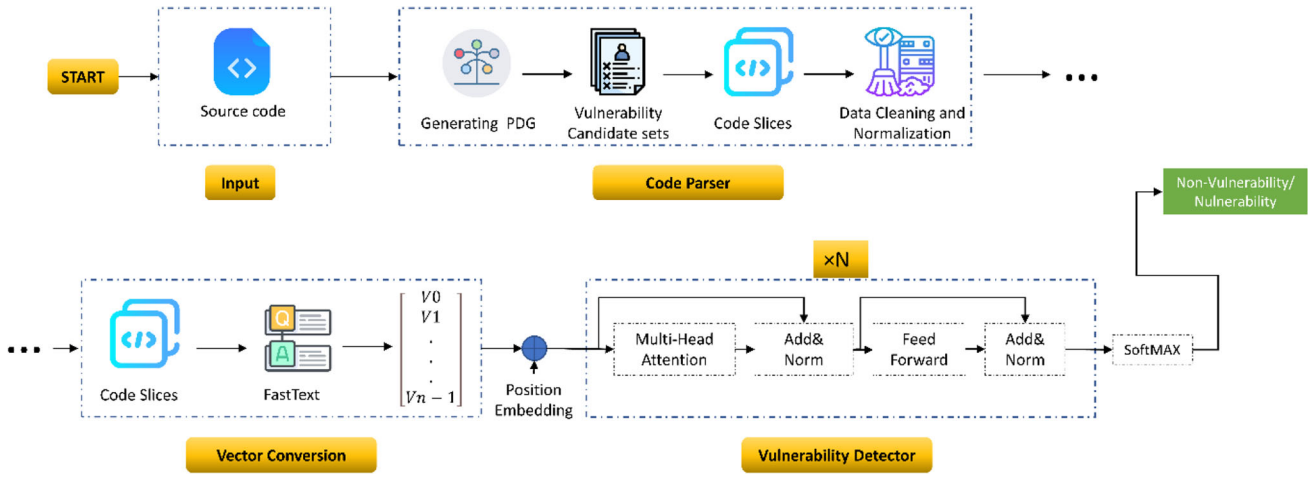

Fig. 4 Framework of VulD-transformer. Inspired by Fig. 1 and 2 in [95]

Vulnerability detector: The vulnerability detector assesses whether the code slices contain vulnerabilities. It includes two main operations:

- Vector conversion: Utilizing FastText, the code slices are converted into vectors. FastText's character-level embedding allows capturing subtle relationships between identifiers in the source code, enhancing the detection accuracy.
- Vulnerability detector: The vulnerability detector learns the vector representation of the code slices, obtains the global features of the code slice, and obtains the final vulnerability detection result at the last layer. The vulnerability detection model leverages the Transformer architecture to learn vector representations of the code slices. Only the encoding part of the standard Transformer structure is used in the vulnerability detection task in this model. The decoder, which belongs to the generative model often used in natural language generation, is not used.

This model comprises several key components:

- Position encoding: Adding positional information to the code slice vectors to preserve their sequential order.
- Multi-head attention mechanism: Capturing long-distance dependencies between code tokens by computing mutual attention.
- Feed-forward layer: Consisting of fully connected layers with ReLU activation to extract higher-level features from the code slice representations.
- Add & Norm layer: Adding and normalizing the output of the attention and feed-forward layers.
- Fully connected layer with softmax activation: Producing the final vulnerability detection results, categorizing code slices as either non-vulnerable or vulnerable.

In the Final step, the vulnerability detection results are presented to the user, indicating whether vulnerabilities are detected in the analyzed source code.

The proposed VulD-Transformer model demonstrated superior performance across several key metrics. In synthetic datasets: It achieved an average improvement of 6.12% in accuracy, 8.01% in recall, and 7.63% in F1-score compared to VulDeePecker, SySeVR-BGRU, SySeVR-ABGRU, and Russell, particularly in code slices longer than 256 tokens. In real vulnerability datasets (Devign and REVEAL): VulD-Transformer achieved detection accuracies of 59.34% and 80.44%, respectively, outperforming the comparison methods.

RQ5 Answer: *We showed that Large Language Models (LLMs) can be applied to software vulnerability detection in several ways. These include, but are not limited to, their integration into automated code repair frameworks for detecting vulnerabilities. LLMs can also be utilized as a code analyzer, significantly outperforming traditional tools by identifying more vulnerabilities and providing detailed fixes. Additionally, frameworks containing LLMs, like VulDetect, can demonstrate superior performance in classifying vulnerabilities with an accuracy of up to 92.59%. LLMCs can also be applied to both single-line and multi-line vulnerability repairs, and achieve great results. In conclusion, Large Language Models (LLMs) possess the capacity to be integrated into existing software vulnerability detection framework or to serve as the central component of new ones.*

8 RQ6: How does fine-tuning LLMs affect their performance in software vulnerability detection and repair?

Fine-tuning means adapting a pre-trained model, originally trained on a large general dataset, for a new or related task. This is done by continuing the training on a smaller, task-specific dataset. Fine-tuning allows us to leverage the model's existing knowledge and build upon it, resulting in faster training times and enhanced performance.

If the data used for the initial pre-training of the model is closely related to the dataset chosen for specific task at hand, it can significantly enhance the model's ability to understand and perform that task. For example, if we wish to fine-tune a model for software vulnerability detection, and the model was initially trained on large corpora of domain-specific data—such as a code-based dataset like GitHub Code—it already possesses a solid foundation of relevant knowledge. This allows the model to better identify and repair vulnerabilities, making the subsequent fine-tuning process more efficient and effective [96].

Additionally, not all domains have large datasets available for pretraining a model from scratch and creating such datasets can be time and resource consuming. So, an effective solution is to try to gain the most benefit from a smaller dataset. But the question remains: does fine-tuning always result in improvements? To answer this question, we will analyze a few studies where the effect of fine-tuning LLMs for software vulnerability detection or mitigation was tested.

To address this question, we will conduct a literature review of academic papers that explicitly test the impact of fine-tuning on LLMs or discuss this impact through the analysis of other papers.

Ferrag et al. [75] fine-tuned the FalconLLM base model to create SecureFalcon, a specialized model for detecting software vulnerabilities in C code. This fine-tuning process significantly improved the model's accuracy, achieving a 94% success rate in identifying vulnerabilities. The fine-tuning addressed common issues in deep learning approaches, such as model inadequacy and data imbalance, by carefully pre-processing the data and leveraging the strengths of FalconLLM. The study utilized a new dataset, FormAI, which was crucial for training and evaluating the model. Additionally, the fine-tuned model was integrated into a system that not only detected vulnerabilities but also suggested repairs, providing valuable insights for developers.

Huang et al. [34] conducted a study to comprehensively evaluate the program repair capabilities of LLMCs within the fine-tuning paradigm. The researchers aimed to explore whether LLMCs could effectively repair more complicated bugs, such as multi-hunk bugs, which had not been extensively studied before. They selected five popular LLMCs—CodeBERT, GraphCodeBERT, PLBART, CodeT5, and UniXcoder—and tested them on typical program repair scenarios involving bugs, vulnerabilities, and errors across Java, C/C + + , and JavaScript programming languages.

The authors utilized diverse datasets and carefully selected methodologies to fine-tune the LLMCs for APR. For fine-tuning, they relied on widely-used bug-fix pair (BFP) datasets, including BFP-small (BFP S), BFP-medium (BFP M), and SequenceR, which provide method-level buggy code and corresponding fixes. These datasets encompass a variety of single-hunk defects, enabling the models to learn from a substantial volume of curated examples. For multi-hunk bug repair, the study employed the CPatMiner dataset. This dataset specifically includes multi-hunk defect scenarios, requiring the models to account for intricate dependencies across multiple buggy locations. To address vulnerabilities, the VulRepair dataset was used, comprising security-related defects in C/C + + drawn from sources like Big-Vul and CVE-fixes. For programming errors, the TFix dataset was utilized, which contains annotated JavaScript error-fix pairs.

The study results demonstrated that fine-tuning LLMCs significantly enhances their capabilities in program repair, particularly in addressing more complex defects such as multi-hunk bugs and security vulnerabilities. The fine-tuning approach allowed LLMCs to outperform existing state-of-the-art APR tools across various benchmarks, including Defects4J and VulRepair datasets.

The study's findings are substantiated by strong numerical evidence, demonstrating the effectiveness of fine-tuned LLMCs in addressing software vulnerabilities and bugs. For example:

Improved Vulnerability Repair Accuracy: On the VulRepair dataset, CodeT5 achieved a repair accuracy of 64.71%, which is a 20.04% improvement over the baseline tool VulRepair's 44.67%. Other LLMCs, such as UniXcoder (63.77%) and PLBART (60.90%), also significantly outperformed VulRepair.

Multi-Hunk Defect Performance: For complex multi-hunk bugs on Defects4J V1.2, UniXcoder fixed 25 more bugs than DEAR, while CodeT5 outperformed it by repairing 24 more bugs. This highlights the significant advances fine-tuned LLMCs have made in handling complex defects.

Benchmark Results for Error Repair: On the TFix dataset, CodeT5 achieved an error repair accuracy of 60.62%, improving by 11.32% over the baseline TFix (49.30%). Similarly, UniXcoder reached an accuracy of 60.10%, showcasing consistent performance across models.

Overall, the paper highlights the transformative impact of fine-tuning on LLMCs' capabilities in detecting and mitigating software vulnerabilities, while also providing empirical guidance for future improvements in the field.

Yang et al. [97] paper investigates the impact of fine-tuning LLMs for detecting and mitigating software vulnerabilities. The study introduces a method called Multitask Self-Instruct

Vulnerability Detection (MSIVD), which combines multitask sequence-to-sequence fine-tuning with graph neural networks (GNNs) to enhance the model's ability to identify vulnerabilities. MSIVD significantly outperforms existing methods and demonstrates promise in addressing unseen vulnerabilities.

The MSIVD approach focuses on multitask fine-tuning, leveraging both code snippets and vulnerability explanations. This fine-tuning strategy improves the model's generalization and accuracy, achieving a remarkable F1 score of 0.92 on the BigVul dataset, compared to the baseline LineVul's F1 score of 0.81. Additionally, the MSIVD model attains a precision of 0.93 and a recall of 0.91, highlighting its balanced performance in identifying both vulnerable and non-vulnerable samples. These metrics demonstrate MSIVD's efficacy, particularly when compared to the recall of 0.78 and precision of 0.86 achieved by LineVul on the same dataset. The MSIVD approach demonstrates superior performance compared to non-LLM methods as well, with the best non-LLM results achieved by DeepDFA (a deep learning-based approach for detecting function-level vulnerabilities in source code) with an F1 score of 0.67, precision of 0.54, and recall of 0.90.

When evaluated on the novel PreciseBugs dataset, designed to mitigate issues of data leakage by excluding vulnerabilities present in the training data of modern LLMs, MSIVD achieves an F1 score of 0.48, outperforming LineVul, which only manages a score of 0.31. This improvement is particularly significant given the challenges posed by PreciseBugs, which includes vulnerabilities released after January 2023 and features a more realistic distribution of vulnerability types. For instance, PreciseBugs has a mean sample size of 377 lines of code, in contrast to BigVul's 30 lines per sample, making it a more challenging dataset for vulnerability detection.

The study highlights the critical role of vulnerability explanations in the fine-tuning process. Loss curve analysis reveals that models trained with explanations exhibit significantly more stable training dynamics and avoid overfitting, even on datasets like BigVul, where data contamination from pre-training is a concern. Similarly, on the PreciseBugs dataset, MSIVD with multi-round self-instruct fine-tuning achieves superior generalization, reducing overfitting to pre-existing patterns in the dataset.

The improvements in F1 score underscore the effectiveness of combining vulnerability explanations with multitask fine-tuning. This approach ensures that the model not only learns to detect vulnerabilities but also gains a deeper understanding of their causes and potential fixes, making it better suited for deployment in real-world scenarios where new and unseen vulnerabilities frequently emerge.

Guo et al. [94] conducted a through experiment to analyze the capabilities of LLMs is software vulnerability detection. They conducted their experiments on six open-source models that are specifically trained for vulnerability detection against six general-purpose LLMs (CodeBERT-base, Mistral-7b-base, Mixtral-8 × 7b-base, CodeLlama 7B & 13B, and GPT-4-base).

The analysis highlights the contextual benefits of fine-tuning in vulnerability detection tasks. Fine-tuned models often outperform their pre-trained counterparts on specific datasets. For instance, while state-of-the-art models generally achieved F1-scores ranging from 30 to 50% across balanced and unbalanced datasets, all fine-tuned models in the study performed significantly better on the authors' custom dataset. Among these, CodeLlama-7b-fine-tuned achieved an impressive 97% F1-score, demonstrating the transformative potential of fine-tuning when tailored to specific data contexts.

Fine-tuning also shows promise in optimizing smaller, resource-efficient models to achieve comparable or superior performance relative to larger models. This is particularly evident when fine-tuning smaller models like CodeBERT-fine-tuned or VulBERTa-based variations, which achieved competitive results without the computational overhead of larger commercial LLMs. This trade-off enables the application of low-resource-demanding models in constrained environments, emphasizing the importance of making model weights publicly available for task-specific fine-tuning.

The quality of the datasets used for fine-tuning is a critical factor. The authors observed that fine-tuned models showed significant variability in performance depending on the dataset used, revealing potential issues like mislabelling and dataset inconsistencies. For instance, datasets like Choi2017 and LineVul demonstrated imbalances or labeling errors that could undermine the benefits of fine-tuning. The authors validated these issues using the Semgrep tool, which faced difficulties in reliably scanning code snippets, reinforcing the need for better-curated datasets to maximize fine-tuning effectiveness.

While fine-tuning improves performance on specific tasks, the models often fail to generalize across diverse testing environments. This limitation is tied to the training data quality and diversity, as models trained on narrow datasets tend to exhibit unexpected behavior and classification errors when applied to new contexts. Addressing these challenges requires robust data curation and benchmarking to ensure that fine-tuned models maintain reliability beyond their training domains.

The authors propose exploring strategies like quantization and reducing the number of parameters in smaller LLMs to create resource-efficient, fine-tuned models suitable for constrained environments. They also emphasize the need for

systematic dataset improvements to mitigate issues like mislabeling, ensuring that models trained via fine-tuning can generalize more effectively across tasks. These efforts aim to enhance the practical applicability of LLMs in software vulnerability detection.

Shestov et al. [98] examines the effects of fine-tuning LLMs for software vulnerability detection, specifically in Java source code. The researchers focused on the adaptation of WizardCoder, a state-of-the-art code LLM based on StarCoder. Their approach included optimizing training regimes and addressing challenges such as class imbalance, which is common in real-world datasets.

The authors used datasets from sources like CVEfixes, a manually curated Java vulnerability dataset, and VCMatch. The final datasets were partitioned into X1 without easy negatives or P3 (samples that are straightforward for the model to classify as negative because they lack complexity or ambiguity) and X1 with P3, each tailored to evaluate the models' performance under balanced and imbalanced conditions. The balanced dataset (X1 without P3) consisted of 1,334 samples, with an equal distribution of positive and negative classes, whereas the imbalanced dataset (X1 with P3) had a severe positive-to-negative class ratio of 1:34, reflecting the real-world challenge.

The research used WizardCoder, a 13-billion parameter transformer model pretrained on GitHub source code. It was fine-tuned for vulnerability detection using LORA (settings: r = 8, alpha = 32, dropout = 0.05). The methodology involved fine-tuning pre-trained models by optimizing key hyperparameters such as batch size, learning rate, and number of epochs, using training and validation datasets, and evaluating the best model on the test dataset. This reduced the 13 billion parameter models to just 25 million parameters, smaller than CodeBERT. The results validate LORA's effectiveness for LLM adaptation under memory limitations.

The experimental results on the balanced dataset (X1) demonstrate that the WizardCoder outperforms the ContraBERT model in the vulnerability detection task. The results, show that WizardCoder surpasses ContraBERT in both ROC AUC and F1 scores, with values of 0.69 and 0.71 respectively, compared to ContraBERT's 0.66 and 0.68. This enhanced performance of WizardCoder can be attributed to its larger model capacity of 13 billion parameters and pre-training on a significantly larger code corpus, which allows it to capture more nuanced patterns and vulnerabilities in the code.

For the imbalanced dataset, WizardCoder maintained its superiority with a ROC AUC of 0.86 and an F1 score of 0.27, compared to ContraBERT's 0.85 and 0.22. However, the margin of improvement was smaller, partly due to the challenges posed by the highly skewed class distribution. Further enhancements using focal loss and sample weighting yielded modest gains, with the best ROC AUC reaching 0.878 under specific hyperparameter configurations.

de-Fitero-Dominguez et al. [99] paper presents a study on the enhancement of automated code vulnerability repair using fine-tuned LLMs, specifically Code Llama and Mistral, targeting C/C + + vulnerabilities. The research highlights significant advancements in the field of automated vulnerability repair by demonstrating the effectiveness of these fine-tuned LLMs over traditional methods and other state-of-the-art approaches like VulRepair and VulMaster.

The models were fine-tuned using two prominent datasets, Big-Vul and CVEFixes, which were curated from CVE databases and span a wide array of vulnerabilities. Together, they offered a comprehensive pool of over 9,000 samples. The study emphasizes addressing data contamination by creating a refined dataset with no overlap between training and test sets, ensuring a realistic evaluation of the model's generalization capabilities.

Fine-tuning employed advanced techniques such as Quantized Low Rank Adaptation (QLORA), combining 4-bit quantization with trainable low-rank decomposition matrices, and NEFTune, which added noise to embedding vectors to regularize training and improve generalization.

The evaluation focused on the "Perfect Predictions" metric, which considers a model output correct only if it exactly matches the reference solution. Results revealed that Mistral consistently outperformed Code Llama, particularly in challenging scenarios with unseen vulnerabilities. On the original dataset, Code Llama achieved accuracies of 37% (beams 1) and 49% (beams 5), while Mistral achieved higher accuracies, with 49% (beams 1) and 57% (beams 5). On the refined dataset, which eliminated data overlap, Code Llama's accuracy dropped to 16% (beams 1) and 23% (beams 5), In contrast, Mistral, while also experiencing reduced performance, outperformed both Code Llama and other benchmarks, achieving 18% (beams 1) and 26% (beams 5).

Notably, for critical vulnerability types like "Out-of-bounds Write" (CWE-787) and "Use After Free" (CWE-416), Mistral demonstrated significant improvements:

- CWE-787: 15/53 cases fixed, compared to VulRepair's 3/58 and VulMaster's 12/58.
- CWE-416: 19/55 cases fixed, significantly outperforming VulRepair's 1/60 and VulMaster's 6/60.

The study's findings indicate that fine-tuning LLMs on targeted datasets significantly enhances their ability to address vulnerabilities. The refined dataset approach ensured a realistic and robust evaluation, highlighting potential overestimations in previous studies due to data overlap. Furthermore, Mistral's superior performance underscores the impact of fine-tuning and innovative training methods on vulnerability repair tasks.

RQ6 Answer: *The findings suggest that fine-tuning LLMs significantly enhances their performance in software vulnerability detection and repair by leveraging pre-trained knowledge and adapting it to specific tasks. Studies demonstrate that fine-tuned models like SecureFalcon, MSIVD, CodeT5, and Mistral outperform baseline tools and pre-trained models in accuracy and efficiency across various benchmarks.*

Fine-tuning also enables smaller, resource-efficient models to achieve competitive results and improves generalization to unseen vulnerabilities, provided datasets are curated to mitigate issues like data contamination. Advanced techniques like QLORA and NEFTune further enhance model adaptation, enabling practical and scalable applications in vulnerability mitigation.

9 RQ7: What techniques are used to fine-tune LLMs for software vulnerability detection?

Pre-trained models typically possess a comprehensive grasp of language and context. Nonetheless, they often need specialized expertise to perform exceptionally well in particular domains. Fine-tuning usually costs less than pre-training but still demands considerable resources for the best outcomes. This has sparked a growing interest in parameter-efficient fine-tuning methods. Fine-tuning the LLMs comes with two primary challenges: the considerable cost associated with the fine-tuning process and the "knowledge cut-off" issue. Post fine-tuning, models remain unaware of new threats or vulnerabilities that arise after their last update. This limitation often necessitates recurrent fine-tuning sessions to ensure the models remain up-to-date and effective [10].

In this section, we will explore some of the fine-tuning techniques commonly employed by researchers to enhance the performance of LLMs in software vulnerability detection and mitigation tasks. These techniques play a crucial role in adapting pre-trained models to the specific requirements of identifying and addressing software vulnerabilities. There are six fundamental approaches to fine-tuning LLMs, and in this section we will explore each approach in brief (Fig. 5).

9.1 Full fine-tuning

Full fine-tuning involves updating all the parameters of a pre-trained LLM to adapt it to a new, specific task. During this process, the entire model is trained on the new data, allowing it to learn representations relevant to the task at hand. Full fine-tuning provides maximum flexibility, as every layer of the model is optimized for the new task. However, it requires significant computational power and a lot of memory and processing power. Additionally, it may lead to *catastrophic forgetting* [100], where the model loses some or most of its pre-trained knowledge as it becomes too specialized in the new data [101]. Few-shot fine-tuning can also be considered a variant of full Fine-tuning, as it's a specific scenario where full fine-tuning is applied but with limited examples.

Hu et al. [102] identifies a challenge in adapting LLMs for specific tasks. Language models like GPT-3 are pre-trained on vast datasets to predict sequences of text but require fine-tuning to perform well on specialized tasks, such as summarizing text or converting natural language into Structured Query Language (SQL) commands. Fine-tuning adjusts the model's parameters for each new task, which can be difficult with models containing billions of parameters. This makes storing and deploying multiple versions of these models difficult to manage. To address this, the authors suggest using a low-rank approach to simplify the adjustments, reducing the number of parameters that need updating.

Due to the significant resource demands associated with full fine-tuning, this approach is rarely employed by researchers for adapting models to software vulnerability detection and mitigation tasks. Instead, many researchers often choose more resource-efficient fine-tuning methods that allow for effective adaptation without the need for extensive computational resources.

9.2 Feature extraction

The feature extraction approach treats a pre-trained LLM as a fixed tool to extract useful information, keeping its core parameters unchanged during fine-tuning. Only the final layer or a small part of the model (often a task-specific classifier) is trained on the new task data. This method leverages the generalized knowledge the model has already learned, using it to analyze new inputs without altering the main model. Feature extraction is computationally efficient and requires far fewer resources than full fine-tuning. It's particularly effective when the new task is similar to what the model was originally trained on. This makes it a popular choice for tasks like text classification or sentiment analysis, where large amounts of data are not required for adaptation [65, 103].

9.3 Multi-task learning (MTL)

This method trains the LLM on multiple related tasks simultaneously, such as vulnerability detection, explanation, and repair, aiming to not only identify vulnerabilities but also provide explanations for them. This approach can improve the model's performance across different but related tasks [36, 97, 104]. MTL is inspired by human learning, where knowledge gained from one task often helps in learning another. In the context of machine learning, MTL works by sharing

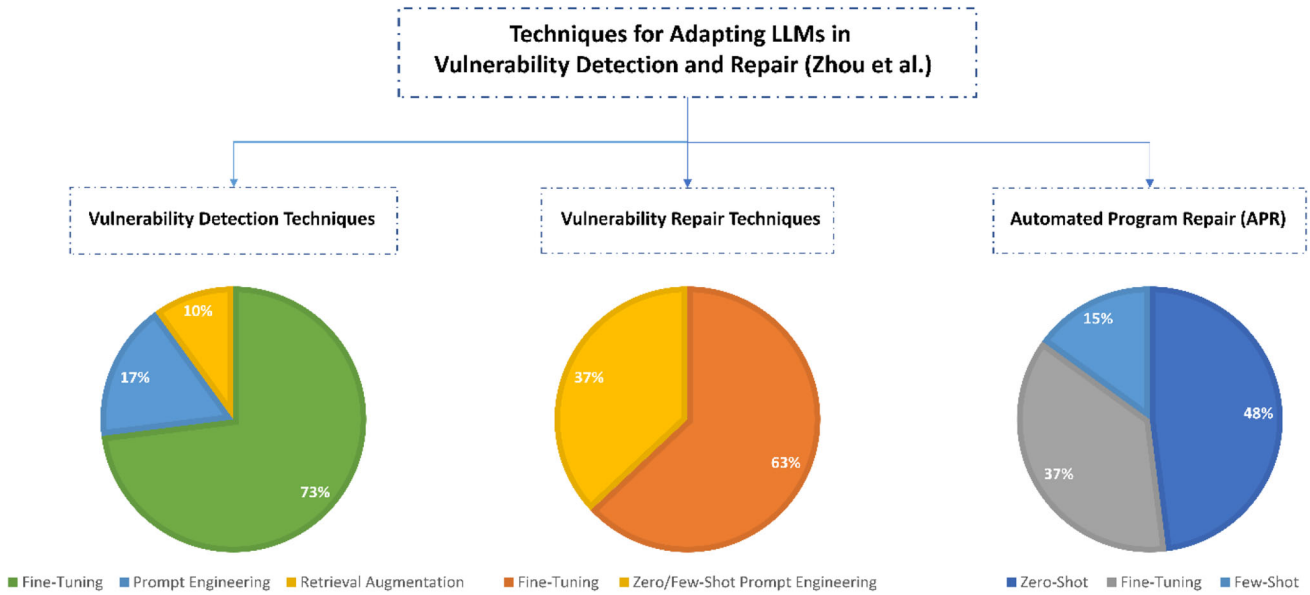

Fig. 5 Popular techniques for adapting LLMs in vulnerability detection and mitigation. Based on the study done by Zhou et al. [96]

information between tasks, allowing the model to generalize better. By optimizing multiple loss functions, which are mathematical formulas that evaluate how well a model's predictions match the actual or expected outputs, MTL allows models to learn from the unique information in each task. Each loss function represents a specific objective that the model needs to improve on, such as minimizing errors in classification, generating accurate text, or identifying patterns in data. By focusing on multiple objectives at once, MTL helps the model learn useful information from different tasks, each related to a specific domain. This approach is particularly valuable for LLMs dealing with a wide range of language tasks [104]. A key motivation behind MTL was to address the issue of having limited labeled data for each task. In such scenarios, the available labeled data for each task is inadequate to train a precise model. MTL combines the labeled data from all tasks, similar to data augmentation, to create a more accurate model for each task. This approach allows for the reuse of existing knowledge and helps lower the cost of manual labeling for various learning tasks [105].

MTL is linked to various machine learning methods, such as transfer learning, multi-label learning, and multi-output regression. However, unlike transfer learning—which focuses on improving one main task using knowledge from other tasks—MTL treats all tasks equally and aims to improve performance across all of them. Transfer learning transfers information in one direction—from source tasks to the target task— but MTL promotes two-way knowledge sharing across all tasks [105].

MTL can cover different but related fields, learning styles (e.g., supervised and unsupervised tasks), objectives (e.g., classification or text generation), and multiple feature levels (e.g., token-level and sentence-level features). MTL can also include tasks involving different types of data, like text and images. Additionally, performing the same task across multiple domains or languages can be treated as multiple tasks, a concept referred to as multi-domain learning in some literature, which can serve as a basis for developing an MTL model [106].

9.4 Parameter-efficient fine-tuning

In the current body of literature, Parameter Efficient Fine-Tuning (PEFT) has emerged as a crucial topic, focusing on minimizing the number of trainable parameters while maintaining optimal performance [23, 27]. Ding et al. [107] show that, even with a significant reduction in tunable parameters, most parameter-efficient fine-tuning methods generally achieve performance comparable to full fine-tuning. These findings underscore the potential for effectively adapting large-scale PLMs through parameter-efficient tuning techniques [107]. The four main techniques for parameter-efficient fine-tuning transformer language models includes: adapter tuning, prefix tuning, prompt tuning, and Low-Rank Adaptation (LoRA).

9.4.1 Adapter-based fine-tuning

Adapter-based fine-tuning, inserts small, trainable adapter layers within the larger pre-trained model, keeping the bulk of the model frozen. These adapters are the only parts updated during fine-tuning, which significantly reduces the number of parameters that need to be optimized. Adapter layers are small task-specific modules that adapt the pre-trained models

to the new task without altering the original model. This approach is highly efficient, both in terms of computational cost and memory usage, making it a good option for tasks where fine-tuning the entire model is not feasible. Adapters can be inserted in parallel or sequentially within the model's architecture, allowing for flexibility in task adaptation. This method also addresses the problem of catastrophic forgetting in fine-tuning [23, 27, 108].

9.4.2 Prefix-tuning and prompt-tuning

Prefix-tuning and prompt-tuning are lightweight and parameter-efficient fine-tuning approaches, where only the input space of the model is adjusted, rather than its internal parameters.

Prefix tuning: Prefix tuning is a lightweight fine-tuning approach that leaves most of the pre-trained model unchanged and adjusts only a small set of additional parameters. It works by adding task-specific trainable vectors, called prefixes, to each layer of a Transformer-based language model. These prefixes act like extra tokens that guide the model to focus on task-relevant information. Instead of updating the original model parameters, prefix-tuning optimizes these newly added vectors, which are initialized separately and trained while keeping the main model frozen. During text generation, the prefixes provide task-specific adjustments, while the rest of the model processes input as usual. This method allows for efficient task adaptation with minimal computational and memory overhead [27, 107, 109].

An optional multi-layer perceptron (MLP) can be used to transform a smaller matrix into the prefix parameters, improving both efficiency and stability during training. Once training is complete, only the optimized prefix vectors are kept, allowing the model to perform task-specific functions with minimal added parameters. Prefix tuning is highly versatile, working well with both autoregressive and encoder-decoder models, and can enhance performance on natural language understanding tasks across models of all sizes [27, 107, 109].

Xiang Lisa Li and Percy Liang [109] demonstrated that updating just 0.1% of task-specific parameters via prefix-tuning is highly effective for table-to-text generation tasks. Their results show that prefix-tuning not only outperformed alternative lightweight fine-tuning approaches, such as ADAPTER and FT-TOP2, but also required approximately 30 times fewer parameters while maintaining similar performance to full fine-tuning. This advantage was consistent across various datasets, including E2E, WebNLG, and DART. When the parameter counts were matched (at 0.1%) between prefix-tuning and ADAPTER, prefix-tuning delivered an average BLEU score improvement of 4.1 points per dataset. Even against full fine-tuning and ADAPTER at 3%, prefix-tuning showed comparable or superior results. This illustrates the Pareto efficiency of prefix-tuning, which achieves higher generation quality with fewer parameters.

Prompt tuning: Prompt tuning is a method distinct from traditional fine-tuning or prefix tuning because it focuses on optimizing trainable prompt embeddings at the input layer of a language model. Rather than adjusting the model's internal layers, prompt tuning involves learning continuous embeddings (known as "soft prompts") that are prepended to the input tokens. These learned embeddings guide the model's behavior for specific tasks without changing its core parameters. This approach offers significant advantages in terms of parameter efficiency. Only a small set of trainable parameters is added at the input layer, reducing computational costs compared to full fine-tuning, where all model parameters are updated. This makes prompt tuning particularly suitable for large-scale models, as smaller parameters allow for efficient training while maintaining performance. Additionally, prompt tuning has shown competitive results, especially with large models. For example, experiments with T5-XXL on the Super General Language Understanding Evaluation (SuperGLUE) benchmark demonstrate that models using soft prompts can achieve performance comparable to fully fine-tuned models. This scalability suggests that larger language models can effectively utilize prompt tuning to adapt to diverse tasks with minimal parameter changes [27, 107].

Prompt tuning is also sensitive to factors such as the length and initialization of the soft prompts. Careful initialization can significantly enhance the model's performance on downstream tasks, suggesting that the initial configuration of the prompt embeddings plays a crucial role in guiding the model's learning process. Optimal initialization can lead to faster convergence and better overall accuracy.

Another key advantage of prompt tuning is its transferability across tasks. Once soft prompts are effectively initialized, they can often be adapted to new tasks with minimal adjustments, making the approach versatile and efficient. This characteristic is particularly beneficial in scenarios where models need to be deployed across multiple applications without extensive retraining. However, the success of prompt tuning is closely linked to the model's capacity. Larger models tend to leverage soft prompts more effectively, resulting in better task-specific performance. This is because larger models possess a greater ability to capture complex patterns and relationships from the soft prompts, enhancing their adaptability to various tasks [107].

9.4.3 Low-rank adaptation (LoRA)

Low-rank adaptation (LoRA) is an advanced fine-tuning technique designed to enhance the efficiency of adapting LLMs to specific tasks. In neural networks, weight matrices are the core components that encode learned representations. Standard fine-tuning involves updating all the parameters in

these matrices, which is computationally expensive for large models. LoRA addresses this challenge by representing the necessary updates to the weight matrices using low-rank approximations. Instead of directly modifying the weight matrices during fine-tuning, LoRA decomposes the weight updates into the product of two smaller, trainable matrices (denoted as AA and BB) for each dense layer in the Transformer model. The original model parameters remain frozen, and only these small matrices are optimized, significantly lowering computational overhead and memory requirements while preserving task-specific adaptation. This approach is particularly advantageous for scenarios requiring fine-tuning of large-scale models but facing resource constraints, striking a balance between computational efficiency and maintaining task performance [27, 102, 108].

LoRA has been extensively employed for parameter-efficient fine-tuning of open-source LLMs such as LLaMA and BLOOM. Within these research endeavors, LLaMA and its variants have garnered significant attention for their parameter-efficient tuning capabilities. For instance, Alpaca-LoRA has been fine-tuned using LoRA to create a lightweight version of Alpaca [27]. The researchers in [98] applied the LoRA method to the WizardCoder model using specific settings. This process significantly reduced the model size from 13 billion parameters to just 25 million parameters, making it even smaller than CodeBERT. The experiment validates the effectiveness of LoRA in adapting LLMs under memory constraints. It shows that LoRA can drastically reduce the number of parameters while maintaining performance, making it a viable option for resource-limited environments.

In LoRA, only two small matrices are trained for each dense layer in the Transformer model, while the original model parameters remain frozen. These matrices approximate the adjustments needed to adapt the model to a specific task, significantly reducing the total number of parameters that need to be updated—by up to 10,000 times compared to full fine-tuning [102]. This reduction lowers memory and computation costs during training, making it easier to fine-tune large models on specialized tasks. Additionally, LoRA does not add any extra inference latency. Once fine-tuning is complete, the low-rank matrices can be merged with the pre-trained weights, allowing the model to behave like a fully fine-tuned model during inference. This characteristic is crucial for deploying LLMs in latency-sensitive environments, as it avoids the performance overhead commonly associated with methods like adapter layers [102].

RQ7 Answer: *Fine-tuning techniques for LLMs in software vulnerability detection include full fine-tuning, feature extraction, multi-task learning, and parameter-efficient methods like adapter-based tuning, prefix-tuning, prompt-tuning, and LoRA (Low-Rank Adaptation). These methods vary in computational cost and adaptability, with parameter-efficient techniques offering resource-saving alternatives while maintaining performance.*

10 RQ8: What datasets are used to train, fine-tune, or test LLMs for software vulnerability purposes?

To provide a comprehensive and practical answer to RQ7, we compiled a detailed list of datasets commonly utilized in this domain. We focused on code-based models and included only *publicly available datasets* that are accessible to all researchers. The only exception is the REFINEDWEB dataset [21], which is exclusively text-based. However, due to its relatively small size and the strong performance of the FalconLLMs model trained on this dataset, we chose to include it in our report. These datasets include both code-based and text-based data, each playing a crucial role in training and fine-tuning LLMs to detect, predict, and mitigate software vulnerabilities. For detecting and handling software vulnerabilities with LLMs, a combination of text-based and code-based datasets tends to be most effective [16]. Text-Based datasets are crucial for tasks involving bug fixing, code comprehension, and understanding textual content related to vulnerabilities. They aid LLMs in grasping the context around security issues, identifying patterns in security-related texts (such as bug reports or security advisories), and enhancing their ability to generate secure code or identify vulnerabilities in code [16, 28, 75]. Code-Based datasets Essential for LLMs to comprehend and analyze code, especially for identifying vulnerabilities within the codebase itself. This type of dataset helps LLMs understand the structure, logic, and potential flaws in software code, enabling them to identify vulnerabilities, suggest fixes, or even generate secure code [16, 92].

By combining these datasets, LLMs can learn to correlate textual information (like security advisories or bug reports) with the actual code, which is crucial in cybersecurity. Understanding the context within which vulnerabilities are reported and how they manifest in code allows LLMs to provide more comprehensive and accurate support in detecting, addressing, and potentially preventing software vulnerabilities.

Figure 6 show the complete list of datasets that will be introduced in the following sections. We will divide this section into two parts. In the first part, we will introduce the primary and large datasets that are ideal for pre-training foundation models. These datasets are typically vast and cover a wide range of domains, making them well-suited for providing a solid foundation for LLMs. They enable models to learn diverse patterns, both in general software development and other tasks, and they are often used in early stages to

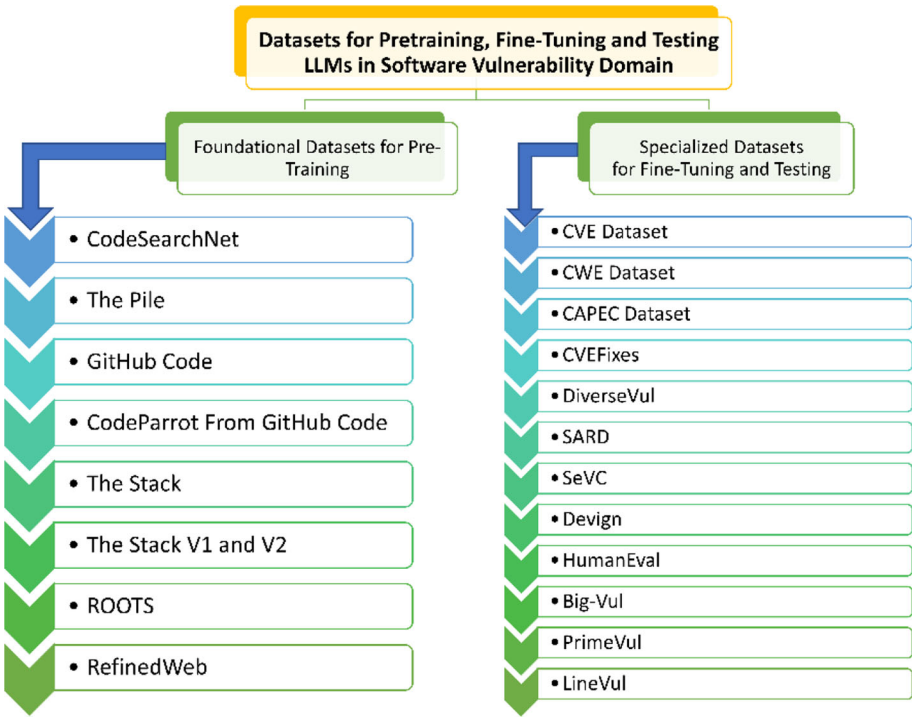

Fig. 6 Important datasets used for training and fine-tuning LLMs for software vulnerability detection and mitigation

develop the core language understanding and code analysis capabilities required by LLMs.

In the second part, we will introduce datasets that are more specialized and are particularly useful for fine-tuning LLMs for software vulnerability tasks. The datasets in this section are relatively smaller and more focused compared to those in the first part. They concentrate on specific vulnerabilities, code security issues, and threat patterns, making them crucial for optimizing LLMs for precise tasks such as vulnerability detection, threat analysis, and secure code generation. Fine-tuning with these datasets allows LLMs to gain a deeper and more nuanced understanding of vulnerabilities, enabling them to handle specialized security challenges more effectively.

This structure ensures that the models trained with large, diverse datasets can later be refined with domain-specific data to address the intricacies of software security. By differentiating between the general-purpose datasets for pre-training and the specialized datasets for fine-tuning, we aim to provide a clear pathway for improving the performance of LLMs in the domain of software vulnerability detection.

We have structured the explanation of each dataset to include key details such as what it contains, its specific relevance to software vulnerability detection, and the main associated academic reference (if available). Additionally, direct links to repositories and sources are provided to assist researchers, developers, and cybersecurity professionals in accessing these resources quickly. This level of detail and accessibility helps streamline further exploration and usage of these datasets.

10.1 Part 1: Foundational datasets for pre-training large language models of code (LLMCs)

In this section, we introduce the foundational datasets that are widely used for pre-training LLMs in coding tasks. These datasets, due to their substantial size and diversity, are crucial for building the basic language comprehension and code generation abilities of LLMs. Pre-training on such datasets allows models to learn general patterns in coding and language, making them versatile enough to be later fine-tuned for more specific tasks such as software vulnerability detection.

The following table provides a summary of the most important details about each dataset used for pre-training. Each dataset will be introduced in more detail after the table (Table 5).

10.1.1 CodeSearchNet

CodeSearchNet is a dataset created to facilitate the training and evaluation of models for semantic code search, specifically focusing on the task of retrieving relevant code given natural language queries. This dataset contains 6,452,446 functions collected from open-source GitHub repositories in six different programming languages: Python, Java, JavaScript, PHP, Ruby, and Go. Alongside these code

Table 5 Publicly available foundational datasets for pre-training LLMs in code-related tasks

Dataset name	Size	Dataset type	Open/close access	Programming languages covered	Content	Year introduced	Primary use	References
CodeSearchNet	20 GB	Text-based and Code-based	Open access	Python, JavaScript, Go, Java, Ruby, PHP	Code paired with natural language comments	2019–2020	Code Retrieval, Natural Language Queries, Training and Evaluation, Improvement of semantic code search	[110]
The Pile	825 GB	Text-based and Code-based	Open access	include programming-related content (Multiple programming languages)	Consists of 22 diverse, high-quality text subsets	2020	General-purpose pre-training of LLMs and improvement of model generalization capabilities	[111]
GitHub Code	300GB compressed text data, up to ~ 1TB uncompressed	Code-based	Open access	Multiple programming languages (32)	115M code files from GitHub in 32 programming languages with 60 extensions	2022	Large-scale code pre-training, Software Vulnerability Detection, Code Analysis and Synthesis	–

Table 5 (continued)

Dataset name	Size	Dataset type	Open/close access	Programming languages covered	Content	Year introduced	Primary use	References
CodeParrot From GitHub Code	180 GB	Code-based	Open access	Python	20 million Python code and scripts	2022	Pre-training of large language of code specifically for Python	[70]
The Stack	3.1 TB	Code-based	Conditional open access; compliance with original licenses and terms required	Multiple programming languages (30 +)	Code from GitHub repositories	2022	General-purpose pre-training for LLMs	[112]
The Stack V1 and V2	6.4TB for V1 67.5TB for V2	Code-based	Conditional open access; compliance with original licenses and terms required	Suport of over 358 programming languages in V1 and 619 in V2	contains over 3B files in 600 + programming and markup languages	2024	Enhancing or pretraining foundation models for code	[113]
ROOTS	1.6TB	Text-based and Code-based	Open access	13 programming language with Java, PHP, and C + + accounting for more than half of all documents	46 natural languages and 13 programming languages	2023	Advances large-scale language model pretraining	[114]
RefinedWeb	1.68 TB	Text-based	Open access	Primarily English. focuses on natural language processing	600 billion token English dataset	2023	Pretraining LLMs with high-quality web data	[21]

snippets, the dataset includes automatically generated natural language documentation for 2,326,976 functions, pairing code with descriptive text to mimic real-world code search scenarios.

CodeSearchNet is valuable for training LLMs in understanding code structures and mapping them to natural language descriptions. This capability can be leveraged in tasks like detecting software vulnerabilities by interpreting queries related to security flaws and retrieving relevant code patterns that exhibit vulnerabilities.

Key components:

- Data Splits: Three splits are available: Train, Test, Valid.
- Languages: Python, Java, JavaScript, PHP, Ruby, Go.
- Content: Cde examples at the function level paired with natural language comments or documentation.
- Year Introduced: 2019.

The CodeSearchNet project was archived, which led to the removal of the S3 bucket that hosted the dataset. To address this issue, the CodeSearchNet dataset is now available for *download from Hugging Face* (Last access date: October 2, 2024). This alternative source ensures that researchers and developers can still access and use the dataset for their purposes.

10.1.2 The Pile (2020)

The Pile is an 825 GB dataset introduced by EleutherAI for training large-scale language models [111]. It is composed of 22 diverse high-quality subsets, combining text from various domains such as academic articles, GitHub repositories, legal texts, and public forums. The dataset is particularly relevant for building general-purpose language models that require strong understanding from various fields. It is suitable for tasks related to general language understanding and generation, known as language modeling. However, it might not be as comprehensive or effective for training models specifically designed to understand and generate programming code (code-based LLMs). The Pile can also function as a comprehensive benchmark to evaluate the model generalization capabilities and knowledge from various fields.

For software vulnerability detection tasks, the GitHub subset within The Pile is especially important. It contains a large corpus of code, enabling LLMs to learn patterns in programming languages, identify potential vulnerabilities, and generate secure code. Additionally, the inclusion of datasets like Stack Exchange allows models to interpret and respond to security-related queries, enhancing their capability in identifying vulnerabilities from both code and text data sources.

Key components:

- Size: 825 GB (English text corpus).
- Download link: *Pile dataset*. (Last access date: October 2, 2024).
- Sources: Includes data from Pile-CC, PubMed Central, Books3, OpenWebText2, ArXiv, Github, FreeLaw, Stack Exchange, USPTO Backgrounds, PubMed Abstracts, Gutenberg (PG-19), OpenSubtitles, Wikipedia (en), DM Mathematics, Ubuntu IRC, BookCorpus2, EuroParl, HackerNews, YoutubeSubtitles, PhilPapers, NIH ExPorter, and Enron Emails.
- Languages: Predominantly English, but some multilingual content is present.

10.1.3 GitHub code

The GitHub Code dataset is directly sourced from GitHub, and is available on Google's BigQuery. It is a vast collection of source code hosted on GitHub. It is primarily used for tasks involving machine learning, particularly for training, fine-tuning, and testing LLMs in software-related applications, including software vulnerability detection, code completion, bug detection, and other code analysis tasks. GitHub Code dataset covers an extensive variety of code, ranging from open-source libraries to complete software applications. It includes diverse coding styles, project structures, and programming styles, making it suitable for various code-related machine learning tasks.

The dataset is typically preprocessed before being used in machine learning tasks. This involves cleaning the data (e.g., removing personal information, non-code files), organizing it by programming language, and tokenizing the code into a format suitable for models. You can *download GitHub Code dataset* (Last access date: October 2, 2024) from Hugging Face.

10.1.4 CodeParrot

The CodeParrot dataset is a large-scale dataset of publicly available source code, primarily used for training and fine-tuning machine learning models in code-related tasks. The dataset is heavily focused on Python as the primary programming language. It is particularly well-known for being used in the development of models like CodeParrot, a GPT-like language model designed to generate, complete, and understand Python code. CodeParrot was released as part of the Hugging Face ecosystem and has become a valuable resource for developers working with machine learning on code.

The CodeParrot dataset is publicly available through *Hugging Face's Datasets library* (Last access date: October 6, 2024), where developers and researchers can download and use it for training their own models. There is also a clean version of dataset called *codeparrot-clean*, which is 50 GB in size.

10.1.5 The stack

The Stack dataset [112] is a 3.1 TB collection of permissively licensed source code, curated for the purpose of training and evaluating LLMs in coding tasks. The Stack consists of source code from 137.36 million GitHub repositories that were downloaded between November 2021 and June 2022, covering 30 programming languages. A version with near-deduplication is also provided, reducing redundancy and improving model training performance. The dataset can be used for pre-training and evaluating LLMCs. It supports tasks like code generation, code completion, bug detection, and documentation generation.

The dataset supports a wide range of programming languages, including Python, Java, C, JavaScript, HTML, C + + , and CSS, among others, with HTML (746 GB), JavaScript (486 GB), Java (271 GB), and C (222 GB) being the largest components.

The dataset only includes code that is freely licensed, such as MIT, Apache-2.0, and BSD-3-Clause. Licenses like GPL, which require certain rules, are left out to prevent legal or ethical issues. There is also a clear plan for managing the data, allowing developers to remove their code from the dataset by filling out a form and verifying through GitHub.

10.1.6 The Stack V1 and V2

The *Stack V1 dataset on Hugging Face* (Last access date: October 6, 2024) is a massive collection of source code files from various repositories. This dataset was created as part of the BigCode Project, which aims to develop LLMCs generation. The Stack dataset [112], was the foundation for The Stack V1, which after upgrading contains over 6 TB of code written in over 358 programming languages.

The Stack v2 dataset, on the other hand, is a significant development in large-scale source code datasets, offering a vast and diverse collection of freely licensed code. Built on top of Software Heritage's vast archive and enhanced with additional high-quality sources, including GitHub issues, pull requests, Kaggle notebooks, Jupyter notebooks, and code documentation, this dataset marks a substantial leap from its predecessor, The Stack v1. With support for 619 programming languages, The Stack v2 is ten times larger than its previous version. This extensive coverage of programming languages makes it highly suitable for training LLMs specialized in code generation, analysis, and completion [113].

The primary use case of The Stack v2 is the training of LLMs for Code (Code LLMs), such as StarCoder2. StarCoder2 with sizes ranging from 3 to 15 billion parameters, are trained on this dataset to perform various tasks, including code generation, completion, bug detection, and even reasoning about code execution [113]. A version of the dataset, with all duplicates removed, is also available and has a total size of 32.1 TB. Both version of The Stack V2 dataset can be accessed through *Hugging Face* datasets library (Last access date: October 6, 2024).

10.1.7 The ROOTS

The ROOTS dataset is a comprehensive multilingual corpus developed as part of the BigScience project, aimed at training LLMS in an open and collaborative environment. The dataset is one of the largest of its kind, comprising 1.6 TB of text data. The ROOTs dataset was designed with ethical considerations and diversity in mind and was used to train the BLOOM language model. It contains data in 59 languages, including 46 natural languages and 13 programming languages such as Java, PHP, and C + + [114].

ROOTS includes two main components: 62% of the dataset was derived from community-selected and documented sources, while the remaining 38% was sourced from OSCAR [115], a pre-processed web crawl dataset. The sources were carefully selected and reviewed by native speakers to ensure the dataset is accurate and of high quality. The collaborative efforts involved open submissions and competitive events, such as hackathons, engaging local communities focused on NLP and machine learning. These initiatives collected contributions from a broad range of participants, resulting in over 250 diverse sources of linguistic data. This strategy ensured that the dataset included multiple languages and a variety of linguistic features. As a result, the collected data became rich and diverse, allowing language models to be trained more effectively to handle different languages and complex linguistic patterns [114].

From a technical standpoint, ROOTS presents a sophisticated integration of multilingual text data and programming languages. This allows for the training of models that can handle not only human language but also programming code. The ROOTS corpus can be accessed from the official *repository of BigScience Data on Hugging Face* (Last access date: October 7, 2024).

10.1.8 RefinedWeb

The RefinedWeb dataset was introduced by the Falcon LLM team to address the challenges of training LLMs with web data alone, rather than relying on carefully collected collections of text that traditionally require extensive human intervention. The dataset was created as part of an effort to demonstrate that web data, if properly filtered and deduplicated, could outperform models trained on curated datasets, such as The Pile [21].

The RefinedWeb dataset contains about 5 trillion tokens, all gathered from web data, primarily from CommonCrawl dataset, a public repository that crawls the internet. Out of this, 600 billion tokens are publicly available at *Hugging*

Face (Last access date: October 7, 2024). The large size of the dataset makes it a significant resource for training large-scale LLMs with billions of parameters, especially those with 40–200 billion parameters in mind [21].

This dataset is an English-language dataset that has been processed using a proprietary MacroData Refinement (MDR) pipeline. This pipeline follows a strict set of rules for filtering and removing duplicates. The pipeline includes the use of tools like trafilatura [116] for content extraction from raw HTML, and fastText [117] for language identification. Additional steps in creating this dataset include filtering out spam, repeated sequences, and other low-quality content. The final output is designed to match or exceed the quality of curated datasets through automated processes, rather than manual curation [21].

The RefinedWeb dataset, primarily designed for English language processing tasks, can adapt to multiple languages and is used for training models in zero-shot tasks (tasks without prior training examples) like sentence completion and question answering. Its main use is pretraining large-scale language models, proving that carefully cleaned datasets from the web can outperform curated ones, as validated by models like Falcon 2, 180B, and 40B.

Utilizing the RefinedWeb dataset for NLP tasks offers high-quality, well-filtered data that can enhance performance. However, for optimal results in both NLP and code understanding, a specialized dataset for code should be used alongside RefinedWeb. In [21] the authors focus is on RefinedWeb's effectiveness in NLP tasks, but they do not address code-related tasks. Despite RefinedWeb being smaller than some other large datasets, its rigorous filtering makes it highly effective for NLP, but combining it with a dedicated code dataset remains unexplored in this context. This is why we decided to include it in our paper and introduce it alongside other major datasets like ROOTS and The STACK. Our goal was to propose a new approach to pretraining LLMCs, leveraging RefinedWeb for NLP tasks while using specialized datasets for enhanced code comprehension and generation.

10.2 Part 2: Specialized datasets for fine-tuning and testing LLMs in software vulnerability detection and mitigation

In this section, we introduce specialized datasets used for fine-tuning and testing LLMs specifically for tasks related to software vulnerability detection and mitigation. These datasets are more focused and task-specific compared to the foundational datasets used during pre-training. By providing detailed information about known vulnerabilities, weaknesses, and attack patterns, these datasets allow LLMs to acquire a deeper understanding of software security risks. Fine-tuning on such specialized datasets enables models to detect vulnerabilities more accurately, predict potential exploits, and recommend effective mitigation strategies.

The datasets introduced here are typically used not only for fine-tuning but also for testing LLMs in software vulnerability detection. During model development, datasets are commonly split into separate portions for *training*, *validation*, and *testing*. Each of these portions serves a distinct purpose to ensure the model performs well and generalizes effectively to unseen data.

- *Training Set:* This is the data used to "train" the model, meaning the model learns the patterns and relationships between inputs (e.g., code snippets) and outputs (e.g., identified vulnerabilities). The model adjusts its internal parameters to minimize the error on the training data.
- *Validation Set*: The validation set is used during the training process to tune hyperparameters (like learning rate or batch size) and prevent overfitting. It helps to evaluate the model's performance on unseen data, allowing adjustments to be made during training without using the test set.
- *Test Set*: The test set is kept entirely separate from both the training and validation sets. It is used to evaluate the model's final performance after the training and validation stages. The test set provides an unbiased measure of how well the model generalizes to new, unseen data. This is crucial for assessing the model's effectiveness in real-world scenarios.

In the context of software vulnerability detection, a common practice is to use a dataset for fine-tuning while setting aside a portion of it for testing. This ensures the model is evaluated on data it has not seen during training, providing an objective measure of its detection capabilities. By splitting the dataset in this manner, researchers can be confident that the model's performance is not biased by overfitting to the training data.

This approach of splitting datasets into training, validation, and testing portions helps ensure that the model is both accurate and reliable when detecting vulnerabilities across different software codebases.

The following table summarizes the key details of the datasets used for fine-tuning. Each dataset will be described in more detail following the table (Table 6).

10.2.1 CVE dataset

MITRE, a non-profit organization focused on cybersecurity research, provides valuable resources for the security community. One such resource is the CVE dataset. The CVE dataset is a comprehensive vulnerability dataset that is automatically collected and curated from CVE records in the public U.S. National Vulnerability Database (NVD).

Table 6 Datasets for fine-tuning and testing LLMs in software vulnerability detection

Dataset name	Size	Dataset type	Open/close access	Prog Langs covered	Content	Year introduced	Primary use	References
CVE Dataset	240,830 CVE records (July—2024)	Text-based Vulnerability dataset	Open access	Various	Publicly disclosed vulnerabilities, including CVE IDs, descriptions, CVSS scores, published dates, references, and affected products	1999	Training Data, Threat Intelligence, Automated Code Review, Security Audits, Incident Response,Vulnerability Management	[118]
CWE Dataset	–	Text-based Weakness classification	Open access	Various	Classifications of software weaknesses, including IDs, descriptions, relationships, and hierarchies	2006	Identification and classification of software weaknesses	[119]
CAPEC Dataset	559 entries (2024)	text-based Attack pattern classification	Open access	Various	Classifications of attack patterns, including hierarchies, relationships, and detailed descriptions of attacker techniques	2007	Understanding attack patterns and vulnerability exploitation and Security Training	[120]
CVEfixes	12.7 GB	Text-based and Code-based Vulnerability fixes dataset	Open access	Various	Detailed information about source code changes for fixing CVEs, including the commit, file, and method-level details	2021	Software vulnerability detection and mitigation	[121]
DiverseVul	18,945 vulnerable functions	Code-based Vulnerability dataset	Open access	Various (focus on C/C + +)	Vulnerable and non-vulnerable functions, focusing on a variety of common weaknesses, sourced from commits across multiple projects	2023	Vulnerability detection with Deep Learning (DL)	[67]

Table 6 (continued)

Dataset name	Size	Dataset type	Open/close access	Prog Langs covered	Content	Year introduced	Primary use	References
Software Assurance Reference Dataset (SARD)	420,627 SeVCs	primarily code-based Vulnerability dataset but includes descriptive information about these test cases	Open access	C, C + + , Java, PHP, and C#	large number of test programs with documented weaknesses, written in various programming languages	2005	Detecting software vulnerabilities, secure coding	[122]
Semantics-based Vulnerability Candidate (SeVC)	1,591 C/C + + programs	Text-based and Code-based Vulnerability dataset	Open access	C/C + +	Vulnerable and non-vulnerable code, including detailed semantic information about different vulnerability types (e.g., pointer usage, API calls)	2021	Vulnerability detection	[123]
Devign	~ 300,000 functions	Code-based Vulnerability dataset	Close access	C/C + +	Function-level C/C + + source code extracted from open-source projects, labeled for vulnerability detection, with manual verification	2019	Detecting software vulnerabilities	[124]
HumanEval	164 hand-crafted programming challenges	code-based Vulnerability dataset but includes descriptive information about programming tasks	Open access	Python	Function signatures (to guide the function implementation) Natural language descriptions of the programming task and Unit tests to validate the correctness of the code	2021	Used to evaluate the correctness and quality of code generated by AI models. Performance is typically measured with metrics like Pass@k	[32]

Table 6 (continued)

Dataset name	Size	Dataset type	Open/close access	Prog Langs covered	Content	Year introduced	Primary use	References
Big-Vul	Thousands of C/C + + vulnerabilities	Text-based and Code-based Vulnerability dataset	Open access	C/C + +	Vulnerabilities in C/C + + projects, including code changes, CVE summaries, and vulnerability-related metadata	2020	Vulnerability detection, understanding code changes	[125]
PrimeVul	6968 vulnerable and 228,800 benign functions	Code-based Vulnerability dataset	Open access	C/C + +	Labeled functions with details on whether they are vulnerable or benign	2024	Training and evaluating models for software vulnerability detection	[126]
LineVul	188,636 C/C + + functions, with 5,060,449 lines of code	Code-based Vulnerability dataset	Open access	C/C + +	Collected from 348 open-source GitHub projects and includes 91 Common Weakness Enumerations (CWEs) from 2002 to 2019. It provides line-level ground truths indicating specific lines in functions that are vulnerable	2022	line-level vulnerability prediction in C/C + + code	[127]

CVE datasets contain information on publicly disclosed cybersecurity vulnerabilities. These datasets are valuable for researchers, security professionals, and anyone interested in staying up-to-date on the latest threats. As of October 4, 2024, there are 240,830 CVE Records (*source*). It lists reported vulnerabilities in software systems.

CVE dataset typically includes:

- *CVE ID*: A CVE ID is a unique code assigned to a specific cybersecurity vulnerability by Mitre.
- *Description*: A detailed explanation of the vulnerability, including the affected software, potential impact, and how it can be exploited.
- *CVSS Score*: A scoring system (Common Vulnerability Scoring System) that reflects the severity of the vulnerability based on its exploitability, impact, and scope.
- *Published Date*: The date the vulnerability was publicly disclosed.
- *References*: Links to additional resources such as patches, workarounds, and exploit code.
- *Affected Products*: A list of software programs or systems that are vulnerable to the exploit.

To help researchers, we compiled a list of resources where CVE datasets can be found:

- *MITRE CVE*: This is the official source for CVE data and provides downloads in various formats. (Last access date: October 4, 2024).
- *The National Institute of Standards and Technology (NIST)*. (Last access date: October 2, 2024).
- *New CVE official website*. (Last access date: October 4, 2024).
- *Download page in new CVE website*: This page offers downloads of the CVE List in legacy formats until June 30th, 2024, and the new recommended JSON 5.0 format (Last access date: October 2, 2024).
- *CVE Details*: This website offers a searchable CVE database with additional information like exploit details and vendor risk scores (Last access date: October 2, 2024).

10.2.2 The common weakness enumeration (CWE) dataset

CWE is a community-developed catalog of software and hardware weaknesses maintained by MITRE. It is designed to provide a standardized classification of common security flaws that could lead to vulnerabilities in systems. The primary objective of the CWE dataset is to help developers, security professionals, and organizations identify, understand, and mitigate these weaknesses during software development, system design, and security assessments. The CWE entries provide comprehensive descriptions of these weaknesses, examples of how they may be exploited, and guidance on how to avoid or fix them [119].

The CWE framework is a system that classifies software weaknesses. It's organized into categories to help users understand and analyze vulnerabilities from broad, high-level issues to more specific technical details. For example, "Security Features" or "Input Validation," with more specific entries under each. The dataset is regularly updated to include new weaknesses as they are identified, ensuring that it reflects the evolving nature of software and hardware vulnerabilities [119].

CWE is widely used by developers to improve the security of their code, by security testers to identify vulnerabilities, and by organizations for compliance and risk management purposes. The dataset also plays a crucial role in security tools such as static and dynamic analysis tools, which use CWE mappings to report potential issues in code. To download the latest CWE dataset, you can visit the official MITRE CWE download page *here* (Last access date: October 7, 2024) [119].

Each CWE entry includes a detailed description of the weakness, explaining what it is, how it could affect systems, and how attackers might exploit it. It often connects to the Common Attack Pattern Enumerations and Classification (CAPEC), which outlines common attack methods. When possible, real-world examples are provided to illustrate how the weakness has been exploited in actual cases.

Specifically, each CWE entry includes the following details:

- *ID*: A unique identifier for the weakness.
- *Name*: A concise description of the weakness.
- *Description*: A more elaborate explanation of the weakness, its potential consequences, and how it can be exploited.
- *Extended Description:* Additional in-depth information about the weakness.
- *Relationships:* Connections to other CWEs and relevant security concepts.
- *Hierarchies:* Organization of CWEs within a structured classification scheme.

10.2.3 CAPEC dataset

The CAPEC dataset, also from MITRE, focuses on classifying cyber-attack patterns. Similar to CWE, CAPEC provides a standardized language for understanding attacker methods. CWE entries are structured in a hierarchy, showing relationships between different attack patterns. This organization helps users understand how various vulnerabilities and attack methods are connected [91, 120].

The CAPEC dataset is meticulously organized to facilitate its use in various cybersecurity applications. It employs a hierarchical structure that categorizes attack patterns based on mechanisms of attack, such as social engineering, network-based attacks, and software-based attacks, as well as domains of attack like web applications, mobile devices, and cloud services. This organization allows users to navigate the dataset efficiently and apply it to specific contexts. Additionally, attack patterns are classified by other criteria, including the impact of the attack, prerequisites, and typical severity. In terms of usage, the CAPEC dataset is invaluable for enhancing the capabilities of LLMs in several areas. It aids in recognizing and classifying attack patterns, thereby improving threat intelligence and incident response. The dataset also supports security training by helping LLMs understand and predict attack vectors, and it assists in vulnerability analysis by identifying potential attack vectors in software. Furthermore, CAPEC is instrumental in conducting thorough security audits, ensuring comprehensive coverage of potential threats. This detailed organization and multifaceted usage make CAPEC a critical resource for improving cybersecurity measures and defenses [91, 120]. To download the latest CAPEC dataset, you can visit the *official MITRE CAPEC download page* (Last access date: October 7, 2024).

10.2.4 CVEfixes

The CVEfixes dataset was first introduced by Bhandari et al. [121]. This comprehensive vulnerability dataset is a rich resource for studying vulnerabilities. It is automatically compiled from publicly available CVE records in the U.S. NVD. The dataset supports security research by linking CVE-related information to different levels of software development, such as code commits, individual files, specific methods, repositories, and CVEs themselves. This allows researchers to analyze security fixes in detail and study how they address vulnerabilities at multiple levels of abstraction. The initial release of the dataset covers all published CVEs up to June 9, 2021, and includes information from 5,495 vulnerability fixing commits in 1,754 open-source projects, covering a total of 5,365 CVEs across 180 different CWE types. Additionally, the dataset includes the source code before and after fixing for 18,249 files and 50,322 functions1 [24, 34, 121].

The most recent version, v1.0.8 from the *project GitHub repository* (Last access date: October 8, 2024), includes all CVEs published up to July 23, 2024. For this release, all open-source projects referenced in CVE records within the NVD during this period, and which had publicly accessible Git repositories, were collected and integrated into the dataset. This vulnerability dataset is structured as a relational database, encompassing 12,107 vulnerability-fixing commits from 4,249 open-source projects, representing a total of 11,873 CVEs across 272 distinct CWE categories. Additionally, the dataset provides source code snapshots before and after the modifications affecting 51,342 files and 138,974 functions.

Researchers leveraging LLMs for software vulnerability tasks can benefit from this dataset by using its structured, interlinked data to train models on real-world vulnerability scenarios. The dataset also provides an excellent foundation for fine-tuning models that aim to improve security practices in open-source projects. However, researchers should be aware of potential challenges, such as noisy data or the need to preprocess the information across different levels of abstraction (e.g., file, method, or commit level).

10.2.5 DiverseVul

DiverseVul dataset was first introduced by Chen et al. in [67], designed for the detection of software vulnerabilities through DL techniques. DiverseVul comprises 18,945 vulnerable functions across 155 CWEs and 330,492 non-vulnerable functions, sourced from 7,514 commits. This dataset covers 295 more projects than any previously published dataset, making it one of the most diverse resource available for C/C + + vulnerabilities. DiverseVul includes real-world data sourced from various projects, significantly expanding the variety and number of functions compared to previous datasets like CVEFixes.

Utilizing DiverseVul, the study conducted in [67] investigates the efficacy of various DL architectures in vulnerability detection. It explores 11 distinct DL architectures from four model families: GNN, RoBERTa, GPT-2, and T5. Findings suggest that the increased diversity and volume of training data contribute to enhanced vulnerability detection, particularly for LLMs [67]. You can download the DiverseVul dataset from the *following link* (Last access date: October 10, 2024).

10.2.6 The software assurance reference dataset (SARD)

The SARD Dataset, maintained by the NIST, provides a comprehensive collection of test cases designed to aid researchers, tool developers, and end users in evaluating software assurance tools. These test cases cover various phases of the software lifecycle, including designs, source code, and binaries, and feature known security vulnerabilities along with their corresponding fixes. The dataset includes samples from synthetic sources (designed specifically for testing), real-world production software, and academic research. Its primary goal is to support the evaluation of software security tools and facilitate the development of new methods for detecting and mitigating vulnerabilities [122].

With approximately 500,000 test cases, the SARD dataset is one of the largest publicly available resources for software

vulnerability testing. It covers 6 programming languages, including C, C + + , Java, PHP, and C#. Users can search, view, and download individual test cases or entire test suites, making it an essential tool for researchers and developers in the field of software security. SARD continues to evolve, with ongoing enhancements to improve usability and search functionality, making it easier for users to access and analyze test cases efficiently [122].

SARD is particularly attractive due to its inclusion of both security vulnerabilities and non-vulnerable alternatives. This feature enables LLMs to discern between these distinct categories effectively. Subsequently, it is possible to implement a preprocessing stage to eliminate any undesired artifacts that could potentially cause overfitting in the model. The SARD dataset was used in many experiments, including [13, 15, 67, 74, 75, 87].

The best and most official source for downloading the SARD Dataset is from the NIST. You can access and download the dataset directly from their *official website* (Last access date: October 10, 2024).

10.2.7 Semantics-based vulnerability candidate (SeVC).

SeVC dataset contains 1,591 C/C + + open-source programs from the NVD, along with 14,000 open-source programs from SARD. It contains a total of 420,627 instances of source code elements, referred to as SeVCs. Of these, 56,395 are classified as vulnerable, while 364,232 are considered non-vulnerable [13, 74, 123]. The dataset focuses on four main types of source code patterns that can impact security:

- Library/API Function Calls: Usage of external libraries or APIs.
- Array Usage: Operations involving arrays.
- Pointer Usage: Handling of pointers in the code.
- Arithmetic Expressions: Mathematical operations within the code

The SeVC dataset is publicly accessible within the *SySeVR project's GitHub repository* (Last access date: October 6, 2024).

Researchers can leverage the SeVC datasets to enhance LLMs in software vulnerability detection and mitigation by providing rich, real-world examples of vulnerabilities and fixes. The SeVC dataset, with its labeled vulnerable and non-vulnerable code segments, enables LLMs to understand semantic code patterns related to vulnerabilities.

10.2.7 Devign

The Devign dataset, initially presented in [124], stands as a real-world dataset designed for the identification of software vulnerabilities. The Devign dataset is designed to support vulnerability identification through DL, particularly using GNNs. Built upon manually labeled datasets from four large-scale open-source C projects (Linux Kernel, QEMU, Wireshark, and FFmpeg), Devign includes 58,965 code snippets and aims to enhance the understanding of code semantics. The dataset focuses on vulnerable and non-vulnerable functions, extracted from vulnerability-fixing commits and non-vulnerability-fix commits, providing real-world complexity and variety rather than synthetic or artificial code. To ensure quality, a team of security experts manually labeled the dataset through a two-stage process that involved filtering security-related commits and extracting vulnerable functions from the source code [124].

Devign employs an advanced method for detecting vulnerabilities in code by using graph-based representations. It combines several key elements:

- Abstract Syntax Trees (ASTs): These capture the structure of the code.
- Control Flow Graphs (CFGs): These show the order in which different parts of the code execute.
- Data Flow Graphs (DFGs): These trace how data moves through the code.
- Natural Code Sequences (NCSs): These represent the code as written by developers in its natural order.

By integrating these different perspectives, Devign creates a more comprehensive analysis of the code to identify potential vulnerabilities. These representations capture different semantic and structural aspects of the code, enabling the GNN model to identify various patterns associated with vulnerabilities. The Devign dataset enables detailed analysis at the function level, providing an exceptional opportunity for ML models to understand and learn from the complex relationships within code. By focusing on this level of detail, researchers can train models to recognize patterns and connections that are crucial for identifying vulnerabilities. The dataset stands out due to its comprehensive manual labeling, which adds significant depth and reliability. These qualities make Devign a highly valuable resource for creating sophisticated systems capable of detecting vulnerabilities with precision [124].

Devign has been thoroughly tested and shown to be more effective at detecting vulnerabilities than older methods. It achieved up to 10.51% higher accuracy and an 8.68% better F1 score, highlighting the power of its graph-based analysis. Moreover, Devign proved to be highly reliable in identifying newly discovered vulnerabilities in real-world projects, demonstrating its practical value in enhancing software security [124].

10.2.8 HumanEval

The HumanEval dataset was introduced by Chen et al. in [32] as part of their work evaluating large language models trained on code. This dataset provides a benchmark specifically designed to measure the capability of LLMs in code generation tasks. HumanEval consists of 164 Python programming problems, each accompanied by a function signature, a natural language description of the task, and unit tests to verify the correctness of the generated solutions. These unit tests ensure that the generated code performs as expected and meets the problem requirements.

The primary purpose of the dataset is to evaluate the functional correctness of code produced by LLMs, measured using metrics like Pass@k, which assesses whether correct solutions appear among the top kk generated outputs. HumanEval focuses exclusively on Python, making it a targeted benchmark for evaluating LLMs in this language.

The dataset is an essential tool for researchers developing or fine-tuning LLMs for tasks related to software vulnerability detection and mitigation. While it does not explicitly address vulnerabilities, its emphasis on generating correct and functional code is critical for producing secure software solutions. Researchers leveraging HumanEval benefit from its well-defined problems and robust evaluation metrics, enabling detailed analysis of LLM performance. The HumanEvaldataset is publicly accessible within the *human-eval GitHub repository* (Last access date: December 11, 2024). The dataset is also accessible through Hugging Face and TenserFlow.

10.2.9 Big-Vul

The Big-Vul dataset [125] is a comprehensive collection of C/C + + code vulnerabilities gathered from GitHub projects and the CVE database. It was created by extracting detailed information from the CVE database, including vulnerability IDs, severity scores, and summaries. Covering vulnerabilities reported between 2002 and 2019, the dataset includes 21 distinct features for each entry. Included features are:

- Access Complexity: How difficult it is to exploit the vulnerability.
- Authentication Required: Whether authentication is needed for exploitation.
- Availability Impact: The effect on system availability if exploited.
- Commit ID: Commit ID in code repository, indicating a mini-version.
- Commit Message: Commit message from the developer about the code changes.
- Confidentiality Impact: The risk to sensitive information.
- CWE ID: Common Weakness Enumeration ID.
- CVE ID: Common Vulnerabilities and Exposures ID.
- CVE Page: URL to more information about the vulnerability.
- CVE Summary: CVE summary information.
- CVSS Score: The seriousness of the vulnerability.
- Files Changed: Details of updated code and fixes.
- Integrity Impact: The risk of data manipulation or tampering.
- Mini-version After Fix: Mini-version ID after the fix.
- Mini-version Before Fix: Mini-version ID before the fix.
- Programming Language: The programming language used in the project.
- Project name.
- Publish date of the CVE.
- Reference Link: Reference link in the CVE page.
- Update date of the CVE.
- Vulnerability Classification: Vulnerability type.

These vulnerabilities were then linked to corresponding code changes in GitHub repositories. Each Big-Vul entry pairs a CVE with the specific GitHub code commit that resolves it, offering a clear link between vulnerabilities and their fixes. In total, Big-Vul includes 3,754 code vulnerabilities from 348 open-source projects, covering 91 different vulnerability types. The dataset is stored in CSV format, and the code changes are associated with the descriptive CVE information, making it useful for research on detecting and fixing vulnerabilities [125].

Big-Vul stands out from other vulnerability datasets due to its comprehensive nature. It not only provides details on vulnerability types and code fixes but also includes a wide range of features such as security scores and code modifications. Compared to other datasets, Big-Vul contains more vulnerabilities and detailed information, with 4,432 code commits, 8,143 modified files, and 11,823 vulnerable functions. This makes it an essential resource for researchers aiming to analyze vulnerability characteristics, improve detection mechanisms, and study vulnerability-related code changes at various levels [60, 125]. To access the BigVul dataset, you can visit the *project's official repository on GitHub* (Last access date: October 10 2024).

10.2.10 PrimeVul

PrimeVul is a newly introduced dataset designed to address the limitations of existing datasets used for vulnerability detection in code. It incorporates improved data collection, labeling, and evaluation techniques. Unlike previous datasets, which suffer from issues such as data duplication and low label accuracy, PrimeVul implements stringent de-duplication processes to eliminate repeated or similar data samples. The dataset merges security-related commits and functions from previous datasets such as BigVul,

CrossVul, and CVEFixes, while excluding sources like Devign/CodeXGLUE, which were found to contain irrelevant security data. PrimeVul de-duplicates both code copies and functions that differ only in formatting to ensure clean and unique data [126].

PrimeVul uses two innovative methods for labeling: one labels a function as vulnerable if it's the only one modified in a security fix, while the other matches vulnerability descriptions from the NVD with security-related code changes. This process resulted in a large, accurate dataset with 6,968 vulnerable and 228,800 non-vulnerable functions across 755 projects and 140 CWE types, rivaling the quality of manually verified datasets like SVEN [126].

PrimeVul also includes improved evaluation methods for vulnerability detection models. It uses temporal splits (a method where the training data is separated from future data to avoid training models on information they wouldn't have in a real-world scenario) to prevent data leakage (the unintended use of information from the test set during training, which can lead to overly optimistic results). It also introduces a Vulnerability Detection Score (VD-S) to measure a model's accuracy while controlling false positives (incorrectly labeling non-vulnerable code as vulnerable). Additionally, PrimeVul offers a pair-wise evaluation (a technique where models are tested to distinguish between pairs of code samples, one being vulnerable and the other a fixed, non-vulnerable version) to help models better differentiate between vulnerable code and its corrected version, ensuring a more realistic assessment of performance [126]. The PrimeVul dataset is publicly accessible within the *PrimeVul project's GitHub repository* (Last access date: October 10, 2024).

10.2.11 LineVul

LineVul is a dataset designed to address limitations in vulnerability detection by focusing on fine-grained predictions at the line level (this means it identifies vulnerabilities in specific lines of code, not just at the function or general code block level), particularly for C and C + + code. The dataset consists of 188,636 functions, with over 91 types of CWEs, and includes line-level ground truth (accurate labels indicating which exact lines of code are vulnerable). This fine-grained prediction helps in pinpointing exact locations of vulnerabilities, making it easier for developers to address them. One of its key aspects is that only 5.7% of the total functions are labeled as vulnerable, providing a realistic and challenging environment for testing machine learning models aimed at vulnerability detection [127]. The LineVul dataset and related resources can be accessed through the *project's official GitHub repository* (Last access date: October 10 2024).

RQ8 Answer: *Datasets used to train, fine-tune, or test LLMs for software vulnerability detection include both foundational and specialized datasets. Foundational datasets like CodeSearchNet, The Pile, GitHub Code, ROOTS, RefinedWeb, and The Stack are used for pre-training LLMs, helping models learn general coding patterns across various programming languages and domains. These datasets provide the broad linguistic and programming knowledge necessary for models to understand code structures and semantics.*

For fine-tuning and testing LLMs specifically for vulnerability detection, specialized datasets like the CVE, CWE, CAPEC, CVEFixes, DiverseVul, SARD, SeVC, Devign, Big-Vul, PrimeVul, and LineVul are critical. These datasets focus on detailed information about software vulnerabilities, weaknesses, and security risks, allowing models to specialize in identifying, predicting, and mitigating vulnerabilities at both the function and line levels in code.

11 RQ9. What metrics are used to assess LLMs in addressing software vulnerabilities?

Evaluation metrics are crucial for assessing the effectiveness and success of LLMs in software vulnerabilities and cybersecurity applications. These metrics provide a framework for quantifying the performance of these models across various tasks related to software vulnerabilities and cybersecurity threats, such as vulnerability detection, threat prediction, and automated patching. Given the diverse nature of these tasks, employing a range of evaluation metrics tailored to specific problem types is common practice.

In this section, we will categorize the evaluation metrics for LLMs used in software vulnerability detection and mitigation. We will divide them into three distinct categories based on the types of tasks (Classification, Recommendation, and Generation) they address.

For each metric, we will provide a short introduction, including its significance and application. Additionally, we will outline the formula or steps required for calculating each metric, ensuring clarity and precision. Where applicable, we will include succinct examples to facilitate a better understanding of the metrics for our readers.

11.1 Classification tasks

For tasks involving classification, such as identifying types of vulnerabilities or predicting potential threats, F1-score (F-measure), Precision, and Recall are commonly used metrics [1, 11, 16, 67, 73, 74, 80, 123, 128].

11.1.1 F1-score

The F1-score, also known as the F-measure [128], is a metric used to evaluate the performance of a binary classification model by considering both precision and recall. Precision is the proportion of true positive results among the total number of positive results predicted by the model, while recall is the proportion of true positive results among the total number of actual positives. The F1 score is the harmonic mean of precision and recall, providing a single measure of a model's accuracy. In the context of software vulnerability detection and mitigation, the F1 score is particularly important because it balances the trade-off between precision and recall. High precision means that the model accurately identifies vulnerabilities with few false positives, while high recall means that the model detects most of the actual vulnerabilities with few false negatives. A high F1 score indicates that the model performs well in both aspects, which is crucial for effectively identifying and mitigating vulnerabilities in software systems [128].

$$F1 - score = \frac{2 \times Precission \times Recall}{Precision + Recall}$$

Limitations of F1-score.

- Imbalance Sensitivity: Struggles with imbalanced datasets, common in vulnerability detection.
- Threshold Dependency: Requires setting a threshold for binary classification.
- Single-Label Focus: Not suitable for multi-label classification tasks.

11.1.2 Precision

Precision is the ratio of true positives (correctly predicted vulnerabilities) to all positive predictions (all predicted vulnerabilities).

$$Precision = \frac{True\ Positives(TP)}{True\ Positives(TP) + False\ Positives(FP)}$$

Limitations of Precision.

- Ignores False Negatives: false negatives are critical in security contexts.
- Imbalance Impact: Highly affected by imbalanced datasets with few positive cases.
- Limited Scope: Only focuses on positive predictions and fails to measure the overall effectiveness of the model.

11.1.3 Recall

Recall is the ratio of true positives to all actual positives (all actual vulnerabilities).

$$Recall = \frac{True\ Positives(TP)}{True\ Positives(TP) + False\ Negatives(FN)}$$

Limitations of Recall.

- Overlooks false positives, leading to low specificity.
- Encourages models to predict more positives, reducing precision.
- Misleading in scenarios where exactness is more critical than coverage.
- Provides no measure of output reliability or correctness.

11.1.4 True/false positive and true/false negative rate

In the context of classification or detection tasks, such as software vulnerability identification using LLMs, True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN) are essential components for evaluating model performance.

- True Positive (TP): A result where the model correctly identifies a positive case (e.g., a correctly detected vulnerability).
- False Positive (FP): A result where the model incorrectly identifies a positive case (e.g., falsely classifying secure code as vulnerable).
- True Negative (TN): A result where the model correctly identifies a negative case (e.g., correctly identifying secure code as non-vulnerable).
- False Negative (FN): A result where the model fails to identify a positive case (e.g., missing an actual vulnerability).

11.1.5 Accuracy

In classification tasks, accuracy represents the ratio of correctly classified instances (both positive and negative) to the total number of instances.

$$Accuracy(\%) = \frac{number\ of\ correctly\ classified\ instances}{total\ number\ of\ instances} \times 100$$

$$= \frac{TP + TN}{TP + TN + FP + FN} \times 100$$

where:

- TP: True Positives (vulnerabilities correctly identified).
- TN: True Negatives (non-vulnerabilities correctly identified).
- FP: False Positives (non-vulnerabilities incorrectly classified as vulnerabilities).
- FN: False Negatives (vulnerabilities incorrectly classified as non-vulnerabilities).

Limitations of Accuracy.

- Ineffective for imbalanced datasets, masking poor performance on key classes.
- Doesn't differentiate between false positives and false negatives.
- Treats all errors equally, which is not ideal for critical errors in security contexts.
- Misleading in cases where class distributions heavily vary.

11.1.6 AUC-ROC

The Area Under the Receiver Operating Characteristic Curve (AUC-ROC) is a widely used metric for evaluating the performance of classification models. The ROC curve is a graphical representation of the trade-off between the true positive rate (TPR) and the false positive rate (FPR) at various threshold settings. The area under this curve (AUC) provides a single scalar value that summarizes the performance of the classifier. AUC-ROC assesses the model's ability to distinguish between positive and negative cases across different classification thresholds. A higher AUC-ROC value indicates better overall performance [129].

AUC-ROC is particularly useful for evaluating:

- Binary classification tasks, such as identifying whether a piece of code is vulnerable or not.
- Multi-class classification tasks, such as categorizing code into different vulnerability types or software components.
- Imbalanced datasets, where it is important to evaluate how well the model performs across thresholds.

ROC Curve is a graphical representation of $TPR = \frac{TP}{TP+FN}$ against $FPR = \frac{FP}{FP+TN}$, and AUC is the area under the ROC curve. Its value can range from 0 to 1 where:

- 1.0: Perfect discrimination (ideal model).
- 0.5: Random guessing (no discriminatory power).
- < 0.5: Worse than random guessing.

An example of the ROC curve and AUC area can be seen in Fig. 7.

Steps to Calculate AUC-ROC:

1. Plot the points (FPR, TPR) for each threshold.
2. Calculate the Area Using the Trapezoidal Rule:

The trapezoidal rule approximates the area under the curve by summing the areas of trapezoids formed between consecutive points.

$$AUC = \sum_{i=1}^{n-1}(FPR_{i+1} - FPR_i) \times \frac{TPR_i + TPR_{i+1}}{2}$$

where:

- n is the number of points on the ROC curve.
- FPR_i and TPR_i are the false positive rate and true positive rate at the i-th threshold.

For example: for two points like (FPR1, TPR1) and (FPR2, TPR2), the area of the trapezoid is:

$$AUC = (FPR2 - FPR1) \times \frac{TPR_1 + TPR_2}{2}$$

Advantages of AUC-ROC:

- Threshold Independence: Evaluates the model across all classification thresholds.
- Class Imbalance Robustness: Less sensitive to imbalanced datasets than metrics like accuracy.
- Intuitive Interpretation: Higher AUC-ROC values indicate better model performance in distinguishing between classes.

Limitations of AUC-ROC:

- Does not provide specific information about performance at a particular threshold.
- May not be ideal for severely imbalanced datasets where other metrics (e.g., Precision, Recall) are more insightful.
- Assumes uniform importance of false positive and true positive rates.

11.2 Recommendation tasks

In the domain of software vulnerability detection and mitigation, recommendation tasks refer to the process of suggesting potential vulnerabilities, fixes, or mitigations to developers or security teams. The goal is to help them identify and prioritize vulnerabilities, and provide guidance on how to address them.

Some common recommendation tasks in this domain include:

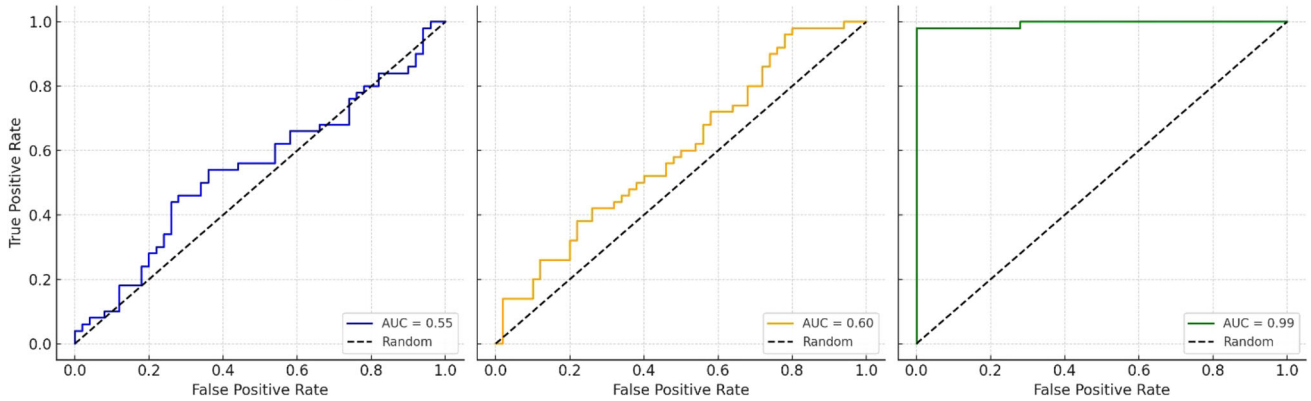

Fig. 7 Examples of AUC-ROC plot: **A** is for completely random guesses, **B** is for AUC of 60% and **C** is for AUC of 99%

- Vulnerability prediction: predicting the likelihood of a piece of code containing a vulnerability.
- Vulnerability ranking: ranking potential vulnerabilities in order of severity or likelihood.
- Fix recommendation: suggesting potential fixes or patches for identified vulnerabilities.
- Mitigation recommendation: recommending strategies or techniques to mitigate the impact of a vulnerability.

MRR, Precision@k, Recall@k, F1-score@k, amd Accuracy, are metrics commonly used to evaluate the performance of recommendation systems and the quality of ranked results, including those that recommend code snippets or completions. These metrics help assess how accurately a model prioritizes relevant results within the top k items in its output [16, 33, 110]. In evaluation metrics, the @k notation is used to specify that the metric is being evaluated at a specific cut-off point, typically the top-k items in a ranked list. In other words, @k means "at k" or "up to k", where k is a positive integer. It indicates that the metric is being calculated only considering the top-k items in the list, rather than the entire list (Fig. 8).

11.2.1 Mean reciprocal rank (MRR)

MRR is a metric commonly used in information retrieval and natural language processing to evaluate the performance of ranking systems. It is also a commonly used metric for recommendation systems related to code completion. MRR measures the effectiveness of a model by determining how well it ranks the most relevant result for a given query (how quickly the first relevant recommendation appears in the list of suggestions). MRR can be particularly useful for assessing the effectiveness of LLMs in identifying and prioritizing potential vulnerabilities in code [130].

MRR is calculated as the average of the reciprocal ranks of the first relevant item for a set of queries:

$$MRR = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{rank_i}$$

where:

- **N** is the total number of queries.
- $\boldsymbol{rank_i}$ is the rank position of the first relevant item for the i-th query.

Example Calculation.

Suppose we have 3 queries, and the ranks of the first relevant results are as follows:

Query 1: Relevant result is at rank 1.

Query 2: Relevant result is at rank 3.

Query 3: No relevant result (reciprocal rank = 0).

The MRR calculation would be:

$$\text{MRR} = \frac{1}{3}\left(\frac{1}{1}+\frac{1}{3}+0\right) = \frac{1}{3}(1+0.333+0) = 0.444$$

The MRR score of 0.444 indicates the model's average effectiveness in ranking relevant results.

11.2.2 F1-score@k

F1-score@k provides a balance between Precision@k and Recall@k, highlighting the trade-off between precision and the coverage of relevant results within the top k.

$$F1-score@k = \frac{2 \times Precission@k \times Recall@k}{Precision@k + Recall@k}$$

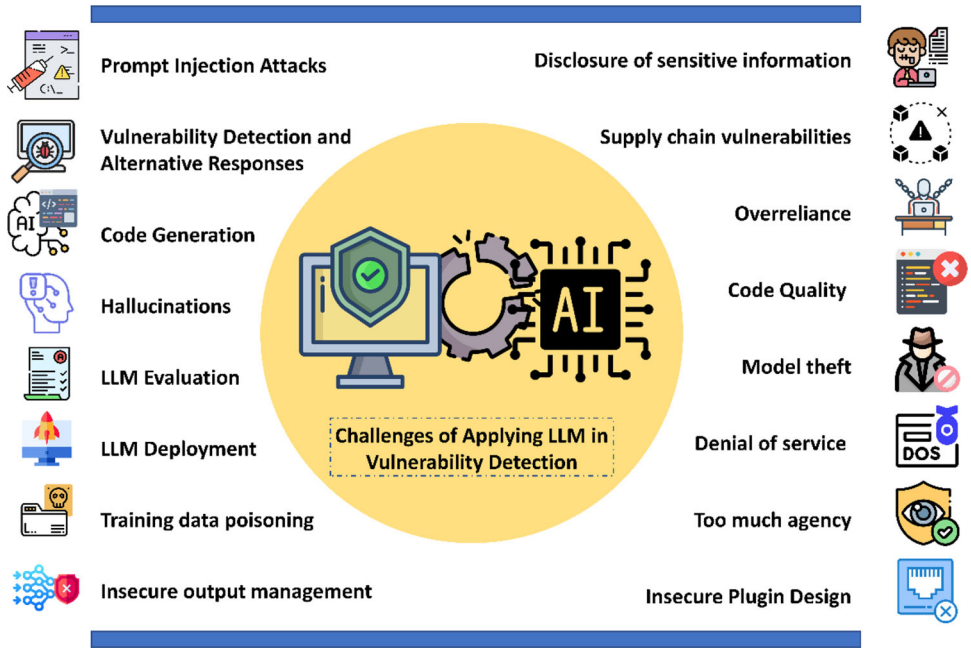

Fig. 8 Challenges of using LLMs in software vulnerability domain

11.2.3 Precision@k

Precision@k evaluates the proportion of relevant results among the top k recommendations, emphasizing the model's ability to avoid false positives.

$$Precision@k = \frac{|Relevant\,Results\,in\,Top\,k|}{k}$$

where:

- $|Relevant\ Results\ in\ Top\ k|$, is the number of relevant results among the top k items recommended by the model.
- *K, is* the size of the subset of top recommendations considered.

11.2.4 Recall@k

Recall@k measures how many relevant results are captured in the top k recommendations. A higher value implies that the model retrieves a greater proportion of all possible relevant results.

$$Recall@k = \frac{|Relevant\ Results\ in\ Top\ k|}{|Total\ Relevant\ Result|}$$

where:

|Total Relevant Results| is the total number of relevant results available for the query.

11.2.5 Accuracy

In recommendation tasks, accuracy is used to measure the ratio of correctly recommended items (e.g., secure coding practices, vulnerability mitigation strategies) to the total recommendations.

$$Accuracy(\%) = \frac{\text{Number of Relevant or Correct Recommendations}}{Total\ Number\ of\ Recommendations} \times 100$$

11.2.6 Accuracy@k

Accuracy@k is a metric used to evaluate the performance of models in tasks where the top-k recommendations or predictions are considered. It measures the proportion of instances for which the correct item is included in the top-k recommendations or predictions made by the model.

$$Accuracy@k(\%) = \frac{\text{Number of } Instances \text{ with Correct Item in Top} - \text{k Recommendations}}{Total\ Number\ of\ Instances} \times 100$$

Steps to Calculate Accuracy@k:

- Define k**:** Determine the value of k, which is the number of top recommendations or predictions to consider. For example, if you are evaluating the top-3 recommendations, k = 3.
- Identify Correct Items**:** For each instance (e.g., a code snippet or vulnerability), identify whether the correct item

(e.g., a secure code suggestion or mitigation strategy) is included in the top-k recommendations.
- Count Correct Instances**:** Count the number of instances for which the correct item is included in the top-k recommendations.
- Calculate Accuracy@k**:** Use the formula to calculate the accuracy percentage

11.3 Generation tasks

Generation tasks involve using LLMs to produce new content, such as code snippets, patches, or explanations, based on given inputs. In the context of software vulnerability detection and mitigation, generation tasks can include:

Code Generation:

- Vulnerable Code Identification: Generating code snippets that highlight potential vulnerabilities.
- Secure Code Suggestions: Generating secure alternatives to vulnerable code snippets.
- Automated Patching: Generating patches or fixes for identified vulnerabilities.

Explanation Generation:

- Vulnerability Explanations: Generating natural language explanations of why a particular code snippet is vulnerable.
- Mitigation Strategies: Generating recommendations on how to mitigate identified vulnerabilities.
- Providing guidelines or templates for writing secure code: For example, suggesting secure API usage patterns.

Documentation Generation:

- Security Guidelines: Generating documentation that includes security best practices and guidelines.
- Code Comments: Generating comments that explain the security implications of code snippets.

Evaluating LLMs for generation tasks in this domain requires metrics that measure the relevance, accuracy, and quality of the outputs. Commonly used metrics include BLEU or its variants including BLEU-4, CodeBLEU and BLEU-DC, and Pass@k. These metrics are widely used for code-to-code translation models and code generation assessment. They evaluate the quality and accuracy of generated code snippets compared to reference solutions. Additionally, other metrics like ROUGE/ROUGE-L, METEOR, EM (Exact Match), and ES (Edit Similarity) are utilized in specific studies to assess the quality of generated code or natural language code descriptions [16, 32, 131, 132].

11.3.1 Code coverage

Code Coverage [133] is primarily used for evaluating software testing tasks. It measures the extent to which the lines, branches, or paths of a program's source code are executed during testing. While it is used for assessing the thoroughness of generated tests or the execution of generated code, it is not commonly categorized alongside other metrics like BLEU or CodeBLEU for text/code generation tasks. Therefore, its usage may be more limited. It is primarily used to evaluate generation tasks related to code synthesis and code completion, ensuring that generated code is effective, functional, and adequately tested. In the context of software vulnerability detection and mitigation, it can be used to assess the comprehensiveness of automated patch generation or test case generation tasks.

Types of Code Coverage.

There are several types of code coverage metrics, each focusing on different aspects of the code:

- Statement Coverage: Measures the percentage of statements in the code that are executed by the test suite.
- Function Coverage: Measures the percentage of functions in the code that are called by the test suite.
- Line Coverage: Measures the percentage of executable lines of code executed at least once.
  - Indicates how much of the codebase has been directly tested.
- Branch Coverage: Measures the percentage of branches in the code that (e.g., "if" or "else" paths) are executed.
  - Ensures logical branches in code are adequately tested.
- Path Coverage: Measures the percentage of possible paths through the code that are executed by the test suite.
  - Provides the most comprehensive evaluation but is computationally expensive.

The general formula for calculating code coverage is:

$$Code\,Coverage(\%) = \left(\frac{\text{Number of Covered Items}}{\text{Total Number ofItems}}\right) \times 100$$

where "Items" can refer to statements executed, branches executed, functions called, lines executed, or paths executed, depending on the type of coverage being measured.

Code coverage has several limitations that should be considered. Firstly, high code coverage does not guarantee the absence of bugs; even with extensive testing, defects may still persist. Additionally, code coverage metrics focus solely on executed code, meaning they do not assess whether the code behaves correctly under various conditions. Finally,

for complex applications, achieving full path coverage can be computationally infeasible, making it expensive and resource-intensive to ensure thorough testing. These limitations highlight the need for complementary testing strategies beyond code coverage metrics.

11.3.2 Accuracy

In generation tasks, accuracy measures the ratio of correctly generated outputs, such as patches, vulnerability descriptions, or exploit scenarios, compared to the total number of generated outputs. This is often evaluated as Exact Match Accuracy. Accuracy is calculated as the ratio of correctly generated outputs to the total number of generated outputs.

$$Accuracy(\%) = \frac{\text{Number of Correctly Generated Outputs}}{Total\ Number\ of\ \text{Generated Outputs}} \times 100$$

11.3.3 BLEU, BLEU-4, BLEU-DC

BLEU (Bilingual Evaluation Understudy) Score [134] is a metric used to evaluate the quality of text generated by a model compared to a reference text. It is widely used in machine translation and natural language generation tasks. In the context of software vulnerability detection and mitigation, BLEU Score can be used to evaluate the quality of code generated or modified by an LLM to fix vulnerabilities.

$$BLEU = BP.exp\left(\sum_{n=1}^{N} w_n \log p_n\right)$$

where:

- BP is the brevity penalty to penalize short translations.
- p_n is the precision for n-grams (n-grams is a sequences of consecutive words or tokens).
- w_n is the weight for n-grams. Allow assigning different importance to n-grams of varying sizes.
- N is the maximum order of n-grams considered.

Brevity Penalty (BP): The BP is a component of the BLEU Score that penalizes translations that are too short compared to the reference translations. This is important because a model could achieve a high precision by generating very short outputs, which might not be meaningful or complete. The Brevity Penalty ensures that the generated text is of a reasonable length. The steps to calculate BP are as follows:

1. Define the lengths:
   - Let c be the length of the candidate translation (the generated text).
   - Let r be the length of the reference translation (the ground truth text).
2. Calculate the Brevity Penalty:

$$if c > r, then BP = 1(nopenalty)$$

$$if c < r, then BP = exp\left(r - \frac{r}{c}\right)$$

An n-gram is simply a sequence of n consecutive words or tokens in a sentence or code. For example:

For the sentence "The function is defined", its 2-g (bigrams) are:

- “The function”
- “function is”
- “is defined”

And for the same sentence, its 4-g would be the entire phrase, "The function is defined".

BLEU-4

BLEU-4 is a specific variant of the BLEU Score that considers up to 4-g. Higher-order n-grams (like 4-g) capture more context and structure, making metrics like BLEU-4 particularly valuable for tasks like evaluating code snippets, where understanding relationships between multiple tokens is crucial. In BLEU-4, N equals 4 and the range for n is from 1 to 4. So, the formula to calculate BLEU-4 is:

$$BLEU - 4 = BP.exp\left(\sum_{n=1}^{4} w_n \log p_n\right)$$

where:

- BP is the brevity penalty.
- p_n is the precision for n-grams (n = 1 to 4).
- w_n is the weight for n-grams (usually $w_n = 1/4$ for BLEU-4).

BLEU-DC.

BLEU-DC is a variant of the BLEU (Bilingual Evaluation Understudy) score, which is widely used for evaluating the quality of text generation. BLEU-DC incorporates the concept of Discounted Cumulative Gains (DCG) to account for the varying importance of different parts of the generated text. This is especially relevant in software vulnerability detection and mitigation, where some parts of the code (e.g., security-critical sections) are more important than others.

$$BLEU - DC = BP.exp\left(\sum_{n=1}^{N} w_n \log(\frac{DCG_n}{IDCG_n})\right)$$

where:

- BP is the brevity penalty.
- DCG_n is the Discounted Cumulative Gain for n-grams.
- $IDCG_n$ is the Ideal Discounted Cumulative Gain for n-grams.
- w_n is the weight for n-grams. Discounted Cumulative Gain (DCGnn):

DCG is a measure that evaluates the relevance of a sequence while giving higher weights to more relevant parts, especially those appearing earlier in the sequence. For n-grams, it is calculated as:

$$DCG_n = \sum_{i=1}^{P} \frac{Relevance\ Score_i}{\log_2(i+1)}$$

where:

- P: Total number of n-grams considered.
- $RelevanceScore_i$: A measure of how well the i-th n-gram matches the reference text.

Ideal Discounted Cumulative Gain ($IDCG_n$)**:**

IDCG is the maximum possible DCG value for n-grams, assuming perfect matching with the reference text. It is calculated similarly to DCG, but with the n-grams sorted in the order of their highest relevance scores. Comparing DCG_n to $IDCG_n$ helps measure how close the generated text is to ideal relevance.

Application of BLEU score and its variants in software vulnerability detection and mitigation:

- BLEU Score
  - Use: Evaluates the overall quality of the code generated by an LLM to fix vulnerabilities by comparing it to a reference solution.
- Example: If an LLM generates a patch for a vulnerability, the BLEU Score measures how closely the generated patch matches a reference patch in terms of word/token overlap.
- BLEU-4
  - Use: Provides a more nuanced evaluation by considering up to 4-g, capturing the local structure and semantic coherence of code snippets.
  - Example: When addressing a buffer overflow vulnerability, BLEU-4 can evaluate whether the generated code correctly handles multi-token constructs like boundary checks and array indexing.
- BLEU-DC
  - Use: Extends BLEU by incorporating the importance of different parts of the code, making it particularly useful when evaluating patches for vulnerabilities where some sections are more critical than others.
  - Example: In mitigating a SQL injection vulnerability, BLEU-DC can emphasize the correctness of input validation and sanitization over less critical sections.

11.3.4 CodeBLEU.

CodeBLEU is an extension of the widely used BLEU metric, specifically tailored to evaluate the quality of code generation tasks. It was first introduced by Ren et al. [132], and combines the traditional BLEU score with additional components that capture the syntactic and semantic aspects of code. In the context of software vulnerability detection and mitigation, CodeBLEU provides a more comprehensive evaluation of generated patches or code snippets by considering both their correctness and adherence to best coding practices [132].

The overall CodeBLEU score is calculated as a weighted sum of its four components. While the weighted n-gram match and the syntactic AST match are used to measure grammatical correctness, the n-gram match emphasizes the direct overlap of language patterns, ensuring that the generated code not only is grammatically correct but also closely mirrors the structure and wording of exemplary code. This dual focus helps ensure both fluency and fidelity to expected coding conventions. And finally, the semantic data-flow match is used to calculate logic correctness.

$$\begin{aligned} CodeBLEU = {} & \alpha.\text{n} - \text{gram}match(BLEU) + \beta.Weighted\ \text{n} \\ & - \text{gram Match} + \gamma.SyntacticASTMatch \\ & + \delta.SemanticData - FlowMatch \end{aligned}$$

where:

- α, β, γ, δ: Are weights assigned to each component (customizable based on the task).
- n-gram match (similar to BLEU): Measures how closely the generated code matches a reference code snippet and is calculated by standard BLEU formula.
- $Weighted$ n − gram Match : refers to a modified version of the traditional n-gram match (as used in BLEU) that incorporates weights for different n-grams. This means that not all n-grams are treated equally; instead, some n-grams are given more importance or weight depending on their significance in the context of code generation.
- AST Match: Evaluates the structural similarity between the generated and reference code, using Abstract Syntax Trees (ASTs).

$$Suntax\ Match = \frac{Number\ of\ matching\ AST\ nodes}{Total\ number\ of\ AST\ nodes\ in\ the\ refrence}$$

- Data Flow Match: Captures the alignment in how variables and data are used. It also assesses the data flow in the code, ensuring that variables and their usages are correctly handled.

$$DataFlowMatch = \frac{Number\ of\ matching\ data\ flow\ nodes}{Total\ number\ of\ data\ flow\ edges\ in\ the\ reference}$$

11.3.5 Pass@k

The Pass@k metric is a commonly used evaluation metric for assessing the performance of LLMs in tasks such as code generation, software vulnerability detection, and mitigation. It measures the likelihood that at least one of the top k generated solutions is correct. This metric is particularly useful in scenarios where the model generates multiple candidate solutions, and the goal is to determine how often at least one of these candidates is correct [32].

$$Pass@k = E_{\text{Problems}}\left[1 - \frac{\binom{(n-c)}{k}}{\binom{n}{k}}\right]$$

where:

- E_{Problems} : Expectation over Problems denotes the expected value (average) over all problems in the dataset. It means we are averaging the pass rate across multiple problems and we are averaging the pass rate across multiple problems.
- n: The total number of samples generated by the model for a given task. This represents all possible outputs considered during evaluation.
- c: The number of correct samples among n. A correct sample is one that passes all the unit tests or satisfies the evaluation criteria.
- k: The number of top samples considered for the metric. This defines how many of the generated samples are evaluated for correctness.
- $\binom{a}{b}$: The binomial coefficient represents the number of ways to choose b items from a items without regard to the order. It is calculated as: $\binom{a}{b} = \frac{a!}{b!(a-b)!}$.
- $1 - \frac{\binom{(n-c)}{k}}{\binom{n}{k}}$: Calculates the probability that at least one of the k samples is correct.
- $\binom{(n-c)}{k}$: Number of ways to choose k incorrect samples out of n – c.
- $\frac{\binom{(n-c)}{k}}{\binom{n}{k}}$: Probability of choosing k incorrect samples out of n.

Limitations of Pass@k

- Dependence on k Value: The choice of k heavily impacts the results. A high k may inflate the metric, making it easier for the model to achieve a "pass", even if most outputs are incorrect. Conversely, a low k may underestimate the model's capability, particularly if the correct solution is consistently ranked slightly outside the top k.
- Lack of Penalization for Incorrect Outputs: Pass@k does not account for the number or severity of errors in the generated outputs. For example, if a model produces $k - 1$ incorrect outputs and 1 correct output, the metric treats it as a success without considering the potential consequences of those incorrect outputs.
- Focus on Syntax Over Semantics: Pass@k often relies on strict criteria for correctness, such as exact syntax matching or passing unit tests. It does not evaluate the semantic correctness or quality of the generated code beyond these checks.
- Execution Requirement: Requires executing the generated code, which may not be feasible in all environments.

11.3.6 Exact match (EM)

The Exact Match (EM) metric is used to evaluate the performance of Large Language Models (LLMs) in text generation tasks, including in the domain of software vulnerability detection and mitigation. EM measures whether the model's generated output exactly matches the ground-truth or reference output (what percentage of correctly predicted answers, exactly match the ground-truth answer). It is particularly relevant for tasks where precision is critical, such as identifying specific vulnerabilities, generating mitigation scripts, or mapping vulnerabilities to specific patterns [135].

The EM metric is binary in nature, meaning that if the model's output matches the ground truth exactly, the result is 1; otherwise, it is 0 [135].

In the context of software vulnerability detection, EM can be used to assess the model's accuracy in tasks like:

- Generating exact identifiers of vulnerabilities.
- Producing code snippets that replicate a given solution.
- Providing precise mappings of vulnerabilities to mitigation techniques.

Limitations of EM.

- No Partial Credit: EM does not consider outputs that are partially correct and fail to exactly match the reference output.
- Over-Reliance on Format: Slight formatting differences can lead to an EM of 0, even if the core information is correct.
- Does Not Account for Synonyms: In textual outputs, semantically equivalent but lexically different responses will not be considered matches.

11.3.7 ROUGE/ROUGE-L

ROUGE (Recall-Oriented Understudy for Gisting Evaluation) metric is an automatic evaluation method for determining the quality of summaries by comparing them to ideal summaries created by humans. ROUGE measures the number of overlapping units such as n-grams, word sequences, and word pairs between the computer-generated summary and the reference summaries. There are four main ROUGE measures introduced: ROUGE-N, ROUGE-L, ROUGE-W, and ROUGE-S. These metrics have been used in various large-scale summarization evaluations, including the Document Understanding Conference (DUC) [136]. In the domain of software vulnerability detection and mitigation, ROUGE metrics can be used to assess how well a language model generates descriptions, code snippets, or mitigation strategies that match reference outputs. For the purpose of this study, we will focus on ROUGE-N and ROUGE-L, as they are the most widely used. For a more comprehensive examination of ROUGE, we suggest referring to the study conducted by Chin-Yew Lin [136].

ROUGE-N

ROUGE-N measures the overlap of n-grams between the generated text and the reference text. It is primarily recall-oriented, measuring how many n-grams in the reference are captured by the generated text. The formula for ROUGE-N is:

$$ROUGE-N = \frac{\sum_{S \in Reference\ Summaries} \sum_{n-gram \in S} Count_{match}(n-gram)}{\sum_{S \in Reference\ Summaries} \sum_{n-gram \in S} Count(n-gram)}$$

$Count_{match}(n-gram)$: The maximum number of n-grams in the generated output that also appear in the reference outputs.

$Count(n-gram)$: The total number of n-grams in the reference outputs.

$\sum S \in Reference\ Summaries$: Summation over all reference summaries.

ROUGE-L

ROUGE-L measures the similarity between two texts based on the Longest Common Subsequence (LCS). Unlike n-gram-based metrics, it captures in-sequence matches without requiring strict adjacency. ROUGE-L itself uses three formulas to compute Precision, Recall, and F1-score:

$$1.\ Recall(R_{LCS}) = \frac{LCS(X, Y)}{Lengthof X}$$

$$2.\ Precision(P_{LCS}) = \frac{LCS(X, Y)}{Lengthof Y}$$

$$3.\ F1-score(F_{LCS}) = \frac{(1+\beta^2).R_{LCS}.P_{LCS}}{R_{LCS}+\beta^2.P_{LCS}}$$

where:

- X is a reference summary sentence and Y is a candidate summary sentence
- LCS(X,Y) is the length of a longest common subsequence of X and Y
- β is a parameter in the ROUGE-L score that controls the relative importance of precision and recall. When beta is 1 the ROUGE-L score is simply the harmonic mean of precision and recall. However, if beta is greater than 1, then recall is given more weight than precision.

Limitations of ROUGE

- Focuses on surface-level matches, ignoring semantic accuracy.
- Overly sensitive to formatting and tokenization issues.
- Fails to assess logical or functional correctness in code.

These approach to evaluating LLM performance recognizes the nuanced nature of SE tasks, employing specific metrics tailored to the task at hand, whether it's classification, recommendation, or generation, to comprehensively measure model effectiveness and accuracy. While these metrics are useful, their limitations highlight the importance of using complementary metrics and qualitative evaluations to get a more comprehensive view of model performance. For example, combining Pass@k with metrics like Exact Match (EM), BLEU, or CodeBLEU and conducting manual error analysis can provide deeper insights into a model's strengths and weaknesses.

RQ9Answer: *Evaluation metrics for assessing LLMs in addressing software vulnerabilities and cyber threats include F1-score, Precision, Recall, Accuracy, and AUC-ROC for classification tasks. MRR, F1-score@k, Precision@k, Recall@k, Accuracy and Accuracy@k, for recommendation tasks, Code Coverage, Accuracy, BLEU, CodeBLEU, Exact Match, ROUGE/ROUGE-L and Pass@k for generation tasks.*

These metrics offer a comprehensive assessment of LLM performance across various tasks in software security.

Each of these evaluation metrics possesses distinct advantages and limitations, making them suitable for specific aspects of assessing LLM performance in addressing software vulnerabilities.

For instance, classification metrics such as Precision and Recall are particularly effective for quantifying the balance between detecting true positives and avoiding false positives, which is critical in identifying security vulnerabilities. On the other hand, generation metrics like BLEU and CodeBLEU focus on the syntactic and semantic accuracy of generated code, providing insights into how well the model produces contextually and functionally correct outputs. Metrics like Pass@k assess the likelihood of generating at least one correct solution within the top kk attempts, which is essential for evaluating models in code completion or repair tasks.

12 RQ10: what are the challenges of using LLMs in software vulnerability detection?

What are the limitations or challenges associated with using LLMs for software security, and how can they be mitigated? In this section we will try to shed some light to this question.

Prompt injection attacks: One of the significant security concerns related to LLMs, is prompt injection attacks [69, 84, 103]. Prompt injection attacks involve manipulating the input prompts given to a LLM in order to coax it into generating responses that are harmful, unauthorized, or disclose sensitive information. Malicious users may attempt to deceive the LLM into producing outputs that could compromise security, breach privacy, or cause harm in various ways. These attacks exploit vulnerabilities in the LLM's processing of prompts, potentially leading to disruptive outcomes or security breaches. This form of attack, especially potent within LLM-integrated applications, has recently been identified as the primary LLM-related threat by OWASP (Open Web Application Security Project) Foundation. Such manipulation can result in adverse consequences, such as providing incorrect guidance or unauthorized divulgence of confidential data [61, 137, 138].

Prompt injection is important because it highlights the need to address issues related to prompt abuse and prompt leak as we transition further into the era of LLMs. It is crucial to protect LLM-integrated applications from prompt injection threats, a fact recognized by many developers who have demonstrated increasing vigilance in the implementation of prompt protection systems and the quest for novel solutions [61, 137, 138]. There are two main categories of prompt injections:

Direct Prompt Injections: Commonly referred to as "jailbreaking," direct prompt injections involve altering or exposing the system prompt, often resulting in partial loss of intellectual property. This process may entail creating prompts with the specific goal of bypassing safety and moderation measures implemented by creators of LLMs [138].

Indirect Prompt Injections: indirect prompt injections occur when an LLM accepts input from external sources that can be manipulated by an attacker, such as websites or files. In this scenario, attackers can trick the LLM into interpreting its input as "commands" rather than "data" for processing, leading to unexpected behavior in LLM-based applications or compromising the security of the entire system [138].

Vulnerability detection challenges: Detecting vulnerabilities with LLMs can be problematic due to their ability to generate various alternative responses for the same issue. This diversity, beneficial in language processing and text generation, may pose challenges in pinpointing the actual vulnerability. The presence of multiple solutions, although advantageous in certain contexts, can complicate the identification of even the simplest software security vulnerabilities [17].

LLM code generation challenges: LLMs may struggle to generate accurate code when faced with multiple valid solutions, leading to functionally correct but contextually inappropriate code. They might perform well on specific tasks they were trained on but struggle with different tasks, languages, or domains outside of their training scope. Their performance can deteriorate significantly when inputs undergo semantic-preserving transformations [16, 131].

The study done in [59] analyzed 2033 programming tasks and 4066 ChatGPT-generated code snippets implemented in two popular programming languages: Java and Python, with 2,556 codes with quality issues to comprehensively assess AI-generated code quality and uncover performance-influencing factors. While ChatGPT3.5 could produce functional code for various tasks, their research findings uncovered a range of code quality concerns in ChatGPT3.5-generated code, spanning from compilation and runtime errors to incorrect outputs and issues with maintainability. This discovery underscores the critical need to tackle these issues diligently to safeguard the sustained efficacy of AI-driven code generation and uphold the standards of high-quality software systems.

Hallucinations: LLMs frequently generate false information, known as "hallucinations," which appear statistically plausible. Studies indicate that incorporating external knowledge and automated feedback mechanisms can mitigate these hallucinations [77, 89, 138].

Code quality challenges: Code quality issues pose significant concerns due to their potential to incur financial and reputational losses [37, 59, 80].

LLM deployment challenges: LLMs have become instrumental in software development, but they come with their own set of challenges. Their enormous size demands significant resources, making deployment challenging in resource-limited scenarios. They rely heavily on large and diverse datasets for training, and limited or biased data can lead to inaccurate predictions. There are also concerns about privacy leaks with Personally Identifiable Information (PII) in training data [16].

LLM evaluation: Existing evaluation metrics might not capture all aspects of model performance, such as interpretability, robustness, or sensitivity to certain errors. LLMs often lack interpretability and transparency in their decision-making processes, leading to uncertainty among developers. Concerns exist around the ownership of training data, derivative data, and potential adversarial attacks by seeding vulnerabilities into LLMs [16].

When comparing the performance of LLMs in software vulnerability and cybersecurity domains, a pertinent question arises regarding the fairness of such comparisons, given the variability in testing conditions across different scenarios. For instance, the robustness of the system could significantly impact LLM performance when both training and testing the model on the same dataset. Notably, not all researchers have access to supercomputers or high-performance workstations, which could affect the scalability and efficiency of model training and evaluation. Furthermore, changes made to the dataset over time or the selective use of specific portions of the dataset during testing may introduce additional variability, potentially influencing the outcomes of comparative analyses.

Insecure output management: Developers need to be careful since LLMs may produce harmful outputs. Insecure output management occurs when LLM outputs are not properly validated or sanitized before use, which can lead to security risks like Cross-Site Scripting and Cross-Site Request Forgery in web browsers. Furthermore, neglecting to validate LLM outputs may lead to downstream security exploits, including code execution that compromises systems and exposes data. Attackers can also exploit these outputs for privilege escalation and remote code execution on backend systems [70, 138].

Poisoning of educational data: Training data poisoning refers to the deliberate manipulation of the data used to train models with malicious intent. Adversaries insert deceptive or biased examples into the training dataset during pre-training or fine-tuning to influence the model's learning process. This can involve introducing backdoors, biases, or other vulnerabilities that compromise the security, performance, and reliability of the model [70, 138]. Manipulated training data can disrupt LLM models, leading to responses that may compromise security, accuracy, or ethical behavior.

Denial of service model: Training and running Large Models (LMs) requires substantial resources. An attacker can engage with LMs in a way that causes them to use resources excessively, thereby reducing the quality of service or even denying service to other users, and increasing compute costs. Attackers can create prompts that are computationally demanding in terms of context length or language patterns [70, 97, 138].

Supply chain vulnerabilities: The supply chain encompasses the complete journey from gathering data and training the model to its deployment. This process includes different elements like the training data, pre-trained models, and deployment infrastructure. Each element is susceptible to vulnerabilities: the crowd-sourced training data might be tainted, the pre-trained model could be compromised, or the third-party packages employed in LLM development could be insecure. Depending on the compromised components, services or datasets, they undermine system integrity, causing data breaches and system crashes [138].

13 Disclosure of sensitive information:

LLMs are initially trained on varied datasets containing snippets of real-world information. When generating responses, these models may unintentionally disclose sensitive details. For instance, conversational agents like OpenAI's ChatGPT and Google's Gemini gather user prompts during interactions to improve their performance. However, this approach poses a security and privacy risk, as the model might generate outputs inadvertently revealing confidential or private information. Additionally, by employing meticulously constructed prompts, an attacker could deliberately exploit this vulnerability to reveal or expose sensitive details. Failure to protect against disclosure of sensitive information in LLM outputs can lead to legal consequences or loss of competitive advantage [35, 138].

Insecure plugin design: Those LLM plugins lacking proper access control or input validation may lead to vulnerabilities such as SQL injection, and remote code execution. Frequently, these plugins accept user input as unrestricted text, making them susceptible to exploitation by attackers [138].

Too much agency: LLMs powered systems base their decisions on user prompts or inputs received from integrated components. Excessive autonomy or authorization granted to LLMs can introduce vulnerabilities susceptible to exploitation by malicious actors, potentially compromising the entire system. However, even without deliberate attacks, unintended user prompts, or wrong actions from connected systems can lead LLMs to generate misleading or unforeseen outputs, causing system malfunctions. As an illustrative

example, consider an LLM-based file summarizer that utilizes a third-party plugin for user file access. This plugin, beyond reading capabilities, might also possess functionalities for file modification and deletion. If a user encounters discrepancies in the LLM's summary, their attempt to report the error to the application could inadvertently trigger the LLM to modify or delete the original files, highlighting the potential for unintended consequences. Ultimately, giving LLMs unchecked autonomy to act can lead to unintended consequences that compromise reliability, privacy and trust [138].

Overreliance: The utilization of these models for source code generation presents a potential avenue for the inadvertent introduction of security vulnerabilities. These vulnerabilities can pose significant threats to the safety and security of applications and their users. The uncritical application of information or code generated by LLMs, without appropriate scrutiny, can lead to a cascade of negative consequences. These consequences include security breaches, the dissemination of misinformation, communication disruptions, legal issues, and reputational damage [138].

Model theft: The unauthorized copying or extraction of weights, parameters, or data from closed-source LLMs constitutes a form of intellectual property theft. This illicit practice can inflict significant economic losses on developers and damage brand reputation, ultimately jeopardizing a company's competitive edge. Perpetrators may exploit the purloined proprietary information for their own gain or utilize the stolen model for malicious purposes [138].

It's important to recognize both the opportunities and threats, presented by AI technologies. We must acknowledge the transformative potential of LLMs, but we must also accept that there are emerging risks. As LLMs continue to advance, there is a simultaneous need to understand and control the associated risks. The key to maximizing automation's benefits and enhancing functionalities through LLMs across various fields, lies in mitigating the inherent risks associated with AI systems.

To effectively reduce risks, increased awareness and preparation are crucial. SE play a vital role by analyzing past methods and incorporating those learnings into secure data and system mechanisms. New technologies are constantly emerging to monitor threats and mitigate risks. As pioneers in AI design and development, SE have a responsibility to identify new vulnerabilities and implement safeguards to protect users.

RQ10 Answer: *Challenges of using LLMs in cybersecurity tasks includes but not limited to: prompt injection attacks, vulnerability detection difficulties, code generation challenges, hallucinations, deployment hurdles, evaluation limitations, insecure output management, data poisoning risks, denial of service, supply chain vulnerabilities, sensitive information disclosure, insecure plugin design, unchecked autonomy, overconfidence, and model theft. These challenges underscore the importance of understanding and mitigating risks associated with LLMs to maximize their benefits while ensuring security and reliability in various applications.*

14 RQ11. How can we enhance LLMs effectiveness in software vulnerability detection?

Integration with Existing Tools: To enhance software vulnerability and cyber threat detection, LLMs can be effectively combined with complementary methods and tools. Static code analyzers excel at detecting known vulnerabilities and coding errors, while dynamic analysis tools like fuzzers uncover runtime vulnerabilities. Integrating LLMs into security testing frameworks and threat intelligence platforms allows for comprehensive vulnerability detection and proactive threat mitigation.

Human expertise remains crucial for domain-specific knowledge and validation of LLM-generated results. LLMs also can be combined with BMC to create automated code repair frameworks.

Continuous monitoring systems with LLM integration enable prompt incident detection and response. Collaborative platforms facilitate knowledge sharing and coordinated efforts in addressing security issues. By combining these approaches, organizations can achieve comprehensive coverage and improve the effectiveness of software vulnerability and cyber threat detection.

Larger and More Diverse Datasets: Training LLMs on a wider range of vulnerable and non-vulnerable code across various programming languages can enhance their ability to generalize and identify vulnerabilities in unseen code. However, given the challenges associated with training LLMs on extensive datasets, there are endeavors aimed at enhancing LLM accuracy by training them on smaller datasets. We believe that achieving this objective could substantially enhance the public usability of LLMs.

Focus on Specific Vulnerabilities: In some scenarios, training LLMs on datasets focused on specific types of vulnerabilities can improve their accuracy in detecting those vulnerabilities, but this method is most suitable for scenarios where a system is only prone to specific attacks.

Preprocessing Techniques: Techniques like constructing code gadgets can help LLMs distinguish actual vulnerabilities from irrelevant patterns.

Prompt Engineering: Craft effective prompts to guide LLMs towards vulnerability detection tasks, improving focus and accuracy [15].

Continuous Learning: Regularly update LLMs with new vulnerability and threat data to enhance detection of emerging security risks.

Efforts toward optimizing model size, enhancing data diversity and quality, improving code generation in ambiguous scenarios, enhancing generalizability, developing better evaluation methodologies, and focusing on interpretability and ethical use of LLMs. By combining these strategies, we can unlock the full potential of LLMs and make them even more powerful tools in the fight against software vulnerabilities and cyber threats.

RQ11 Answer: *To enhance LLMs' effectiveness in software vulnerability and cyber threat detection, organizations can integrate them with existing tools like static code analyzers and dynamic analysis tools. Additionally, training LLMs on larger and more diverse datasets, focusing on specific vulnerabilities, and employing preprocessing techniques can improve their accuracy. Crafting effective prompts, ensuring continuous learning by updating LLMs with new data, and optimizing model size and data quality are essential. Developing better evaluation methodologies and prioritizing interpretability and ethical use further unlock the full potential of LLMs in addressing software vulnerabilities and cyber threats.*

15 Conclusion

In this article we explored the potential of LLMs, examining their various applications across the SDLC, particularly in software vulnerability detection and mitigation.

Our analysis revealed that LLMs, when properly trained and integrated with existing tools, can significantly enhance software security. LLMs can outperform traditional static code analyzers in vulnerability detection and propose potential fixes, reducing the burden on developers. Furthermore, their ability to process and understand natural language empowers them to analyze textual information from logs and reports, aiding in threat identification and incident response. However, it is crucial to acknowledge the limitations of LLMs.

As LLM technology continues to evolve, so too will its applications in software security. By addressing current limitations and implementing best practices for training and integration, LLMs have the potential to revolutionize how we approach software security, enabling the development of more secure and robust systems.

16 Future research directions

Future research efforts should focus on addressing key challenges to enhance the robustness and reliability of LLMs in software vulnerability detection. LLMs can still be considered black-box systems, as their internal workings are not fully understood or transparent. This lack of transparency poses significant challenges in understanding how these models arrive at their conclusions. Improving explainability and transparency is therefore crucial for several reasons. Firstly, it can help build trust among users and stakeholders. Secondly, greater transparency can aid in identifying and mitigating biases that may be present in the models. Lastly, advancements in this area could lead to more robust and reliable AI systems, as developers would be better equipped to refine and improve the models based on a clearer understanding of their internal mechanisms. Incorporating techniques such as attention-based mechanisms [139], SHapley Additive exPlanations (SHAP) [140, 141], Local Interpretable Model-agnostic Explanations (LIME) [140, 142], and Diverse Counterfactual Explanations (DiCE) [140, 143] can significantly aid in interpreting LLM decisions, especially for complex vulnerability assessments.

Another essential area is domain adaptation. Research into strategies that enable LLMs to generalize across different programming languages and environments is necessary to prevent performance degradation when encountering unfamiliar tasks or semantic variations. This adaptation will ensure that LLMs maintain effectiveness across diverse coding scenarios.

Continuous learning also presents a promising direction. Implementing incremental learning mechanisms and integrating real-time threat intelligence can help LLMs stay current with evolving threats, reducing reliance on outdated information. Techniques such as Reinforcement Learning from Human Feedback (RLHF) can further enhance adaptability, making models more responsive to new vulnerabilities. Addressing security threats like prompt injection attacks is vital. Research focused on robust input validation, adversarial training, and prompt-filtering mechanisms can mitigate these risks. Establishing standardized defenses recognized by organizations like OWASP would further strengthen protection measures against both direct and indirect prompt injections.

To reduce hallucinations, leveraging external knowledge bases and feedback loops for cross-validation could be effective. Automated verification steps integrated into LLM workflows may improve accuracy and reliability, minimizing the impact of such errors. Deployment challenges also need attention, particularly optimizing LLM architectures for resource-constrained environments and exploring privacy-preserving techniques such as federated learning. Efforts to

secure the LLM supply chain and address data poisoning risks are essential for safe and reliable implementation.

Finally, enhancing evaluation metrics is critical. Developing comprehensive frameworks that assess not just accuracy but also interpretability, robustness, and fairness will provide a more holistic view of LLM performance in real-world applications. These advancements will help bridge the gap between theoretical research and practical deployment, making LLMs more dependable tools for software security. Collaborative efforts and continued research in these areas can drive significant progress and inspire further innovation.

17 Glossary

Adapters: Small, trainable modules integrated into pre-trained language models to enable efficient, task-specific fine-tuning without altering the core model parameters.

Application Programming Interface (API): A set of rules and tools that allows different software programs to communicate with each other.

Automatic Program Repair (APR): A technology that automatically fixes bugs in software without human intervention.

Bidirectional Encoder Representations from Transformers (BERT): A machine learning model used for understanding the context of words in natural language processing tasks.

Bilingual Evaluation Understudy (BLEU): A metric for evaluating the quality of machine translation by comparing it to human translations.

Bounded Model Checker (BMC): A tool that checks for bugs in software by exploring all possible states within a fixed range.

Dynamic Application Security Testing (DAST): A method of testing the security of running applications by simulating attacks.

Diverse Counterfactual Explanations (DiCE): A technique used in AI to provide alternative outputs while explaining decisions.

Encoder & Decoder: Encoders convert various inputs, such as text, audio, or images, into a condensed format known as a latent vector. Decoders do the opposite by transforming this condensed representation back into its original form or type.

Few-shot Learning: This machine learning method trains models to handle new tasks using only a small amount of labeled data. In LLMs, this technique is often applied by including a few examples within the input prompt, a process also called in-context learning.

Fine-tuning: This involves taking a pre-trained model and further training it on a more specific or smaller dataset to enhance its performance for a particular task or domain.

General Language Understanding Evaluation (GLUE): A benchmark suite for evaluating NLP models across various language understanding tasks, such as sentiment analysis and natural language inference.

Interactive Application Security Testing (IAST): A tool that analyzes an application's security by monitoring it during execution.

Long Short-Term Memory (LSTM): A type of neural network that is good at processing sequences, like time-series data or text.

Low-Rank Adaptation (LoRA): A fine-tuning method that adapts pre-trained models by adjusting small, low-rank matrices, thus preserving most of the original model's parameters.

MacroData Refinement (MDR): The process of organizing and improving large-scale data for better analysis or usage.

Mean Reciprocal Rank (MRR): A measure of how well a system ranks the correct answer in a list of results.

Multi-task learning (MTL): A machine learning technique that trains a model to perform multiple tasks simultaneously. The model uses information from related tasks to improve its generalization across all tasks.

Neural Machine Translation (NMT): An AI-based method for translating text between languages using neural networks.

National Vulnerability Database (NVD): A U.S. government database of information about software vulnerabilities.

Natural Language Processing (NLP): A field in AI that focuses on the interaction between computers and human language, encompassing tasks like language generation, translation, and sentiment analysis.

Parameters: These are the internal values, such as weights and biases, adjusted during an LLM's training. Weights influence the connections between neurons, while biases modify input values before activation. The total number of parameters indicates the LLM's complexity and capacity.

Parameter Efficient Fine-Tuning (PEFT): A technique to adapt large AI models using fewer resources by modifying only some parameters.

Prefix-Tuning: A type of soft prompt tuning that involves prepending a trainable sequence to the input at each model layer, enhancing adaptation for specific tasks.

Prompt: The input text or query given to an LLM to generate a relevant output or response.

Prompt Engineering: The practice of crafting and refining input prompts to achieve specific, desired responses from an LLM.

Program Dependency Graph (PDG): A diagram showing the relationships between parts of a program, useful for understanding its structure.

Quantization: The process of reducing the precision of model weights and activations, often from 32-bit to 8-bit or

lower, to decrease memory usage and improve computational efficiency.

Quantized LLMs: Large language models that have undergone quantization to reduce precision, enhancing efficiency in terms of memory and computation.

Reinforcement Learning from Human Feedback (RLHF): training technique for AI where humans guide the model's learning process by providing feedback.

Software Assurance Reference Dataset (SARD): A collection of test cases for evaluating software vulnerability detection tools.

Static Application Security Testing (SAST): A method of analyzing source code for vulnerabilities without running the application.

Software Development LifeCycle (SDLC): The process of planning, developing, testing, and deploying software.

SHapley Additive exPlanations (SHAP): method for explaining how machine learning models make decisions by analyzing feature contributions.

Structured Query Language (SQL): A language used to manage and manipulate databases.

Super General Language Understanding Evaluation (SuperGLUE): A benchmark for testing AI models' ability to understand and reason with language.

Token: The smallest unit of input or output that an LLM processes. Tokens can be individual words, subwords, characters, or segments of text and code.

Zero-shot Learning: A technique where models make accurate predictions about new tasks or categories they haven't encountered during training, relying solely on their prior knowledge without any labeled data for these tasks.]

Author contribution Mohammad conducted the primary research and data collection for this study, including the review and structuring of literature on the research questions. He were responsible for organizing and analyzing the research material, as well as contributing to the development of the article content. Farid served as the project supervisor, providing guidance and oversight throughout the research process. He played a critical role in reviewing and revising the manuscript to ensure coherence and quality in the overall structure and writing of the article.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

References

1. Li, H., Hao, Y., Zhai, Y., Qian, Z.: The Hitchhiker's guide to program analysis a journey with large language models (2023). arXiv:2308.00245v3
2. Sadowski, C., Yi, J.: How developers use data race detection tools. In: Proceedings of the 5th Workshop on Evaluation and Usability of Programming Languages and Tools, pp. 43–51 (2014)
3. Gadelha, M.Y.R., Steffinlongo, E., Cordeiro, L.C., Fischer, B., Nicole, D.A.: Smt-based refutation of spurious bug reports in the clang static analyzer. In: Atlee, J.M., Bultan, T., Whittle, J. (eds.) Proceedings of the 41st International Conference on Software Engineering: Companion Proceedings, ICSE 2019, Montreal, QC, Canada, May 25–31, 2019, pp. 11–14. IEEE (2019)
4. White, M., Tufano, M., Vendome, C., Poshyvanyk, D.: Deep learning code fragments for code clone detection. In: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, pp. 87–98 (2016)
5. Gupta, R., Pal, S., Kanade, A., Shevade, S.: Deepfix: fixing common c language errors by deep learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 31, p. 1 (2017)
6. Li, Y., Wang, S., Nguyen, T.N.: Dlfix: context-based code transformation learning for automated program repair. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, pp. 602–614 (2020)
7. White, M., Tufano, M., Martinez, M., Monperrus, M., Poshyvanyk, D.: Sorting and transforming program repair ingredients via deep learning code similarities. In: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering (SANER), pp. 479–490. IEEE (2019)
8. Jiang, N., Lutellier, T., Tan, L.: Cure: code-aware neural machine translation for automatic program repair. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), pp. 1161–1173 (2021)
9. Li, Y., Wang, S., Nguyen, T.N.: Dear: a novel deep learning-based approach for automated program repair. In: Proceedings of the 44th International Conference on Software Engineering, pp. 511–523 (2022)
10. Saha, D., Tarek, S., Yahyaei, K., Saha, S.K., Zhou, J., Tehranipoor, M., Farahmandi, F.: LLM for SoC security a paradigm shift (2023). arXiv:2310.06046v1
11. Ferrag, M.A., et al.: Revolutionizing cyber threat detection with large language models: a privacy-preserving BERT-based lightweight model for IoT/IIoT devices. IEEE Access **12**, 23733–23750 (2024). https://doi.org/10.1109/ACCESS.2024.3363469
12. Noever, D.: Can large language models find and fix vulnerable software? (2023). arXiv:2308.10345v1
13. Omar, M., Shiaeles, S.: "VulDetect: a novel technique for detecting software vulnerabilities using language models. In: 2023 IEEE International Conference on Cyber Security and Resilience (CSR), Venice, Italy, pp. 105–110 (2023). https://doi.org/10.1109/CSR57506.2023.10224924
14. Verizon 2023 Data Breach Investigations Report: https://www.verizon.com/business/resources/T3b1/reports/2023-data-breach-investigations-report-dbir.pdf. Accessed 2 Oct 2024
15. Zhang, C., Liu, H., Zeng, J., Yang, K., Li, Y., Li, H.: Prompt-enhanced software vulnerability detection using ChatGPT (2024). arXiv:2308.12697v2
16. Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Wang, H.: Large language models for software engineering: a systematic literature review (2024). arXiv:2308.10620v4
17. Yiannis, C., Norbert, T., Ridhi, J., Youcheng, S., Mohamed Amine, F., Lucas, C.: A New era in software security: towards self-healing software via large language models and formal verification (2023). https://doi.org/10.48550/arXiv.2305.14752
18. OpenAI. GPT-4 Technical Report. 2023. arXiv: 2303.08774 [cs.CL]
19. Devlin, J., Ming-Wei, C., Lee, K., Toutanova, K.: Bert: pre-training of deep bidirectional transformers for language understanding (2019). arXiv.org, arXiv:1810.04805v2

20. Almazrouei, E., Alobeidli, H., Alshamsi, A., Cappelli, A., Cojocaru, R., et al.: The Falcon series of open language models (2023). arXiv:2311.16867
21. Penedo, G., Malartic, Q., Hesslow, D., Cojocaru, R., Cappelli, A., Alobeidli, H., Pannier, B., Almazrouei, W., Launay, J.: The RefinedWeb dataset for Falcon LLM: outperforming curated Corpora with Web Data, and Web Data Only (2023). arXiv:2306.01116 [cs.CL]
22. Bommasani, R., et al.: On the opportunities and risks of foundation models. Eng, arXiv.org (2021). Available: https://crfm.stanford.edu/assets/report.pdf
23. Sakaoglu, S.S.: KARTAL: web application vulnerability hunting using large language models: novel method for detecting logical vulnerabilities in web applications with finetuned large language models. KTH Royal Institute of Technology, Dissertation (2023)
24. Purba, M., Ghosh, A., Radford, B., Chu, B.: Software vulnerability detection using large language models. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW), Florence, Italy, pp. 112–119 (2023). https://doi.org/10.1109/ISSREW60843.2023.00058
25. Boyko, J., Cohen, J., Fox, N., Veiga, M.H., Li, J.I.H., Liu, J., Modenesi, B., Rauch, A.H., Reid, K.N., Tribedi, S., Visheratina, A., Xie, X.: An interdisciplinary outlook on large language models for scientific research. https://doi.org/10.48550/arXiv.2311.04929
26. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. https://doi.org/10.48550/arXiv.1706.03762
27. Zhao, W.X., Zhou, K., Li, J., Tang, T., et al.: A survey of large language models (2023). arXiv:2303.18223v13
28. Xia, C.S., Wei, Y., Zhang, L.: Practical program repair in the era of large pre-trained language models (2022). arXiv:2210.14179v1
29. Zheng, Z., Ning, K., Chen, J., Wang, Y., Chen, W., Guo, L., Wang, W.: Towards an understanding of large language models in software engineering tasks (2023). arXiv:2308.11396v1
30. Pan, S., Luo, L., Wang, Y., Chen, C., Wang, J., Wu, X.: Unifying large language models and knowledge graphs: a roadmap. IEEE Trans. Knowl. Data Eng. **36**(7), 3580–3599 (2024). https://doi.org/10.1109/TKDE.2024.3352100
31. Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al.: Codebert: a pre-trained model for programming and natural languages. Find. Assoc. Comput. Linguist. **2020**, 1536–1547 (2020)
32. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.D.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code (2021). arXiv:2107.03374
33. Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S.K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., Zhou, M.: GraphCodeBERT-Pre-training code representations with data flow (2021). arXiv:2009.08366 [cs.SE]
34. Huang, K., et al.: An empirical study on fine-tuning large language models of code for automated program repair. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), Luxembourg, Luxembourg, pp. 1162–1174 (2023). https://doi.org/10.1109/ASE56229.2023.00181
35. Wang, S., Chollak, D., Movshovitz-Attias, D., Tan, L.: Bugram: bug detection with n-gram language models. In: ASE, pp. 708–719 (2016)
36. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J.: Exploring the limits of transfer learning with a unified text-to-text transformer (2020). arXiv:1910.10683 [cs.LG]
37. Yetiştiren, B., Özsoy, I., Ayerdem, M., Tüzün, E.: Evaluating the code quality of AI-assisted code generation tools: an empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT (2023). arXiv:2304.10778 [cs.SE]
38. Karl, T., Hayretdin, B.: Harnessing large language models for software vulnerability detection: a comprehensive benchmarking study (2024). arXiv:2405.15614 [cs.CR]
39. McGraw, G.: Automated code review tools for security. In: Computer 41.12, pp. 108–111 (2008). https://doi.org/10.1109/MC.2008.514
40. Ferda, O.S., Banu, G.: Holistic web application security visualization for multi-project and multi-phase dynamic application security test results. IEEE Access. (2021). https://doi.org/10.1109/ACCESS.2021.3057044
41. Azar, K.Z., et al.: Fuzz, penetration, and AI testing for SoC security verification: challenges and solutions. In: Cryptology ePrint Archive (2022)
42. Mateo Tudela, F., Bermejo Higuera, J.-R., Bermejo Higuera, J., Sicilia Montalvo, J.-A., Argyros, M.I.: On combining static, dynamic and interactive analysis security testing tools to improve owasp top ten security vulnerability detection in web applications. Eng. Appl. Sci. **10**(24), 9119 (2020)
43. Tahmid, A., Tawhid, Md., Ahad, N., Ahmed, S., Sakib, K.: Code sniffer: a risk based smell detection framework to enhance code quality using static code analysis. Int. J. Softw. Eng. Technol. Appl. **2**, 41 (2017). https://doi.org/10.1504/IJSETA.2017.086988
44. Foo, D., Yeo, J., Xiao, H., Sharma, A.: The dynamics of software composition analysis (2019). arXiv:1909.00973 [cs.SE]
45. Nasif, I., Seaver, T., Laurie, W.: A comparative study of vulnerability reporting by software composition analysis tools. In: Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (ESEM '21), Article 5, pp. 1–11. Association for Computing Machinery, New York (2021). https://doi.org/10.1145/3475716.3475769
46. Zhao, L., Chen, S., Xu, Z., Liu, C., Zhang, L., Wu, J., Sun, J., Liu, Y.: Software composition analysis for vulnerability detection: an empirical study on java projects. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023), pp. 960–972. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3611643.3616299
47. Čisar, P., Čisar, S.M.: The framework of runtime application self-protection technology. In: 2016 IEEE 17th International Symposium on Computational Intelligence and Informatics (CINTI), Budapest, Hungary, pp. 000081–000086 (2016). https://doi.org/10.1109/CINTI.2016.7846383
48. Alexander, F.: Runtime application self-protection (RASP), investigation of the effectiveness of a RASP solution in protecting known vulnerable target applications. SANS Whitepaper. 30 April 2019. https://www.sans.org/white-papers/38950/
49. Huaming, C., Muhammad, A.B.: Security for machine learning-based software systems: a survey of threats, practices and challenges (2022). arXiv:2201.04736v2 [cs.CR]
50. Li, Z., Dutta, S., Naik, M.: LLM-assisted static analysis for detecting security vulnerabilities (2024). arXiv:2405.17238 [cs.CR]
51. Goseva-Popstojanova, K., Perhinschi, A.: On the capability of static code analysis to detect security vulnerabilities. Inf. Softw. Technol. **68**, 18–33 (2015). https://doi.org/10.1016/j.infsof.2015.08.002
52. Kulenovic, M., Donko, D.: A survey of static code analysis methods for security vulnerabilities detection. In: 2014 37th International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), Opatija, Croatia, pp. 1381–1386 (2014). https://doi.org/10.1109/MIPRO.2014.6859783

53. Puspaningrum, A., Al Hilmi, M.A., Muhamad Mustamiin, D., Ginanjar, M.I.: Vulnerable source code detection using SonarCloud code analysis (2023). arXiv:2307.02446 [cs.CY]
54. Fan, A., et al.: Large language models for software engineering: survey and open problems. In: 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), Melbourne, Australia, pp. 31–53 (2023). https://doi.org/10.1109/ICSE-FoSE59343.2023.00008
55. Wilhjelm, C., Younis, A.A.: A threat analysis methodology for security requirements elicitation in machine learning based systems. In: 2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C), pp. 426–433. IEEE (2020)
56. Cheng, G.J., Cleland-Huang, J.: Semantically enhanced software traceability using deep learning techniques. In: Proceedings of the 39th International Conference on Software Engineering, Buenos Aires, Argentina, pp. 3–14 (2017)
57. White, J., Hays, S., Fu, Q., Spencer-Smith, J., Schmidt, D.C.: ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design (2023). arXiv:2303.07839
58. He, J., Vechev, M.: Large language models for code: security hardening and adversarial testing. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23), pp. 1865–1879. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3576915.3623175
59. Liu, Y., Ratnadira Widyasari, T.L.C., Tantithamthavorn, C., Li, L., Le, X.B.D., Lo, D.: Refining ChatGPT-generated code characterizing and mitigating code quality issues (2023). arXiv:2307.12596v2
60. Imgrund, E., Ganz, T., Härterich, M., Pirch, L., Risse, N., Rieck, K.: Broken promises: measuring confounding effects in learning-based vulnerability discovery. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), pp. 149–160. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3605764.3623915
61. Pedro, R., Castro, D., Carreira, P., Santos, N.: From prompt injections to SQL injection attacks: how protected is your LLM-integrated web application? (2023). arXiv:2308.01990v3
62. Giacomo, L., Manuel, S., Volker, H., Tommaso, C.: Analyzing declarative deployment code with large language models, pp. 289–296 (2023). https://doi.org/10.5220/0011991200003488
63. Lemieux, C., Inala, J.P., Lahiri, S.K., Sen, S.: Codamosa: escaping coverage plateaus in test generation with pre-trained large language models. In: ICSE (2023)
64. Inala, L.J.P., Lahiri, S.K., Sen, S.: Codamosa: escaping coverage plateaus in test generation with pre-trained large language models. In: International Conference on Software Engineering (ICSE) (2023)
65. Alhamed, M., Storer, T.: Evaluation of context-aware language models and experts for effort estimation of software maintenance issues. In 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), pp. 129–138. IEEE (2022)
66. Szabó, Z., Bilicki, V.: A new approach to web application security: utilizing GPT language models for source code inspection. Future Internet **15**, 326 (2023). https://doi.org/10.3390/fi15100326
67. Chen, Y., Ding, Z., Alowain, L., Chen, X., Wagner, D.: DiverseVul: a new vulnerable source code dataset for deep learning based vulnerability detection. In: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID '23), pp. 654–668. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3607199.3607242
68. Sean Patrick, M.: Applications of transfer learning from malicious to vulnerable binaries. Graduate Student Theses, Dissertations, & Professional Papers, p. 12187 (2023). https://scholarworks.umt.edu/etd/12187
69. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection (2023). arXiv:2302.12173v2 [cs.CR]
70. Mohamed Amine, F., Fatima, A., Ammar, B., Bilel, C., Abdechakour, M., Norbert, T.: Generative AI and large language models for cyber security: all insights you need (2024). https://doi.org/10.48550/arXiv.2405.12750
71. Benjamin, K.Y.: Building resilient SMEs: harnessing large language models for cyber security in Australia (2023). arXiv preprint arXiv:2306.02612v1
72. Zhou, X., Verma, R.M.: Vulnerability detection via multimodal learning: datasets and analysis. In: Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security, pp. 1225–1227 (2022)
73. Mamede, C., Pinconschi, E., Abreu, R., Campos, J.: Exploring transformers for multi-label classification of java vulnerabilities. In: 2022 IEEE 22nd International Conference on Software Quality, Reliability and Security (QRS), Guangzhou, China, pp. 43–52 (2022). https://doi.org/10.1109/QRS57517.2022.00015
74. Thapa, C., Jang, S.I., Ahmed, M.E., Camtepe, S., Pieprzyk, S., Nepal, S.: Transformer-based language models for software vulnerability detection. In: Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC '22), pp. 481–496. Association for Computing Machinery, New York (2022). https://doi.org/10.1145/3564625.356798
75. Ferrag, M.A., Battah, A., Tihanyi, N., Debbah, M., Lestable, T., Cordeiro, L.C.: SecureFalcon the next cyber reasoning system for cyber security (2023). arXiv:2307.06616v1
76. Tihanyi, N., Bisztray, T., Jain, R., Ferrag, M.A., Cordeiro, L.C., Mavroeidis, C.: The FormAI dataset: generative AI in software security through the lens of formal verification. In: Proceedings of the 19th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE 2023), pp. 33–43. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3617555.3617874
77. Steenhoek, B., Rahman, M.M., Roy, M.K., Alam, M.S., Barr, E.T., Le, W.: A comprehensive study of the capabilities of large language models for vulnerability detection (2024). arXiv:2403.17218v1
78. Pearce, H., Tan, B., Ahmad, B., Karri, R., Dolan-Gavitt, B.: Examining zero-shot vulnerability repair with large language models (2022). arXiv:2112.02125v3
79. Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R.: Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In: IEEE S&P (2022). https://doi.org/10.1109/SP46214.2022.9833571
80. Khan, M.F.A., Ramsdell, M., Falor, E., Karimi, H.: Assessing the promise and pitfalls of ChatGPT for automated code generation (2023). arXiv:2311.02640v1
81. Motlagh, F.M., Christoph, M., Feng, C., Pejman, N., Mehrdad, H., Mehryar, M.: Large language models in cybersecurity: state-of-the-art (2024). https://doi.org/10.48550/arXiv.2402.00891
82. Heiding, F., Schneier, B., Vishwanath, A., Bernstein, J., Park, P.S.: Devising and detecting phishing large language models versus smaller human models (2023). arXiv:2308.12287v2
83. Koide, T., Fukushi, N., Nakano, H., Chiba, D.: Detecting phishing sites using ChatGPT (2024). arXiv:2306.05816v2
84. Apruzzese, G., Anderson, H.S., Dambra, S., Freeman, D., Pierazzi, F., Roundy, K.V.: Real attackers don't compute gradients: bridging the gap between adversarial ml research and practice. In: SaTML (2023)

85. Tang, L., Mahmoud, Q.H.: A survey of machine learning based solutions for phishing website detection. Mach. Learn. Knowl. Extract. **3**(3), 672–694 (2021)
86. Weng, G., Andrzejak, A.: Automatic bug fixing via deliberate problem solving with large language models. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW), Florence, Italy, pp. 34–36 (2023). https://doi.org/10.1109/ISSREW60843.2023.00040
87. Huang, K., Yang, S., Sun, H., Sun, C., Li, X., Zhang, Y.: Repairing security vulnerabilities using pre-trained programming language models. In: 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W), Baltimore, pp. 111–116 (2022) https://doi.org/10.1109/DSN-W54100.2022.00027
88. Zhang, Q., Fang, C., Xie, Y., Ma, Y.S., Sun, W., Yang, Y., Chen, Z.: A systematic literature review on large language models for automated program repair (2024). arXiv:2405.01466 [cs.SE]
89. Happe, A., Cito, J.: Getting pwn'd by AI: penetration testing with large language models. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023), pp. 2082–2086. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3611643.3613083
90. Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y.: GPTScan: detecting logic vulnerabilities in smart contracts by combining GPT with program analysis (2023). arXiv:2308.03314v2
91. Das, S.S., Dutta, A., Purohit, S., Serra, E., Halappanavar, M., Pothen, A.: Towards automatic mapping of vulnerabilities to attack patterns using large language models. In: 2022 IEEE International Symposium on Technologies for Homeland Security (HST), Boston, pp. 1–7 (2022). https://doi.org/10.1109/HST56032.2022.10025459
92. Cheshkov, A., Zadorozhny, P., Levichev, R.: Evaluation of ChatGPT model for vulnerability detection. arXiv:2304.07232v1
93. Ullah, S., Han, M., Pujar, S., Pearce, H., Coskun, A., Stringhini, G.: LLMs cannot reliably identify and reason about security vulnerabilities (Yet?): a comprehensive evaluation, framework, and benchmarks (2024). arXiv:2312.12575v2
94. Guo, Y., Patsakis, C., Hu, Q., Tang, Q., Casino, F.: Outside the comfort zone: analysing LLM capabilities in software vulnerability detection (2024). arXiv:2408.16400 [cs.CR]
95. Zhang, X., Zhang, F., Zhao, B., Zhou, B., Xiao, B.: VulD-transformer: source code vulnerability detection via transformer. In: Proceedings of the 14th Asia-Pacific Symposium on Internetware (Internetware '23), pp. 185–193. Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3609437.3609451
96. Zhou, X., Cao, S., Sun, X., Lo, D.: Large language model for vulnerability detection and repair: literature review and the road ahead (2024). arXiv:2404.02525v3 [cs.SE]
97. Yang, A.Z.H., Tian, H., Ye, H., Martins, R., Goues, C.L.: Security vulnerability detection with multitask self-instructed fine-tuning of large language models (2024). arXiv:2406.05892 [cs.CR]
98. Shestov, A., Levichev, R., Mussabayev, R., Maslov, E., Cheshkov, A., Zadorozhny, P.: Finetuning large language models for vulnerability detection (2024). arXiv:2401.17010v5 [cs.CR]
99. deFitero-Dominguez, D., Garcia-Lopez, E., Garcia-Cabot, A., Martinez-Herraiz, J.J.: Enhanced automated code vulnerability repair using large language models. Eng. Appl. Artif. Intell. **138**, 109291 (2024). https://doi.org/10.1016/j.engappai.2024.109291
100. Zhai, Y., Tong, S., Li, X., Cai, M., Qu, Q., Lee, Y.J., Ma, Y.: Investigating the catastrophic forgetting in multimodal large language models (2023). arXiv:2309.10313v4 [cs.CL]
101. Zhang, J., Bu, H., Wen, H., Liu, Y., Fei, H., Xi, R., Li, L., Yang, Y., Zhu, H., Meng, D.: When LLMs meet cybersecurity: a systematic literature review (2024). arXiv:2405.03644v2 [cs.CR]
102. Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: Lora: low-rank adaptation of large language models. In: The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25–29, 2022. OpenReview.net (2022)
103. Perez, F., Ribeiro, I.: Ignore previous prompt: attack techniques for language models. In: NeurIPS ML safety workshop (2022)
104. Ruder, S.: An overview of multi-task learning in deep neural networks (2017). arXiv:1706.05098v1 [cs.LG]
105. Zhang, Y., Yang, Q.: A survey on multi-task learning. IEEE Trans. Knowl. Data Eng. **34**(12), 5586–5609 (2022). https://doi.org/10.1109/TKDE.2021.3070203
106. Chen, S., Zhang, Y., Yang, Q.: Multi-task learning in natural language processing: an overview (2024). arXiv:2109.09138v2[cs.AI]
107. Ding, N., Qin, Y., Yang, G., et al.: Parameter-efficient fine-tuning of large-scale pre-trained language models. Nat. Mach. Intell. **5**, 220–235 (2023). https://doi.org/10.1038/s42256-023-00626-4
108. Parthasarathy, V.B., Zafar, A., Khan, A., Shahid, A.: The ultimate guide to fine-tuning LLMs from basics to breakthroughs: an exhaustive review of technologies, research, best practices, applied research challenges and opportunities (2024). arXiv:2408.13296v1[cs.LG]
109. Li, X.L., Liang, P.: Prefix-tuning: optimizing continuous prompts for generation. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing, vol 1: Long Papers, pp. 4582–4597. Association for Computational Linguistics (2021)
110. Husain, H., Wu, H.H., Gazit, T., Allamanis, M., Brockschmidt, M.: CodeSearchNet challenge: evaluating the state of semantic code search (2020). arXiv:1909.09436v3 [cs.LG]
111. Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, A., Presser, S., Leahy, C.: The pile: an 800GB dataset of diverse text for language modeling (2020). arXiv:2101.00027v1 [cs.CL]
112. Kocetkov, D., Li, R., Allal, L.B., Li, J., Mou, C., Ferrandis, C.M., Jernite, Y., Mitchell, M., Hughes, S., Wolf, T., Bahdanau, D., von Werra, L., de Vries, H.: The stack: 3 TB of permissively licensed source code (2022). arXiv:2211.15533v1 [cs.CL]
113. Lozhkov, A., Li, R., Allal, L.B., et al.: StarCoder 2 and the Stack v2: the next generation (2024). arXiv:2402.19173v1 [cs.SE]
114. Laurençon, H., Saulnier, L., Wang, T., Akiki, C., et al.: The BigScience ROOTS corpus: A 1.6TB composite multilingual dataset (2023). arXiv:2303.03915v1 [cs.CL]
115. Abadji, J., Suarez, P.O., Romary, L., Sagot, B.: Towards a cleaner document-oriented multilingual crawled corpus (2022). arXiv:2201.06642v1 [cs.CL]
116. Barbaresi, A.: Trafilatura: A web scraping library and command-line tool for text discovery and extraction. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations, pp. 122–131. Association for Computational Linguistics (2021)
117. Wenzek, G., Lachaux, M.-A., Conneau, A., Chaudhary, V., Guzman, F., Joulin, A., Grave, E.: Ccnet: extracting high quality monolingual datasets from web crawl data. In: Proceedings of the 12th Language Resources and Evaluation Conference, pp. 4003–4012 (2020)
118. Mitre cve datagbase. https://cve.mitre.org/. Accessed 2 Oct 2024
119. MITRE.: CWE-Common Weakness Enumeration. MITRE, https://cwe.mitre.org/data/downloads.html. Accessed 7 Oct 2024

120. Official CAPEC website: https://capec.mitre.org/data/downloads. Accessed 7 Oct 2024
121. Bhandari, G.P., Naseer, A., Moonen, L.: CVEfixes: automated collection of vulnerabilities and their fixes from open-source software. In: PROMISE (2021). arXiv:2107.08760v1
122. Software assurance reference dataset (2024). https://samate.nist.gov/SARD/. Accessed 10 Oct 2024
123. Li, Z., et al.: SySeVR: a framework for using deep learning to detect software vulnerabilities. IEEE Trans. Depend. Secure Comput. **19**(04), 2244–2258 (2022). https://doi.org/10.1109/TDSC.2021.3051525
124. Zhou, Y., Liu, S., Siow, J., Xiaoning, D., Liu, Y.: Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. Adv. Neural. Inf. Process. Syst. **32**, 68 (2019)
125. Fan, J., Li, Y., Wang, S., Nguyen, T.N.: A C/C++ code vulnerability dataset with code changes and CVE summaries. In: Proceedings of the 17th International Conference on Mining Software Repositories (Seoul, Republic of Korea) (MSR '20), pp. 508–512. Association for Computing Machinery, New York (2020). https://doi.org/10.1145/3379597.3387501
126. Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., Chen, Y.: Vulnerability detection with code language models: how far are we? (2024). arXiv preprint arXiv:2403.18624v2 [cs.SE]
127. Fu, M., Tantithamthavorn, C.: Linevul: a transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories, MSR '22, pp. 608–620. Association for Computing Machinery, New York (2022)s
128. Yutaka, S.: The truth of the F-measure. Teach. Tutor Mater. (2007)
129. Fawcett, T.: An introduction to ROC analysis. Pattern. Recognit. Lett. **27**(8), 861–874 (2006). https://doi.org/10.1016/j.patrec.2005.10.010
130. Tsatsaronis, G., Balikas, G., Malakasiotis, P., Partalas, I., Zschunke, M., Alvers, M.R., Weissenborn, D., Krithara, A., Petridis, S., Polychronopoulos, D., et al.: An overview of the bioasq large-scale biomedical semantic indexing and question answering competition. BMC Bioinf. **16**, 138 (2015). https://doi.org/10.1186/s12859-015-0564-6
131. Asare, O.: Security evaluations of GitHub's Copilot. UWSpace (2023). http://hdl.handle.net/10012/19675
132. Ren, S., Guo, D., Lu, S., Zhou, L., Liu, S., Tang, D., Sundaresan, N., Zhou, M., Blanco, A., Ma, S.: Codebleu: a method for automatic evaluation of code synthesis (2020). arXiv:2009.10297v2 [cs.SE]
133. Chen, B., Song, J., Xu, P., Hu, X., Jiang, Z.M.J.: An automated approach to estimating code coverage measures via execution logs. In: 2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE), Montpellier, France, pp. 305–316 (2018). https://doi.org/10.1145/3238147.3238214
134. Papineni, K., Roukos, S., Ward, T., Zhu, W.-J.: BLEU: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pp. 311–318 (2002)
135. Farea, A., Yang, Z., Duong, K., Perera, N., Emmert-Streib, F.: Evaluation of question answering systems: complexity of judging a natural language (2022). arXiv:2209.12617v1 [cs.CL]
136. Lin, C.Y.: ROUGE: A package for automatic evaluation of summaries. In: Text Summarization Branches Out, pp. 74–81. Association for Computational Linguistics, Barcelona (2004)
137. Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., Liu, Y.: Prompt injection attack against LLM-integrated applications (2024). arXiv:2306.05499v2
138. Pankajakshan, R., Biswal, S., Govindarajulu, Y., Gressel, G.: Mapping LLM security landscapes: a comprehensive stakeholder risk assessment proposal (2024). arXiv:2403.13309v1
139. Parra, G., Selvera, L., Khoury, J., Irizarry, H., Bou-Harb, E., Rad, P.: Interpretable federated transformer log learning for cloud threat forensics. In: Proceedings of the Network and Distributed Systems Security (NDSS) Symposium (2022)
140. Belle, V., Papantonis, I.: Principles and practice of explainable machine learning (2020). arXiv:2009.11698v1 [cs.LG]
141. Lundberg, S., Lee. S.I.: A unified approach to interpreting model predictions (2017). arXiv:1705.07874v2 [cs.AI]
142. Ribeiro, M.T., Singh, S., Guestrin, C.: Why should I trust you?: Explaining the predictions of any classifier (2016). arXiv:1602.04938v3 [cs.LG]
143. Altmeyer, P., van Deursen, A., Liem, C.C.S.: Explaining black-box models through counterfactuals (2023). arXiv:2308.07198v1 [cs.LG]

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.